Agenti AI in azienda: dal chatbot alla delega governata

Riccardo Bovetti

Who am I? | LinkedIn | All you need is thought

Complessità & Sistemi di Senso

Ottobre 2026

Indice

1 Parte I. Dal chatbot all'agente: le coordinate concettuali 4

1.1 Che cos'è, davvero, un agente AI 4
1.2 Il continuum dell'autonomia 5
1.3 Che cosa si delega: il compito, non il mestiere 7
1.4 Anatomia di un agente: modello, harness, contesto, ambiente 8

2 Parte II. L'infrastruttura comune: la logica di base indipendente dalle piattaforme 10

2.1 Il ciclo agentico e il tool use 10
2.2 Una mappa terminologica: prompt, istruzioni, skill, tool, connettore, MCP, plugin 14
2.3 Che cosa c'è sotto il connettore 21
2.4 Connettori e Model Context Protocol 22
2.5 I file di configurazione in Markdown (od in altro formato testo) 28
2.6 Contesto, memoria e RAG 30
2.7 Il costo del contesto: che cosa è sempre acceso e che cosa si carica a richiesta . . 31
2.8 Sistemi multi agente: coordinamento, non solo parallelismo 33
2.9 La verifica dell'esito: perché "ho fatto X" non basta 37
2.10 Il mandato: come si formula una delega verificabile 39
2.11 Guardrail, sandbox, permessi e hook 41
2.12 L'esecuzione non presidiata 45

3 Parte III. Il fronte Anthropic 46

3.1 Claude Code e il Claude Agent SDK 47
3.2 Dove vivono le istruzioni, e in quale ordine si caricano 47
3.3 Le skill come standard aperto e i plugin come distribuzione condivisa 50
3.4 Dove vivono le skill, e come si sovrappongono 50
3.5 Dove vivono i permessi, e chi vince 52
3.6 I sottoagenti di Claude Code: che cosa si può configurare 53
3.7 Cowork e i connettori MCP nativi 54
3.8 I tre livelli di costruzione: Messages API, Agent SDK, Claude Managed Agents . 54

4 Parte IV. Il fronte OpenAI 55

4.1 Orientarsi nel catalogo: Chat, ChatGPT Work, Workspace Agents e Codex . . . 55
4.2 I tre livelli di costruzione: Responses API, Agents SDK, Agents API 56
4.3 Codex e il consolidamento del browsing agentico 56

5 Parte V. Il fronte Microsoft 56

5.1 Che cosa chiama "agente" Microsoft: una scala, non un oggetto unico 57
5.2 L'agente come identità aziendale: Entra Agent ID 58

5.3 Agent 365: il control panel (si lo so si chiama control plane, ma non mi piace) della flotta di agenti . . . 59
5.4 Gli agenti dichiarativi: il file di configurazione secondo Microsoft . . . 59
5.5 Copilot Studio: orchestrazione generativa, trigger e agent flow . . . 60
5.6 MCP, connettori e agenti dichiarativi: Copilot Studio e Microsoft Foundry Agent Service . . . 62
5.7 Il Microsoft Agent Framework per lo sviluppo su misura . . . 62
5.8 GitHub Copilot Agent Mode: il fronte coding . . . 62
5.9 Copilot Cowork: una licenza tecnologica tra concorrenti diretti . . . 63
5.10 Frontier Firm e Work IQ: la cornice organizzativa . . . 63

6 Parte VI. Lettura comparata e implicazioni nel processo di A(I)doption 64
6.1 Convergenze . . . 64
6.2 Divergenze di filosofia . . . 65
6.3 Che cosa è davvero portabile tra le piattaforme . . . 66
6.4 Chi costruisce la delega: direzione, laboratorio e diffusione . . . 66
6.5 Un percorso di adozione a gradini . . . 66
6.6 Il quadro economico: abbonamenti, token, crediti e ore di esecuzione . . . 67

7 Parte VII. Conclusioni: la nuova competenza manageriale 71

Il mercato sta investendo con una convinzione quasi assoluta negli agenti, ma, ad oggi, la distanza tra la sperimentazione e l'adozione matura resta ancora ampia e si misura esattamente nella capacità (e nella possibilità) di un'organizzazione di trasformare l'entusiasmo per gli agenti in una disciplina di delega governata.

Questo breve papello nasce con un duplice obiettivo. Il primo è chiarire, con necessaria precisione tecnica ma senza gergo superfluo, che cosa sia oggi un agente AI, quali componenti lo costituiscano e quali di questi componenti siano ormai comuni a qualunque piattaforma seria, indipendentemente dal fornitore che la propone. Il secondo è entrare nel merito dei tre ecosistemi che, insieme, definiscono oggi lo stato dell'arte per chi lavora in azienda: quello di Anthropic, quello di OpenAI e quello di Microsoft, quest'ultimo costruito attorno a una visione di agente peculiare e sufficientemente diversa da meritare un approfondimento (fosse anche solo perché la diffusione della piattaforma è oltremodo capillare).

Per rendere concreti i concetti che seguono, useremo lungo tutto l'articolo un unico caso di riferimento, scelto in un ambito trasversale a qualunque settore: il processo mensile di chiusura e reporting di controllo di gestione di un gruppo organizzato su più unità di business o più Paesi. È un processo che chiunque abbia un minimo di familiarità con l'amministrazione aziendale riconosce immediatamente, ricco abbastanza da attraversare quasi tutti i temi tecnici che affronteremo, dalla raccolta dei dati alla verifica dei risultati, e sufficientemente lontano dal linguaggio informatico da restare comprensibile a un pubblico di manager e non solo di tecnici.

1 Parte I. Dal chatbot all'agente: le coordinate concettuali

1.1 Che cos'è, davvero, un agente AI

La distinzione più utile per orientarsi in questo campo non è (solo) quella tra chatbot e agente, ma quella tra workflow e agente, così come la propone Anthropic in uno dei testi fondativi della letteratura tecnica su questo tema. In un workflow, il percorso di esecuzione è predefinito dal codice che orchestra modello e strumenti secondo passi fissati in anticipo: per quanto sofisticato, la sequenza delle azioni è nota prima ancora che il compito cominci. In un agente, è il modello stesso a dirigere dinamicamente (con un certo livello di autonomia che costituisce una delle tre caratteristiche definitorie) il processo e la scelta degli strumenti, decidendo passo dopo passo come procedere in base a ciò che osserva. È una distinzione architetturale che non implica alcuna "graduatoria di merito" (come invece capita spesso di "percepire" in questa stagione di FOMO da agenti): un processo prevedibile e ripetitivo, come il calcolo di un rimborso spese secondo regole fisse, è spesso servito meglio da un workflow rigido, più economico e più facile da verificare, che da un agente autonomo. Un reclamo incompleto, che richiede di cercare un ordine mancante, confrontarlo con una policy e costruire una proposta di soluzione, è invece il tipo di compito in cui l'autonomia dell'agente potrebbe iniziare a creare valore reale.

Conviene evitare, fin da questa prima definizione, la formula assoluta secondo cui "il chatbot parla, l'agente agisce": un sistema conversazionale può benissimo consultare un documento o

usare uno strumento isolato, per esempio calcolare un totale o cercare un dato, senza per questo ricevere la responsabilità di un intero processo. La differenza va spiegata attraverso il grado di delega dei passaggi, non attraverso un confine netto tra il generare testo e il fare. La definizione operativa che adotteremo per il resto dell'articolo è dunque la seguente: un agente AI è un sistema **basato su di un modello di linguaggio** che, ricevuto un obiettivo, sceglie ed esegue passi utilizzando **strumenti**, osserva i risultati e adatta il seguito del lavoro entro limiti stabiliti, fino a decidere **autonomamente** quando il compito può dirsi concluso. Il modello linguistico è un componente di questo sistema, non il sistema stesso: l'agente comprende anche il software che ne governa l'esecuzione, il contesto a cui accede e i permessi che gli sono assegnati, un punto su cui torneremo subito.

Applicata al nostro caso di riferimento, la differenza si vede subito. Un workflow di chiusura mensile applica regole fisse: consolida i dati, calcola gli scostamenti secondo formule predefinite, popola un template. Un agente di chiusura mensile, oltre a questo, nota che lo scostamento di una business unit supera la soglia di materialità, cerca autonomamente possibili spiegazioni nei documenti disponibili, richiede al responsabile della business unit una nota di commento se non trova nulla di conclusivo, e adatta la propria proposta di reporting in base a quanto scopre lungo il percorso. Il workflow è prevedibile e verificabile riga per riga; l'agente è in grado di "gestire" l'eccezione, ma richiede una progettazione della delega più attenta, il tema a cui è dedicata la parte centrale di questo articolo.

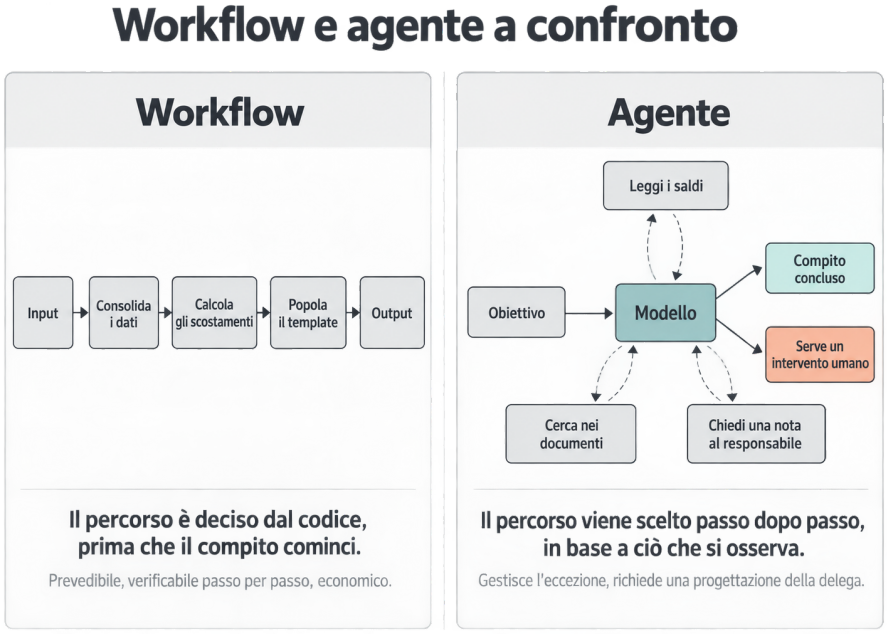


Figura 1: Workflow e agente a confronto

1.2 Il continuum dell'autonomia

"Agente" non è (nel linguaggio dei prodotti AI oggi, settembre 2026, disponibili) una categoria binaria (on-off) ma uno **spettro di autonomia**. All'estremo più cauto si trova il semplice tool

calling supervisionato, in cui l'agente propone un'azione e un umano la approva prima che venga eseguita. All'estremo opposto si trovano flussi che operano per ore, quando non per giorni, con supervisione solo asincrona, verificati a posteriori piuttosto che passo per passo. Nel mezzo si colloca la gran parte dei casi d'uso aziendali oggi realistici, in cui l'autonomia è ampia sui passaggi di analisi e raccolta dati, ma si restringe deliberatamente nei momenti in cui un'azione produce un effetto esterno irreversibile, come l'invio di un documento o l'aggiornamento di un sistema di produzione o che altri consultano.

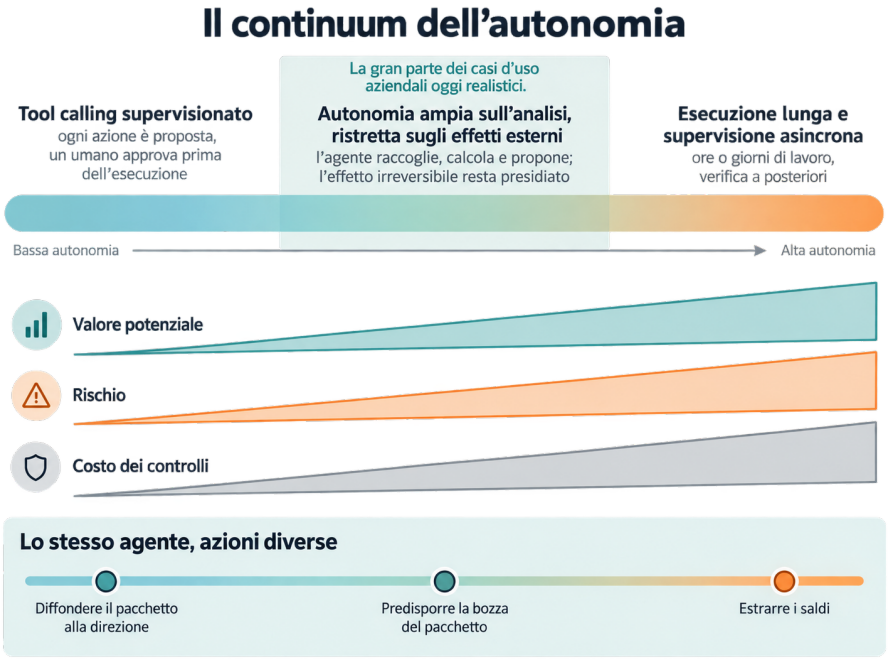


Figura 2: Il continuum dell'autonomia

Questo spettro non è solo una descrizione tecnica: ha implicazioni dirette di governance, di costo e di rischio, che si intensificano man mano che ci si sposta verso l'estremo più autonomo. Un'osservazione che vale la pena di richiamare qui è che questo stesso spettro di autonomia riproduce, in forma nuova, un problema antico quanto l'organizzazione d'impresa, quello dell'agenzia: ogni volta che si delega un compito a un esecutore (umano o artificiale) di cui non si osservano direttamente tutte le azioni, si genera un'asimmetria informativa che va gestita con strumenti di controllo appropriati. Di questo tema ho approfonditamente parlato sia in diversi articoli precedenti che anche in uno specifico capitolo del mio libro.

Se lo spettro dell'autonomia ha un'utilità operativa (e non meramente descrittiva) è perché consente di porre la domanda giusta, che non è quanto sia autonomo un agente, ma quanta autonomia meriti un determinato compito. Il grado di autonomia non è una "proprietà" dell'agente ma del singolo passaggio che gli si affida, e lo stesso agente può legittimamente trovarsi a estremi opposti della scala a seconda di ciò che sta facendo. I criteri che determinano il punto di collocazione sono essenzialmente tre, e vanno progettati e non applicati "a sentimento": 1) la reversibilità dell'effetto, cioè la possibilità di annullare l'azione senza conseguenze residue; 2) l'entità del danno potenziale, che include il danno reputazionale e non solo quello economico;

3) la verificabilità dell'esito, cioè la possibilità di accertare in modo rapido e oggettivo se il risultato sia corretto. Un'azione reversibile, di danno contenuto e verificabile in pochi secondi può stare all'estremo autonomo anche se tecnicamente complessa, mentre un'azione irreversibile, di danno elevato o di correttezza accertabile solo con un giudizio esperto deve restare soggetta ad approvazione anche se banale da eseguire. Nel nostro caso di riferimento, ricalcolare uno scostamento appartiene alla prima categoria, trasmettere il pacchetto al comitato di direzione alla seconda: le due azioni hanno una complessità tecnica paragonabile, ma non la medesima scala di rischio. ### L'autonomia regolata per fase: esplorare, pianificare, eseguire

Un modo concreto per rendere operativa la scala appena descritta, che gli agenti per lo sviluppo software hanno reso esplicito prima di tutti gli altri, consiste nel chiedersi (prima di iniziare) quanta autonomia serva in ciascuna fase del lavoro. Il flusso che produce i risultati migliori nello sviluppo del codice si articola in quattro fasi: analisi, pianificazione, esecuzione e cosolidamento. Nei primi due l'agente opera in sola lettura (in senso lato, scrive il piano l'analisi ed il piano, non il codice) in modo che un errore scoperto nella stesura del piano possa essere risolto con una correzione di poche righe e non ad esecuzione avanzata al prezzo di rework contesto consumato. Le modalità di permesso dei principali agenti per lo sviluppo vanno lette in questa chiave, e non come una manopola di prudenza da alzare o abbassare. La modalità di sola pianificazione serve a esplorare e a ragionare senza poter modificare nulla; quella ordinaria chiede conferma a ogni scrittura ed è adatta agli interventi chirurgici; quella che accetta le modifiche ai file ma continua a chiedere per i comandi di sistema è il punto di equilibrio per eseguire un piano già rivisto; quella senza interruzioni ha senso soltanto dentro un ambiente isolato e con una verifica forte, come vedremo nella sezione sui controlli. Passare dall'una all'altra nel corso della stessa sessione è il modo normare con cui un utilizzatore esperto attraversa le fasi.

Nel nostro caso di riferimento, l'agente di chiusura dovrebbe cominciare dal formulare il piano (quali business unit, quali conti, quali soglie, quali eccezioni già note), e il controller dovrebbe leggerlo e correggerlo prima che parta qualunque operazione di scrittura sui sistemi legata al task: la revisione che costa pochi minuti e sposta l'errore nel punto in cui è meno costoso.

Resta un ordine di grandezza che invita alla sobrietà (un dato che la letteratura divulgativa riporta senza una fonte primaria facilmente verificabile, e che cito quindi con cautela): chi usa questi strumenti vi ricorre per la maggior parte del proprio lavoro, ma la quota di compiti che riesce a delegare per intero, senza accompagnarli, si colloca fra lo zero e il venti per cento. Il confine fra lavoro umano e lavoro della macchina non è una linea netta ma una negoziazione continua, che fa parte del lavoro stesso. Chi si aspetta la delega totale e ne resta deluso, perchè confonde collaboratore con automatismo.

1.3 Che cosa si delega: il compito, non il mestiere

Una precisazione perché, come diceva un tale, le parole contano: un mestiere non è un compito. Anche quando un sistema eguaglia o supera un esperto su una prova che impegna per ore di elaborazione, questo non implica la sostituibilità del ruolo, perché un lavoro è un portafoglio di

compiti tenuti insieme da relazioni, responsabilità, conoscenza tacita, eccezioni e meccanismi di coordinamento che nessuna prova isolata (per quanto complessa) misura. L'agente interviene sui compiti, ed è così che va progettato, ma può modificare l'architettura del ruolo quando più compiti collegati fra loro diventano delegabili insieme, il che è cosa diversa dalla somma di singole automazioni. La distinzione serve a evitare due errori speculari e ugualmente diffusi, quello della sostituzione imminente, che scambia la prestazione su un compito per il presidio di una funzione, e quello rassicurante del "non potrà mai fare il mio lavoro", che ignora l'effetto cumulativo della delega. Nel nostro caso di riferimento il controller non scompare perché un agente raccoglie i dati, prepara le analisi, redige la bozza e svolge i controlli preliminari, ma il suo ruolo cambia *densità*, perché la quota di tempo dedicata alla produzione si riduce e quella dedicata al giudizio, alla relazione con le business unit e alla responsabilità dell'esito cresce in proporzione. Questa considerazione dovrebbe trasformarsi anche in un criterio di progettazione per l'agentificazione (ma sì, inventiamoci parole a caso) partendo sempre dalla catena del valore e dai punti decisionali, non dal catalogo delle funzionalità disponibili. Per indirizzare efficacia e complessità non basterà quasi mai un agente unico ma una pluralità di configurazioni, agenti personali, agenti di processo, agenti specialistici su una base di conoscenza, agenti che ne coordinano altri, in quell'ottica multi agente (di cui diremo più avanti) che va progettata in modo olistico ed implementata in modo incrementale.

1.4 Anatomia di un agente: modello, harness, contesto, ambiente

Il termine tecnico *harness* indica l'insieme di software che organizza il ciclo di lavoro dell'agente: la gestione degli strumenti, l'esecuzione dei comandi, la gestione del contesto, le interruzioni e la ripresa di un compito. Sia OpenAI, con la propria Agents API, sia Anthropic, con i Claude Managed Agents, descrivono oggi esplicitamente infrastrutture gestite dedicate proprio a questo compito, un fatto che segna una distinzione netta tra il semplice accesso a un modello e l'adozione di un vero e proprio ambiente operativo per il lavoro agentico

Per orientarsi in modo consapevole in questo campo (consapevoli anche della sua costante evoluzione), propongo di tenere separati cinque livelli, una distinzione che vale la pena avere sempre presente perché torneremo a usarla in ogni sezione successiva dell'articolo: il **modello**, cioè il sistema linguistico che ragiona; l'**applicazione** che lo utilizza, cioè l'**interfaccia** con cui l'utente o l'organizzazione interagisce; l'**ambiente di esecuzione**, cioè dove effettivamente girano gli strumenti e il codice che l'agente invoca; le **integrazioni**, cioè i collegamenti verso i sistemi aziendali; e i **controlli**, cioè permessi, guardrail e audit. Il motivo per cui questa distinzione è operativamente rilevante e non solo tassonomica è che cambiare modello non trasferisce automaticamente memoria, strumenti, autorizzazioni e workflow: un'organizzazione che decide di passare da un modello a un altro, o da un fornitore a un altro, deve ricostruire o migrare separatamente ciascuno di questi livelli.



Figura 3: I cinque livelli di un sistema agentico

Nel nostro caso di riferimento, il valore pratico di questa distinzione si vede in un dettaglio apparentemente minore ma frequente nella realtà operativa: un'analisi di chiusura mensile automatizzata può interrompersi non per un limite del modello, che magari ragiona perfettamente, ma perché manca un file atteso nel repository condiviso o è scaduto l'accesso al sistema gestionale. Il valore di un buon harness è proprio conservare lo stato del lavoro già svolto e riprendere dal punto corretto, senza dover compiere a mano l'aggiornamento dei sistemi coinvolti quando l'*incidente* viene risolto.

Una quota significativa delle capacità inutilizzate di questi sistemi non dipende da ciò che il modello non sa fare, ma dalla povertà dell'interfaccia attraverso cui glielo si chiede: la conversazione lineare produce muri di testo, disperde lo stato del lavoro e confonde le decisioni con le bozze, penalizzando soprattutto gli utenti meno esperti. Un'interfaccia agentica (uno dei cinque livelli appena distinti, e non un accessorio dei primi tre), molto più che quella di un chatbot, deve rendere visibili il mandato ricevuto, il piano che l'agente si è dato, le fonti che ha consultato, le azioni che ha compiuto, lo stato di avanzamento, le eccezioni incontrate e i punti in cui attende un'approvazione. Non è una questione di estetica (anche se, dopo anni di interazione con interfacce brutte come quelle dei chatbot, quelle agentiche sono una boccata di ossigeno) ma un contributo significativo ad abbattere il costo cognitivo della supervisione che, di fatto, limita la scalabilità degli agenti. Un agente invisibile, anche quando funziona, resta difficile da correggere e impossibile da estendere, perché nessuno se ne assume la responsabilità a scatola chiusa.

2 Parte II. L'infrastruttura comune: la logica di base indipendente dalle piattaforme

Prima di entrare nelle specificità degli editori di modelli commerciali di maggiore diffusione com Anthropic, OpenAI e Microsoft, vale la pena fermarsi sugli elementi che, pur con implementazioni proprietarie diverse, sono ormai trasversali a qualunque piattaforma agentica seria. È la parte più "voluminosa" di questo lavoro, ma anche quella più genericabile e, si spera, duratura. I nomi dei singoli prodotti cambieranno ma questa grammatica di base è quella con cui continueremo a ragionare sugli agenti per diversi anni.

2.1 Il ciclo agentico e il tool use

Il meccanismo di base su cui poggia qualunque comportamento agentico è il cosiddetto **tool use** cioè la vecchia cara RFC (remote function calling). Il modello, oltre a generare testo, può chiamare una funzione esterna con parametri specifici, il software chiamato esegue quella funzione, e il risultato rientra nel contesto del modello, che lo usa per decidere il passo successivo. Questo scambio, ripetuto quante volte necessario, costituisce il **ciclo agentico**: un obiettivo iniziale, la scelta di un passo, l'azione, l'acquisizione e valutazione del risultato, un'eventuale verifica, e infine la decisione di continuare o di fermarsi, dichiarando il compito concluso o segnalando l'impossibilità di proseguire senza un intervento umano.

È utile distinguere, fin da questo primo blocco, gli strumenti che leggono da quelli che "scrivono" (o modificano). Leggere lo stato di una fattura o di un saldo contabile e modificarne il valore sono due operazioni radicalmente diverse sul piano del rischio, anche per quanto risultino tecnicamente simili nella loro implementazione: la prima è quasi sempre sicura da automatizzare con ampio margine di autonomia, la seconda richiede permessi più stretti e, spesso, un punto di verifica umana. Nel nostro caso "esemplificativo" di riferimento, un agente che consulta i saldi del sistema gestionale per calcolare uno scostamento opera in una zona a basso rischio; lo stesso agente che modifica un valore nel sistema contabile, oppure che invia il reporting package fuori dal perimetro autorizzato, si sposterebbe in una zona che richiede controlli diversi, di cui parleremo più avanti in questa stessa parte. Vale infine la pena di ricordare, perché è un errore comune nella comunicazione aziendale sull'AI, che una richiesta formalmente valida non è per questo una richiesta corretta sul piano del merito: un agente può costruire una query sintatticamente ineccepibile su un dato sbagliato, ed è per questo che la verifica dell'esito, di cui parleremo al paragrafo dedicato, non può mai essere sostituita dalla sola eleganza tecnica dell'esecuzione.

Vale la pena guardare da vicino come un tool si presenta (si "espone") effettivamente al modello, perché è il punto in cui la retorica sugli agenti incontra la sua parte più prosaica e più rassicurante. Un tool non è un frammento di intelligenza: è una **dichiarazione**, scritta in un formato strutturato (di norma JSON Schema), che descrive un nome, uno scopo e i parametri accettati. Il modello riceve questa dichiarazione insieme al prompt, e ciò che "decide" è soltanto *se* invocarla e *con quali parametri*.

```
{
  "name": "get_account_balance",
  "description": "Restituisce il saldo di un conto per una BU e un periodo",
  "input_schema": {
    "type": "object",
    "properties": {
      "bu_code": { "type": "string" },
      "periodo": { "type": "string", "description": "formato AAAA-MM" }
    },
    "required": ["bu_code", "periodo"]
  }
}
```

Tre osservazioni ritengo sia necessario subito affrontare. La prima è che il campo `description` è la parte che fa davvero lavorare il sistema: è scritto in linguaggio naturale ed è, a tutti gli effetti, la documentazione sulla cui base il modello sceglie. Una descrizione ambigua produce chiamate sbagliate molto più spesso di quanto non faccia un modello "poco capace", ed è la ragione per cui la qualità di un'integrazione agentica dipende, in misura sorprendente, da competenze di scrittura (ahi ahi ahi ... poveri noi) più che di programmazione. La seconda è che il modello non esegue nulla: invoca una funzione tramite una richiesta strutturata, il software che lo espone la esegue, e il risultato rientra come nuovo messaggio, così che il turno di lavoro in un ciclo agentico non è "domanda e risposta" ma "richiesta di strumento e ricezione del risultato dello strumento", ripetuto finché serve. La terza è che la validazione dello schema avviene prima dell'esecuzione: un parametro fuori formato viene respinto dal software, non dal modello, ed è la prima e più economica barriera di sicurezza disponibile.

Vale la pena, a questo punto, seguire l'intera sequenza, per smontare l'impressione (che avrebbe potuto lecitamente sovvenire) che il modello "sappia" qualcosa dei sistemi a cui è collegato.

Nel momento in cui un collegamento viene attivato, l'applicazione chiede al sistema collegato l'elenco delle capacità disponibili ricevendo in risposta le dichiarazioni, ciascuna con il proprio nome, la propria descrizione e il proprio schema dei parametri. Quelle dichiarazioni vengono inserite nel contesto del modello, insieme alle istruzioni di sistema e alla richiesta dell'utente: sono testo, e il modello le legge come legge tutto il resto. Il modello non sa che il sistema gestionale esiste, ma gli è stato detto, in quella conversazione e soltanto in quella, che esiste una **funzione** con quel **nome**, quella **descrizione** e quei **parametri**. Spento il collegamento, quella conoscenza sparisce, per la semplice ragione che non è mai stata sua.

A quel punto l'utente formula la propria richiesta, e il modello, invece di produrre testo ordinario, produce un'uscita strutturata che nomina una delle funzioni dichiarate e ne valorizza i parametri. La corrispondenza fra la richiesta e la funzione avviene per somiglianza "di significato" (lo so, si potrebbe dire somiglianza semantica ma mi fa ancora senso parlare di semantica riferendomi a sti attrezzi) fra ciò che è stato chiesto e ciò che la descrizione dichiara di fare: è

una selezione fra opzioni dichiarate sulla base di un testo, non una comprensione del sistema. Il software intercetta quella richiesta, valida i parametri contro lo schema, esegue davvero la chiamata, e fa rientrare il risultato come nuovo messaggio, sul quale il modello decide se il compito sia concluso oppure se serva un altro passo.

Una traccia semplificata di ciò che attraversa effettivamente il filo, nel nostro caso di riferimento, rende la sequenza meno astratta:

```
[già in contesto, prima ancora della richiesta]
  { "name": "get_account_balance",
    "description": "Restituisce il saldo di un conto per una BU e un periodo",
    "input_schema": { "bu_code": "string", "periodo": "string (AAAA-MM)" } }
  { "name": "get_budget",
    "description": "Restituisce il budget approvato per una BU e un periodo",
    ↪  ... }

[utente]    Qual è lo scostamento della BU Nord a marzo?
[modello]   → get_account_balance(bu_code="IT-NORD", periodo="2026-03")
[software]  ← {"saldo": 1284300, "aggiornato_al": "2026-04-02"}
[modello]   → get_budget(bu_code="IT-NORD", periodo="2026-03")
[software]  ← {"budget": 1230000, "delibera": "2026-01-18"}
[modello]   Lo scostamento è di 54.300 euro, pari al 4,4 per cento del budget.
```

Si noti che una richiesta apparentemente semplice produce, quasi sempre, più di una chiamata, e che il secondo passo dipende dal risultato del primo: è il ciclo agentico in miniatura, e si osserva bene che l'informazione necessaria al passo successivo il modello non poteva averla prima. E siccome stiamo parlando di modelli linguistici che funzionano su base stocastica **la selezione non è deterministica** ovverosia la stessa richiesta, in due sessioni diverse, può tradursi nella scelta di una funzione diversa o parametri diversi. Ne derivano due regole pratiche da appuntarsi come mantra primo che la qualità della descrizione conta più della capacità del modello, secondo più strumenti si dichiarano, peggio il modello sceglie. Trenta funzioni dichiarate insieme producono errori di selezione che cinque non producono, ed è un errore particolarmente insidioso perché le chiamate risultano comunque formalmente valide.

Da qui discende un principio di progettazione che è anche un principio organizzativo che conviene enunciare con chiarezza perché va contro la strada che a prima vista appare più comoda.

Quando un sistema espone già un'interfaccia di programmazione, la tentazione è forte perché la scorciatoia è disponibile: esistono strumenti che, a partire dalla specifica tecnica di quell'interfaccia, generano automaticamente una funzione dichiarata per ciascuna delle operazioni disponibili. Il risultato è un collegamento con ottanta strumenti, i cui nomi sono quelli pensati dagli sviluppatori per altri sviluppatori, e le cui descrizioni sono frammenti di documentazione tecnica scritti per chi già conosce il dominio. Un agente messo davanti a ottanta dichiarazioni di questo tipo

sceglie **male**, e per le ragioni appena viste sceglie male nel modo più insidioso, cioè con chiamate formalmente ineccepibili.

La ragione di fondo è che un'interfaccia progettata per alimentare uno schermo non è un'interfaccia progettata per un agente. Uno schermo fa molte chiamate piccole e le ricompone davanti all'utente; un agente paga ogni chiamata in tempo e ogni risposta in contesto. Se nel nostro caso di riferimento, per ottenere lo scostamento di una business unit, occorrono una chiamata per il saldo di ciascun conto, una per il budget, una per la soglia di materialità e una per la data di aggiornamento, la funzione giusta da dichiarare non è nessuna di quelle: è una funzione sola, `calcola_scostamento`, che al proprio interno esegue tutte le chiamate necessarie e restituisce un oggetto di sei campi. Vale lo stesso per la forma della risposta. Un'interrogazione contabile restituisce facilmente centinaia di campi, fra codici interni, indicatori di stato e riferimenti tecnici, e riversarli tutti nel contesto del modello costa, non aggiunge nulla alla decisione e ne peggiora la qualità, perché diluisce l'informazione utile in un rumore che il modello è comunque costretto a leggere e dentro cui deve discriminare. Il connettore deve filtrare, e la scelta di che cosa filtrare è una decisione di merito che presuppone di sapere a che cosa il dato serva.

Il criterio, allora, è semplice da enunciare: gli strumenti si disegnano sui compiti, non sulle operazioni tecniche disponibili. La domanda da porsi non è quali chiamate esponga un certo sistema, ma quali siano le cinque o sei cose che l'agente deve saper fare, e per ciascuna di esse si costruisce una funzione sola, con un nome che anche una persona di business non IT possa capire senza spiegazioni, una descrizione scritta per essere letta e non per documentare, e una risposta ridotta all'essenziale e accompagnata dall'indicazione della freschezza del dato.

C'è poi un secondo conto da fare che si aggiunge a quello degli errori di selezione : ogni dichiarazione di strumento occupa spazio nel contesto del modello a ogni singola richiesta, che lo strumento venga usato oppure no, ed un collegamento con descrizioni ricche e parametri annidati costa più di uno "snello". I prodotti più recenti attenuano il problema caricando per intero soltanto una frazione delle dichiarazioni e tenendo le altre in un indice, da cui il modello le recupera quando un compito sembra richiederle (con un passaggio di ricerca che a sua volta consuma contesto e aggiunge latenza), ma l'attenuazione non dispensa dal disegno. Le pratiche che reggono a scala, cioè con decine di strumenti distribuiti su più server, sono sorprendentemente amministrative: un server per ciascun dominio, così che il guasto dell'uno non travolga gli altri; nomi che portano il dominio come prefisso (due strumenti con lo stesso nome su server diversi producono un comportamento indefinito, perché uno oscura l'altro e quale prevalga dipende dall'ordine di inizializzazione); schemi ridotti al minimo, con le spiegazioni lunghe spostate in una skill invece che nella dichiarazione che si paga a ogni giro di giostra e, quando il catalogo è davvero necessariamente ampio, un livello di instradamento che classifica l'intento della richiesta e restringe a cinque o otto gli strumenti fra cui il modello sceglie. Un'ultima possibilità, spesso trascurata, è intervenire sull'uscita: un controllo programmatico che filtra la risposta di uno strumento prima che il modello la legga riduce il consumo di contesto e ne migliora l'attenzione, senza toccare il server.

È a questo punto che la questione smette di essere tecnica e diventa, ahimè come tutto

quello che riguarda l'AI, organizzativa. Decidere che l'agente disponga di una singola funzione `calcola_scostamento` e non di quindici operazioni elementari significa aver deciso quale sia l'unità di lavoro che si intende delegare, ed è esattamente la stessa decisione che si prende quando si scrive una procedura per una persona appena entrata in un ufficio. Ne esce confermata la considerazione che la qualità di un'integrazione agentica dipende in misura sorprendente da competenze di scrittura più che di programmazione, e, si potrebbe aggiungere ora, da competenze di progettazione organizzativa più che di architettura del software.

Nel nostro caso di riferimento la conseguenza è netta: se il perimetro di `get_account_balance` è la sola lettura, nessuna formulazione del prompt, per quanto abile o malevola, potrà trasformarla in scrittura, perché la funzione di scrittura semplicemente non esiste fra gli strumenti dichiarati. È un punto che vale la pena in tutti quei contesti in cui si tende a discutere a lungo di che cosa l'agente "potrebbe decidere di fare": il perimetro di ciò che un agente può fare non è una questione di istruzioni, è una questione di quali funzioni gli sono state messe a disposizione. Punto. (Chi è causa dei suoi mali quindi sa chi piangere, come avrebbe detto mia nonna).



Figura 4: Il ciclo agentico

2.2 Una mappa terminologica: prompt, istruzioni, skill, tool, connettore, MCP, plugin

Anche se verrà percepito come un "complicato approfondimento tecnico" ritengo che dotarsi di un glossario "rigoroso" e riferivicisi con precisione sia il primo passo per una conversazione aziendale sensata sull'argomento.

Elemento	Funzione	Esempio nel caso di riferimento
Prompt	Esprime una richiesta puntuale (one shot) esprimendo (quando i prompt sono fatti bene) contesto di riferimento, attesa, istruzioni ed informazioni (do e do not), step operativi, meccanismi di controllo e vincoli	CONTESTO: Sei l'assistente di un Controller di ... TASK: Prepara la bozza di reporting per la BU Nord secondo gli standard di riferimento ... INFORMAZIONI: utilizza i dati ..., STEP1: inizia a preparare outline della presentazione ...
Istruzioni persistenti	Convenzioni o contesto di riferimento (aziendale, funzionale) applicabili a più attività, caricate ogni volta (mediante configurazione nel bot oppure file di configurazione)	Soglia di materialità, formato delle date, glossario di gruppo
Skill	Procedura riutilizzabile, con risorse (strumenti, template, formati etc) associate, caricata quando pertinente (sia a fronte di un richiamo esplicito nel "prompt" sia in forma "autonoma" a fronte del riconoscimento dell'ambito di competenza richiesto dall'azione)	Metodologia di reporting (modello di controllo, processo di closing etc), standard contabili di riferimento e template di reporting direzionale del gruppo
Tool (strumento)	Sistema informatico che espone uno o più "funzioni" (tramite API od altri protocolli come MCP) che vengono eseguite puntualmente con dati di input e producendo output	Lettura di un saldo dal sistema contabile, calcolo uno scostamento tramite il modello di reporting, aggiornamento di una cella in un foglio di lavoro
Connettore	Collegamento dell'agente a un'applicazione o a una fonte dati specifica	Accesso in sola lettura al sistema gestionale e al sistema di budget
MCP	Protocollo che definisce una modalità universale di esporre e usare capacità (funzioni) e contesto di applicazioni in modo standard	Un server MCP espone le operazioni di consultazione del gestionale a più applicazioni compatibili
Plugin	Pacchetto distribuibile che raggruppa skill, connettori e altre estensioni	Pacchetto "controllo di gestione" con metodo, connettori e controlli pre-configurati

Una precisazione va aggiunta proprio qui, perché la forma stessa della tabella rischia di indurre in errore: connettore e MCP compaiono come due voci affiancate, e questo suggerisce che siano

due oggetti dello stesso genere fra i quali si debba scegliere. Non è così, e la domanda che sento porre più spesso nelle riunioni aziendali, cioè se convenga usare un connettore oppure il MCP, è una domanda malposta.

Connettore è una parola di prodotto. Risponde alla domanda su che cosa io abbia collegato, con quale autorizzazione e in che modo io possa revocarlo. È l'unità che vedo nell'interfaccia, quella che ha un nome, un'icona, un interruttore, una schermata di consenso e un pulsante per rimuoverla: un'unità di installazione, di consenso e di revoca.

MCP è una parola di protocollo. Risponde a una domanda diversa, cioè in che lingua si parlino il programma che sto usando e il sistema collegato. È il formato del dialogo, e non lo vedo mai, esattamente come non vedo mai il protocollo con cui il mio computer parla con la stampante.

L'analogia della stampa è del resto abbastanza fedele da poterla usare: il connettore è il driver che installo, con il suo nome e le sue impostazioni, mentre il MCP è il linguaggio che quel driver parla con la stampante. Posso avere un driver che parla uno standard aperto e uno che parla un formato proprietario del produttore, e per me che stampo non cambia nulla, ma cambia tutto nel momento in cui devo collegare una stampante diversa.

Ne discende che le quattro combinazioni esistono tutte, e che dall'interfaccia non è dato distinguerle.

	Costruito su MCP	Non costruito su MCP
Confezionato come connettore	il caso più diffuso oggi: un server MCP remoto, con autenticazione delegata, presentato all'utente come un interruttore da accendere	un'integrazione proprietaria, costruita prima che il protocollo esistesse oppure dal fornitore stesso verso i propri servizi
Non confezionato	un server MCP configurato a mano in un file, senza consenso interattivo e senza catalogo	un semplice strumento dichiarato nel codice della propria applicazione

La conseguenza pratica è quella che conta, e riguarda una delle discussioni più frequenti sul rischio di dipendenza da un fornitore: il fatto che due prodotti mostrino entrambi un connettore verso il medesimo servizio non dice assolutamente nulla sul fatto che quel collegamento sia trasferibile dall'uno all'altro. Si scambia, in quel ragionamento, la portabilità dell'esperienza d'uso per la portabilità dell'integrazione, che sono cose diverse. È la ragione per cui, quando si valuta una piattaforma, la domanda utile non è se supporti i connettori, ma quali di essi siano

costruiti su uno standard aperto, e che cosa resti in mano all'organizzazione nel giorno in cui quella piattaforma venisse sostituita.

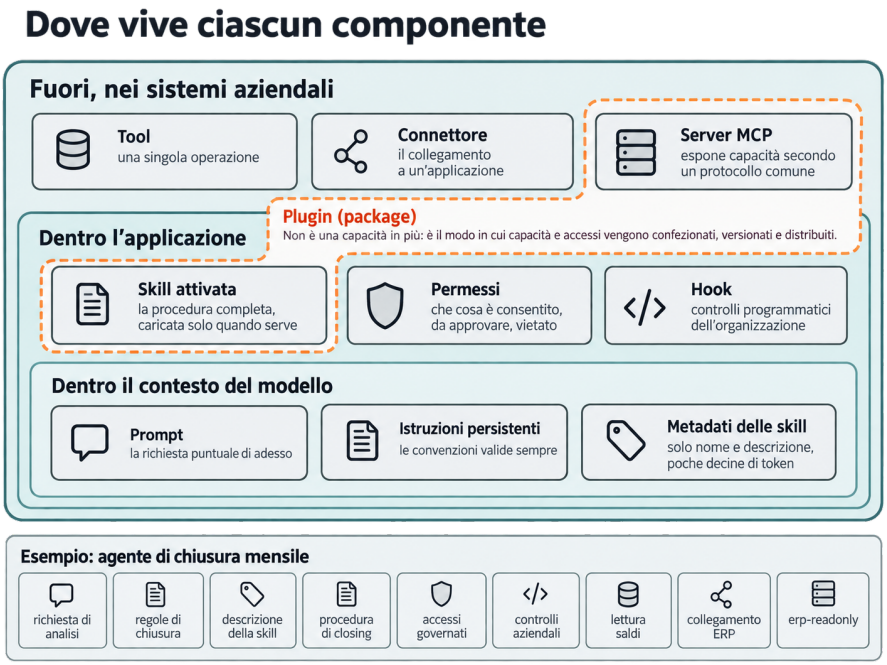


Figura 5: Dove vive ciascun componente

Le **skill** meritano un approfondimento, perché sono probabilmente l'innovazione più concreta ultimi due anni in questo ambito. Una **skill** è una cartella che raccoglie istruzioni, metadati e risorse opzionali (script, "prompt" template, documenti di riferimento) che l'agente utilizza automaticamente quando pertinenti al compito in corso, senza che occorra reinserirle a ogni conversazione. L'architettura con cui Anthropic ha reso pubblico questo formato, oggi aperto, adotta un principio di caricamento progressivo su tre livelli: i metadati della skill (nome e breve descrizione) sono sempre presenti nel contesto del sistema, a un costo computazionale (i.e. token richiesti) minimo, le istruzioni vere e proprie vengono caricate solo quando la skill viene effettivamente attivata, le risorse più corpose, script inclusi, vengono lette solo se effettivamente referenziate durante l'esecuzione. Questo permette a un'organizzazione di dotare un agente di decine di competenze specialistiche senza che il solo elenco di ciò che l'agente "sa fare" occupi una porzione significativa della sua capacità (i.e costi) di ragionamento. Concretamente una skill si presenta come una cartella che contiene un file `SKILL.md`, con un breve intestazione di metadati (nome, descrizione sintetica, condizioni in cui va attivata) seguita dalle istruzioni operative vere e proprie, affiancata da sottocartelle con le risorse effettivamente da richiamare, per esempio un template in formato Excel e uno script di calcolo. ù Nel nostro caso di riferimento, la skill "reporting di controllo di gestione" avrebbe una descrizione del tipo "genera il pacchetto di reporting mensile per una business unit a partire dai saldi del gestionale e dal budget approvato", una sezione di istruzioni che, oltre a fissare le convenzioni di formattazione delle date e delle valute, richiama la soglia di materialità già definita altrove, come istruzione persistente (per esempio 15.000 euro o il 5% del valore di budget, la minore delle due, tenuta in un file come `soglie.md`

e importata dal `CLAUDE.md` di progetto, non ridefinita nella skill), e una cartella di risorse con il template del pacchetto direzionale del gruppo: solo quando l'agente riceve un compito che può essere interpretato come aderente alle specifiche "capacità", il sistema carica le istruzioni complete, e solo se lo script di calcolo viene effettivamente invocato ne legge il contenuto. Questa struttura permette il caricamento progressivo di cui sopra, ed assicura che ogni agente che la richiami produca un output coerente con lo standard aziendale, senza doverlo istruire ogni volta.

In termini di file, una skill è poco più di un documento di testo con un'intestazione strutturata:

```
---
name: reporting-controllo-di-gestione
description: Genera il pacchetto di reporting mensile per una BU a partire dai
  saldi del gestionale e dal budget approvato. Usare per chiusure e forecast.
---

## Procedura
1. Estrarre i saldi del periodo (strumento: get_account_balance)
2. Confrontare con l'ultima versione approvata del budget
3. Evidenziare gli scostamenti oltre soglia (criteri in soglie.md)
```

accompagnato da una struttura di cartelle di questo tipo:

```
reporting-controllo-di-gestione/
├── SKILL.md
├── soglie.md
├── template/pacchetto-direzionale.xlsx
└── scripts/calcola_scostamenti.py
```

L'economia di questo meccanismo è più interessante della sua sintassi. Le due righe di `description` restano sempre nel contesto, al costo da poche decine a qualche centinaio di token (a seconda della lunghezza della descrizione); il corpo del `SKILL.md` entra solo quando la skill viene attivata, al costo di qualche migliaio; i file di supporto vengono letti solo se il percorso li richiede effettivamente. Un'organizzazione può quindi mantenere cinquanta procedure formalizzate pagando, a riposo, il costo di cinquanta descrizioni brevi. È la ragione per cui questo formato, nato come espediente tecnico per risparmiare contesto, è in realtà un fatto organizzativo di un certo peso: rende economicamente sostenibile **codificare il sapere operativo** invece di lasciarlo depositato nell'esperienza delle persone, che è poi il problema che il controllo di gestione si pone da sempre quando prova a documentare un processo.

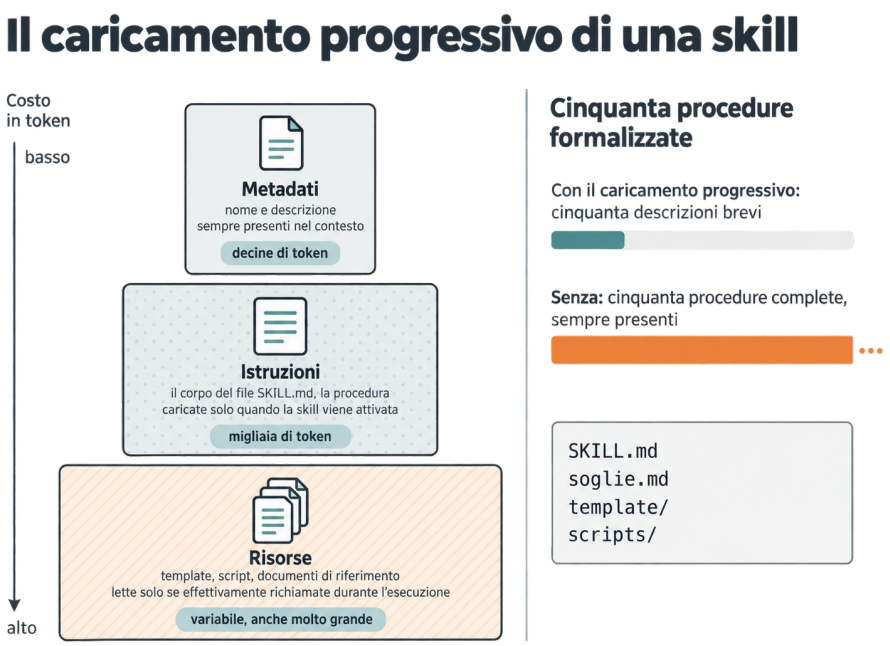


Figura 6: Il caricamento progressivo di una skill

Un punto da chiarire per evitare fraintendimenti comuni nella comunicazione aziendale: una skill non è cioè una forma di addestramento e come tale non modifica i pesi del modello linguistico che la usa (la bufala dell'impara mentre esegue che tanti fornitori marmaldi spacciano a tanti clienti ingenui) e non concede da sola il diritto di accedere a un'applicazione, che resta demandato al connettore e ai permessi assegnati.

Condivido un esempio tratto dalla mia esperienza diretta per mostrare non che cosa sia una skill (cosa che abbiamo già trattato), ma come se ne renda evidente la necessità.

Negli ultimi anni, lavorando con questi strumenti, mi sono accorto che una certa conversazione si ripeteva quasi identica: a partire da un saggio od un articolo lungo da me scritto di solito preparo (con un modello) il documento di istruzioni per un'altra AI incaricata di produrre le immagini di un carosello da pubblicare sui social per "presentare" l'articolo. Stesse tre domande preliminari (il testo va generato insieme all'immagine oppure composto dopo, quale formato, quante tavole), stessa struttura del documento, stesse convenzioni visive, stesse avvertenze sugli accenti che i modelli di immagine sbagliano con regolarità. Diversi saggi scritti a qualche mese di distanza, e la medesima procedura ricostruita ogni volta da capo.

Il criterio per capire se una conversazione ricorrente possa diventare una skill è semplice: se dovessi affidare quello stesso lavoro a un collega appena arrivato, gli consegnerei un foglio di istruzioni oppure dovrei sedermi con lui e discuterne? Nel primo caso è una skill, nel secondo non lo è, o non lo è ancora. Il brief del carosello è una procedura chiusa con un esito riconoscibile, ed è esattamente il genere di sapere che conviene codificare.

La costruzione è tutto sommato semplice. Una cartella, un file `SKILL.md` con un'intestazione di due righe (il nome, e una descrizione che dice che cosa fa la skill e quando va usata, scritta con le parole con cui formulerei la richiesta), e sotto la procedura in testo ordinario. Le parti

più voluminose, cioè le convenzioni cromatiche e un esempio completo di documento, stanno in file separati (in modo da poter scegliere un formato riconoscibile a seconda dell'audience e del contenuto), che il corpo della skill nomina esplicitamente rendendo operativo il terzo livello del caricamento progressivo. Nei prodotti Anthropic la cartella si carica compressa, dalla sezione di personalizzazione dell'applicazione, oppure si colloca in una directory dedicata quando si lavora con lo strumento per sviluppatori; da quel momento si attiva da sola quando la richiesta corrisponde alla descrizione, oppure la si richiama per nome.

Il collaudo va fatto su un caso diverso da quello da cui la skill è stata ricavata, e verifica tre cose: se si attiva senza essere nominata (in caso contrario va riscritta la descrizione), se produce la struttura prevista o se improvvisa (in caso contrario il corpo è troppo vago), e se legge i file di supporto quando servono (in caso contrario non sono stati nominati con sufficiente chiarezza).

Nulla, in tutto questo, è stato addestrato. Nessun peso è stato modificato, e il modello che usa la skill è identico a quello che non la usa. Un sapere operativo che fino al giorno prima viveva nella mia testa e nella cronologia delle conversazioni è diventato un file di testo con un'intestazione, che chiunque può aprire, leggere, correggere e cancellare. Finché resta nella cartella di una persona, però, è poco più di un appunto ben fatto. Diventa materiale a supporto della A(I)doption quando, soprattutto nell'ambito delle attività di LA(I)B diventa pratica comune condivisa in ambito aziendale.

Distinto dalla skill è il **plugin**, il meccanismo di confezionamento e distribuzione che può includere skill, server MCP e altre estensioni. OpenAI popone skill per ChatGPT e Codex e plugin per distribuirle insieme ai connettori; Anthropic propone plugin utilizzabili in Chat e in Cowork, mentre Claude Code dispone di un proprio meccanismo di estensioni. Formato condiviso e disponibilità su più prodotti non sono garanzia di comportamento identico né di piena portabilità delle dipendenze tra piattaforme, un tema su cui torneremo nella parte comparativa più avanti.

Se la skill risponde alla domanda "che cosa l'agente sa fare", il plugin (o *package*, nella terminologia di alcuni prodotti) risponde a una domanda diversa e tipicamente aziendale: **come si distribuisce, si aggiorna e si revoca quel sapere**. È il passaggio in cui una procedura smette di essere un file sulla macchina di qualcuno e diventa un oggetto versionato, e la sua struttura è quella di un normale pacchetto software:

```
controllo-di-gestione/
├── .claude-plugin/plugin.json   # nome, versione, autore, descrizione
├── skills/                      # una o più skill
├── commands/                    # comandi richiamabili esplicitamente
├── agents/                      # subagenti specializzati
└── .mcp.json                    # server MCP inclusi nel pacchetto
```

La distribuzione avviene poi attraverso un repository, che nel caso aziendale è opportuno sia interno e controllato. Ed è esattamente qui che la materia smette di essere di pertinenza del singolo utente: versioni, dipendenze, provenienza del codice e diritto di installazione sono gli stessi problemi che poniamo da trent'anni a qualunque software aziendale. Non vedo alcuna

ragione per cui un pacchetto che contiene istruzioni operative, criteri di calcolo e accessi al sistema contabile debba essere trattato con meno rigore di un modulo del gestionale, e temo che in molte organizzazioni, in questa fase, stia accadendo esattamente il contrario.

2.3 Che cosa c'è sotto il connettore

Se le skill risolvono il problema di "che cosa l'agente sa fare", i connettori e il Model Context Protocol risolvono quello, altrettanto centrale, di "a che cosa l'agente può accedere". Prima però di guardare il protocollo conviene guardare lo step che lo precede perché è spesso quello in cui il vocabolario corrente confonde due cose distinte.

Una **API** è un contratto esposto dal sistema di origine. Sta dalla parte del sistema, è la porta che il gestionale apre verso il mondo, e definisce quali operazioni siano disponibili, con quali parametri e con quali risposte. Chi la usi, e per farne che cosa, non è affare suo.

Un **connettore** è invece software di integrazione che sta dalla parte di chi consuma. Sa parlare con quel sistema specifico e si fa carico di tutto ciò che il sistema di origine non fa per te: autenticarsi e rinnovare le credenziali, gestire il trasporto e i tempi di attesa, paginare i risultati lunghi, tradurre gli errori in qualcosa di comprensibile, riprovare quando riprovare ha senso, e soprattutto mappare il modello dati di partenza (che in un gestionale anziano è spesso un file con nomi di campo di sei caratteri ereditati dagli anni Ottanta) in un modello che abbia senso per chi lo usa. Nel vocabolario dei prodotti AI la parola ha poi assunto un significato più stretto e più commerciale, cioè quello di un'integrazione confezionata e installabile, con la propria schermata di consenso e il proprio interruttore.

Ne discende una precisazione che vale la pena fissare da subito: un connettore non ha bisogno strettamente di una API, ha bisogno di una via di accesso programmatica. Il più delle volte quella via è effettivamente una API, ma può benissimo essere un driver di database, una stored procedure, una coda di dati, un file depositato in una cartella condivisa o, nei casi peggiori, l'emulazione di un terminale (sacrosanta cosa da evitare sempre come la peste o la malaria, quando possibile). E se quella via non esiste, il primo compito del progetto non è scrivere il connettore ma creare la via.

Il caso limite (che nella realtà italiana delle imprese di media dimensione è la norma e non l'eccezione) è quello di un Legacy. Prendiamo un sistema su IBM i, l'ex AS/400, che molte aziende manifatturiere usano ancora per la contabilità e per la produzione. È un sistema che nessuno ha progettato pensando all'intelligenza artificiale, e che tuttavia risulta, contro ogni aspettativa, fra i più facili da integrare: il suo database è accessibile in SQL, e i programmi RPG o COBOL che vi girano sono invocabili per via programmatica senza doverne riscrivere una riga, attraverso stored procedure che li incapsulano, attraverso i servizi web che il sistema operativo sa generare da sé, oppure attraverso i componenti open source che la comunità mantiene da anni al fine di creazione di "backbone" di integrazione multilivello.

La catena completa, per quanto la si ami disegnare come una pila di livelli tecnologici, è in realtà molto semplice: il sistema con i suoi dati e i suoi programmi, sopra di esso una via

di accesso programmatica, sopra ancora il connettore che normalizza e protegge, poi il server che dichiara le capacità in forma tipizzata secondo il protocollo (MCP in questo caso), e infine l'applicazione agentica che le consuma. Il protocollo occupa soltanto il penultimo gradino perché definisce il canale, non costruisce la via di accesso, non decide chi possa usarla e non migliora di un grammo la qualità del dato che ci passa dentro.

Due punti, in questo percorso, meritano di essere decisi all'inizio l'identità con cui il connettore opera e la separazione (mandatoria) fra lettura e scrittura. Per il primo punto si può utilizzare un'utenza tecnica di servizio, oppure un'identità delegata, con la quale l'agente agisce per conto della persona collegata ed eredita i permessi che quella persona già possiede nell'applicativo. Per il secondo tema invece conviene adottare due profili distinti, e possibilmente due server distinti, perché il perimetro di ciò che si può leggere e quello di ciò che si può modificare non hanno alcuna ragione di coincidere, né sul piano del rischio né su quello delle approvazioni.

Nel nostro caso di riferimento, tutto questo si traduce in un esercizio di pragmatica concretezza: prima di intavolare qualunque discorso sugli agenti, occorre sapere quale vista del sistema contabile contenga i saldi che servono, con quale utenza la si possa interrogare, chi in azienda abbia il titolo di autorizzare quell'accesso, e se esista già una funzione che registri una rettifica oppure se quella funzione vada costruita. Sono quattro domande che non hanno nulla di innovativo o sexy, ed è esattamente per questo che si è reticenti a porsele.

2.4 Connettori e Model Context Protocol

Il MCP è stato introdotto da Anthropic nel novembre 2024 come standard aperto per collegare i modelli a dati e strumenti esterni, con l'obiettivo dichiarato di sostituire le integrazioni personalizzate punto- punto, che crescono in modo combinatorio con il numero di fonti dati e di applicazioni coinvolte, con un protocollo comune di tipo client-server. L'architettura va compresa distinguendo tre ruoli: l'host applicativo, cioè il programma con cui l'utente interagisce; il client MCP, integrato nell'host, che parla il protocollo; e il server MCP, che espone dati e capacità secondo tre primitive standard: - gli strumenti (tool) - le risorse (resource) - i prompt riutilizzabili

Vale la pena vedere come si presenta concretamente un server MCP, perché la sua semplicità spiega buona parte della sua diffusione. Esporre una funzione significa, in pratica, annotare una funzione che esiste già:

```
@mcp.tool()
def get_account_balance(bu_code: str, periodo: str) -> dict:
    """Restituisce il saldo di un conto per una BU e un periodo (AAAA-MM)."""
    row = erp.query(bu=bu_code, period=periodo)
    return {"saldo": row.amount, "valuta": row.currency,
            "aggiornato_al": row.updated_at}
```

Del resto si occupa il protocollo: quando un'applicazione compatibile si collega, chiede al server l'elenco delle capacità disponibili e riceve in risposta le firme dichiarate, dopodiché può invocarle. Lo scambio, nella sua forma essenziale, è questo:

```
→  {"method": "tools/list"}
←  {"tools": [{"name": "get_account_balance", "inputSchema": {...}}]}

→  {"method": "tools/call",
    "params": {"name": "get_account_balance",
               "arguments": {"bu_code": "IT-NORD", "periodo": "2026-03"}}}
←  {"content": [{"type": "text",
                 "text": "{'saldo': 1284300, 'aggiornato_al': '2026-04-02'}"}]}
```

Due dettagli meritano attenzione, perché sono quelli che nella pratica separano un progetto che funziona da uno che si arena. Il primo è il **trasporto**: un server MCP può girare in locale, sulla macchina dell'utente, comunicando attraverso i canali standard del sistema operativo, oppure essere esposto in rete su HTTP. Sono due mondi diversi per autenticazione, sicurezza e gestione operativa, e la confusione fra i due è all'origine di parecchie sperimentazioni che funzionano benissimo sul portatile di chi le ha costruite e si rivelano impraticabili quando devono servire quaranta persone. Il secondo è che "supportare MCP" significa cose diverse a seconda del lato: essere **client**, cioè consumare server altrui, ed essere **server**, cioè esporre le proprie capacità, sono capacità distinte, e un prodotto può avere l'una senza l'altra, come mostra il caso di Codex che richiameremo più avanti.

Resta a questo punto una domanda che rischia di generare confusione cioè chi debba scrivere questi server, e se per usare un'applicazione attraverso il protocollo sia necessario attendere che il suo produttore ne metta a disposizione uno.

Il client è la parte più semplice della vicenda perché è un componente dell'applicazione agentica, e lo realizza chi quell'applicazione la costruisce (sia esso un sw di mercato oppure un agent custom). Il server, invece, può avere quattro provenienze, ciascuna con le proprie verifiche da compiere.

Provenienza	Quando accade	Che cosa verificare
Il produttore dell'applicazione	sempre più spesso, ed è ormai prassi per i principali fornitori di software in cloud	che cosa esponga davvero (di norma la lettura è coperta, la scrittura molto meno), con quale identità operi, e se sia incluso nella licenza

Provenienza	Quando accade	Che cosa verificare
Il fornitore della piattaforma agentica	che costruisce il collegamento verso i servizi più diffusi e lo presenta come connettore	che il perimetro dichiarato corrisponda a quello di cui si ha effettivamente bisogno
Un terzo, spesso open source	esistono cataloghi pubblici con migliaia di server	provenienza, manutenzione, e chi ne abbia collaudato il codice
L'organizzazione stessa, o il suo partner di integrazione	quando il sistema è proprio, specifico, o semplicemente nessuno se ne è ancora occupato	soltanto il tempo di chi lo scrive

Il server non gira dentro l'applicazione di destinazione e non richiede il permesso di chi la produce: sta dalla parte di chi consuma, e dialoga con quell'applicazione attraverso la via di accesso che già esiste. Detto in una frase, il produttore controlla l'interfaccia di programmazione, non il protocollo. Se l'applicazione espone una via di accesso, il server lo si può scrivere, e per cinque o sei funzioni è lavoro di giorni, non di mesi; se non la espone, allora sì che si è in ostaggio, ma il blocco sta al livello precedente e il protocollo non c'entra.

La casella più delicata della tabella è la terza. Un server è software che opera con le credenziali dell'organizzazione verso un sistema dell'organizzazione, e installarne uno prelevato da un catalogo pubblico equivale, sul piano del rischio, a installare un componente con privilegi all'interno del perimetro aziendale, con un'aggravante specifica: le descrizioni degli strumenti entrano nel contesto del modello, e possono quindi essere scritte per indurlo a comportamenti non previsti. Valgono qui, identici, i criteri di provenienza, versionamento e collaudo già richiamati a proposito dei pacchetti. Nulla impone, del resto, di pubblicare un server che si è scritto: può vivere interamente dentro la rete aziendale, in un piccolo catalogo interno mantenuto dai sistemi informativi, che è il modello verso cui si stanno orientando le organizzazioni più strutturate.

Anche la seconda casella merita attenzione, per ragioni opposte. Un collegamento a una casella di posta elettronica, nella forma in cui è disponibile alla data di stesura presso uno dei principali fornitori, dichiara trenta strumenti: sei si limitano a leggere, gli altri ventiquattro modificano in qualche modo lo stato della casella, dall'etichettatura dei messaggi fino all'invio e all'inoltro. L'utente che accende quell'interruttore per "farsi riassumere la posta" ha in realtà concesso un perimetro assai più ampio di quello che aveva in mente, e nessuna schermata glielo ha fatto notare.

Lo standard aperto, infine, ribalta la relazione di forza fra chi produce un'applicazione e chi la usa, ma non la annulla. Il produttore continua a controllare l'interfaccia sottostante, e con

essa può cambiarne i formati, introdurre limiti di frequenza, o vietare nei termini contrattuali l'accesso automatizzato di terzi. Un server costruito su un'interfaccia che il fornitore non intende vedere usata in quel modo è tecnicamente possibile e contrattualmente fragile, ed è una verifica che conviene fare con chi si occupa dei contratti prima che con chi scrive il codice.

Tre precisazioni ulteriori sul versante operativo, tutte tratte dall'esperienza di questi anni. La prima riguarda le tre primitive citate poco sopra: gli **strumenti** sono funzioni che il modello decide di invocare, mentre le **risorse** sono dati che l'utente (o l'applicazione) indica esplicitamente, con un riferimento che li fa entrare nel contesto prima che il modello cominci a ragionare, alla stessa stregua di un file allegato. La differenza è sottile e pratica: ciò che si recupera a metà conversazione con una chiamata viene letto come l'esito di un'azione, mentre ciò che è presente fin dall'inizio entra nel ragionamento come premessa, e per un dato sempre necessario (lo schema di una tabella, la versione approvata di un budget) la seconda strada è più affidabile e meno dispendiosa.

La seconda riguarda il motivo per cui conviene un server rispetto alla scorciatoia di lasciare all'agente la riga di comando. Se l'agente interroga un servizio con uno strumento di trasferimento dati da riga di comando (ad esempio *curl*, per intenderci), tutto ciò che compone la richiesta, credenziali comprese, compare nel testo del comando, dunque nel contesto, dunque in ciò che viene inviato al servizio che ospita il modello: le credenziali attraversano un servizio di terzi, l'esito del comando è da interpretare come testo libero, il registro degli accessi richiede di analizzare stringhe di comando e i limiti di frequenza sono affidati al giudizio del modello. Con un server, nel contesto compaiono il nome dello strumento e i parametri, le credenziali restano nell'ambiente del server, che esegue la chiamata autenticata dall'interno della rete aziendale, rientra soltanto la risposta, le restrizioni (per esempio la sola lettura) sono imposte dal lato server e ogni richiesta è registrabile. Non leggerei questo come un questione di fiducia nel fornitore del modello ma come igiene architetturale: le credenziali appartengono all'ambiente dei server e non alle conversazioni, chiunque stia dall'altra parte. Ne discende anche un criterio per scegliere l'architettura di accesso, che consiste nel classificare prima i dati e decidere poi che "distribuzione prevederne" quelli più sensibili restano sul sistema locale o dietro un server controllato, i collegamenti con sistemi governati passano da server che applicano restrizioni e registrazioni, e per il materiale non sensibile (documenti pubblici, letteratura tecnica) si può anche caricare direttamente il testo. La classificazione determina l'architettura, e non il contrario.

La terza è un *failure event* che rischia di provocare imbarazzi in produzione. I collegamenti possono cadere nel corso di una sessione e quando cadono gli strumenti che fornivano scompaiono senza errore e senza avviso. Questo significa che l'agente non si ferma dichiarando di aver perso la funzione di lettura del gestionale, cerca altre strade, le imbocca con la solita sicumera di competenza e produce un risultato sempre più confuso. Lo stesso accade se un server non parte (una dipendenza mancante, una variabile d'ambiente sbagliata) e la sessione comincia senza i suoi strumenti. Nel nostro caso, il primo controllo di una chiusura automatica non riguarda il risultato ma l'equipaggiamento: una verifica all'avvio che tutti i server previsti siano raggiunti e dichiarino il numero atteso di strumenti, e un segnale esplicito se il comportamento dell'agente

cambia di colpo (per esempio se comincia a ricostruire a mano ciò che prima otteneva con uno strumento). Sono trenta secondi di verifica (magari anche forzando la scrittura di un "iso" log) che risparmiano un'ora di diagnosi.

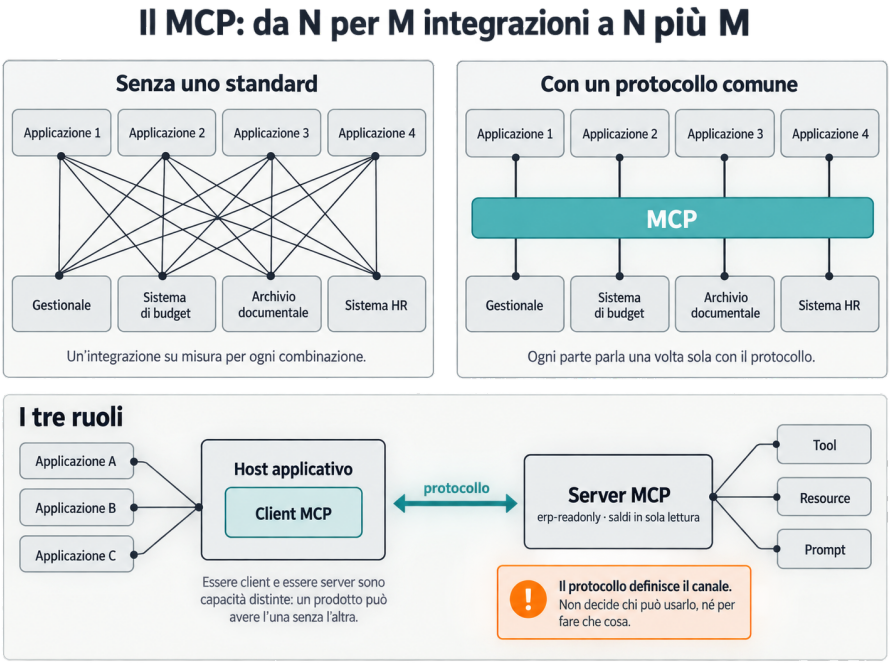


Figura 7: Il MCP: da N per M integrazioni a N più M

A fronte dell'adozione ufficiale del protocollo MCP da parte OpenAI, Google e Microsoft la specifica ha continuato a evolvere nel 2026 e gli aggiornamenti del 28 luglio 2026 (ultima major disponibile alla data di stesura del testo) introducono un'impostazione stateless più adatta al bilanciamento di carico su larga scala, un meccanismo di richieste multi round trip che sostituisce gli stream persistenti precedenti, e alcuni miglioramenti sull'autenticazione, con adozione riportata da fornitori cloud primari e da diverse aziende di prodotto. Non è corretto, in ogni caso, descrivere come pienamente retrocompatibile il ciclo di inizializzazione delle versioni precedenti rispetto a questa revisione: supportare MCP non significa implementarne tutte le estensioni o l'ultima revisione della specifica.

Applicato al nostro caso di riferimento, il valore del MCP si vede bene: un server MCP che espone le operazioni di consultazione del sistema gestionale e del sistema di budget può essere utilizzato, con gli opportuni permessi, da più applicazioni (esecuzioni?) agentiche diverse, dell'uno o dell'altro fornitore, senza che occorra costruire un'integrazione separata per ciascuna. Concretamente, quel server dichiarerebbe uno strumento con un nome esplicito, per esempio `get_account_balance`, con parametri tipizzati come `bu_code` e `periodo`, e restituirebbe non un testo libero ma un valore numerico accompagnato dalla data dell'ultimo aggiornamento del dato sottostante: è un contratto verificabile tra il server e chi lo interroga, non un'istruzione in linguaggio naturale da interpretare volta per volta, ed è proprio questa tipizzazione a permettere a un controllo programmatico, di cui parleremo più avanti in questa stessa parte, di verificare che il valore restituito corrisponda davvero al sistema di origine. Il MCP non risolve ovviamente

i problemi di qualità del dato, di identità e di permessi: definisce il canale, non decide chi può usarlo e per fare che cosa, decisioni che restano di competenza dell'organizzazione.

Conviene chiudere questa parte sciogliendo un equivoco che la catena che abbiamo descritto parlando di ciò che sta sotto il connettore rischia di alimentare. Sistema di origine, via di accesso, connettore, server, applicazione agentica: cinque gradini, disegnati uno sopra l'altro che non rappresentano (necessariamente) sistemi da acquistare, installare e mantenere. Sono livelli concettuali non necessariamente componenti sw distribuiti.

Il connettore, in particolare, non è quasi mai un servizio a sé che si installa da qualche parte ma è codice che sta dentro il server MCP e incapsula la funzione che apre la connessione, si autentica, chiama, traduce l'errore e normalizza la risposta. Quando si dice "scriviamo il connettore verso il gestionale e poi ci costruiamo sopra il server", nella realtà si sta scrivendo un solo programma, in cui un modulo parla con il gestionale e le funzioni dichiarate costituiscono gli strumenti. Lo stesso vale per il gradino della via di accesso, che spesso esiste già: una vista in un database, una stored procedure, un'interfaccia che il gestionale espone da anni ad altri sistemi, per altre finalità.

Inoltre, pubblicare un'interfaccia intermedia fra il database e il server non è di per sé un passaggio obbligato. Se l'unico consumatore di quel dato è l'agente, il server può parlare direttamente con il database attraverso il suo driver, invocando le stored procedure già esistenti ed interporre un'interfaccia ulteriore aggiunge solo un servizio da mantenere, un insieme di credenziali da ruotare e un punto in più in cui le cose si rompono, e si giustifica soltanto quando esistono (o sono previsti) altri consumatori dello stesso dato.

Un esempio concreto, adatto a un contesto aziendale comune, è quello di un gestionale in cloud che espone le proprie interfacce (ad esempio Business Central, per restare nel mondo Microsoft), di un database su cui gira la reportistica direzionale (un SQL Server con Power BI), e di un archivio di documentazione non strutturata da cui l'agente dovrebbe ricavare informazioni utili a fornire commenti ai risultati. Ciò che si distribuisce effettivamente può essere molto meno di cosa a prima battuta si sarebbe portati a credere.

Componente	Nuovo?	Che cosa è, fisicamente
Gestionale	esistente	un servizio del fornitore; non si installa nulla, si ottengono credenziali
Database della reportistica, con viste e stored procedure	esistente, da arricchire	oggetti dentro il database, non un servizio nuovo

Componente	Nuovo?	Che cosa è, fisicamente
Archivio vettoriale per il recupero documentale	**nuovo**	un database dedicato, più una procedura che spezza i documenti, ne calcola le rappresentazioni e li indicizza
Server MCP	**nuovo**	uno o più processi, tipicamente in container, con un indirizzo e un'autenticazione
Applicazione agentica	esistente o acquistata	il prodotto che ospita il client

Il server, dunque, è semplicemente un processo nel senso di un programma in esecuzione su una macchina, con un eseguibile, una configurazione, delle credenziali e dei log, esattamente come qualunque altro servizio interno. Da subito è importante ragionare su di un paio di questioni fondanti (e che come tali meritano di essere smarcate in anticipo). La prima è se costruirne uno solo o uno per sistema, e la risposta ragionevole è quasi sempre la seconda, perché cicli di vita, credenziali e permessi dei diversi sistemi non hanno ragione di coincidere, e perché la possibilità di spegnere un collegamento senza spegnere gli altri è precisamente ciò che la governance richiede. La seconda riguarda il luogo in cui il server gira: deve stare dove sta il dato, cioè nella rete che raggiunge il database, e se l'applicazione agentica è un servizio in cloud occorre stabilire fin da subito come quel servizio possa raggiungerlo senza esporre ciò che non va esposto. È il punto in cui i progetti rischiano di schiantarsi se non progettato anticipatamente.

Non va trascurato, infine, un dettaglio che l'esempio fa emergere: un archivio documentale indicizzato senza portare con sé i permessi di lettura dei documenti di origine restituirà tranquillamente il paragrafo di un documento che chi interroga non avrebbe titolo per leggere. I permessi vanno portati dentro l'indice e applicati al momento del recupero, non aggiunti dopo.

2.5 I file di configurazione in Markdown (od in altro formato testo)

Un elemento tecnico ma concreto sono i file di testo semplice, (ad esempio in formato Markdown), che guidano il comportamento dell'agente in un dato progetto o ambiente. Il Markdown è, di per sé, un formato di testo neutro: il suo significato operativo dipende dal nome del file, dalla sua posizione, e dal prodotto specifico che lo carica, non ogni file con estensione `.md` diventa automaticamente un'istruzione eseguita dall'agente.

In Codex, l'agente per lo sviluppo di OpenAI, la scoperta delle istruzioni di progetto avviene attraverso un file `AGENTS.md`, affiancato in alcuni casi da un file `AGENTS.override.md` per le eccezioni locali . In Claude Code, l'agente per lo sviluppo di Anthropic, il file principale è `CLAUDE.md`: Claude Code non legge direttamente un file `AGENTS.md` come proprie istruzioni native, ma può importarlo indirettamente se richiamato esplicitamente da `CLAUDE.md`. Le gerarchie dei due prodotti, insomma, non vanno presentate come identiche solo perché entrambe si basano su un file Markdown alla radice del progetto, un dettaglio facile da semplificare in modo scorretto in una trattazione divulgativa. Concretamente, un estratto di un file di questo tipo per il nostro caso di riferimento potrebbe contenere righe come "usa sempre il formato data GG/MM/AAAA nei documenti destinati alla direzione", "la soglia di materialità è definita nel file `soglie.md`, non va mai stimata a occhio", "non diffondere il pacchetto direzionale senza l'approvazione esplicita del controller": frasi semplici, dirette, più vicine a una nota operativa lasciata a un collaboratore che a una configurazione software in senso tradizionale, ed è proprio questa leggibilità a renderle uno strumento che un responsabile di funzione, non solo un tecnico, può scrivere e mantenere aggiornato.

Un estratto realistico, nel nostro caso di riferimento, avrebbe questa forma:

```
# Istruzioni di progetto - Controllo di gestione

## Convenzioni
- Formato data GG/MM/AAAA nei documenti destinati alla direzione
- Valuta EUR, arrotondamento al migliaio nelle tabelle di sintesi

## Regole non negoziabili
- La soglia di materialità è definita in `soglie.md`. Non stimarla mai.
- Non diffondere il pacchetto direzionale senza approvazione del controller.

## Fonti autorizzate
- Saldi: server MCP `erp-readonly` (sola lettura)
- Budget: cartella `/budget/approvato/`, solo l'ultima versione approvata
```

Un aspetto rilevante, quanto meno sulla base della mia esperienza pratica, è che questi file esistono di norma su **più livelli che si sommano invece di sostituirsi**: un livello aziendale, valido per tutti i progetti, uno di progetto, versionato insieme al progetto stesso, e uno personale, con le preferenze del singolo, non condivise. La conseguenza organizzativa è notevole: il file di progetto, essendo versionato, è l'unico punto in cui la procedura aziendale diventa leggibile, discutibile e approvabile con gli stessi strumenti con cui si approva una modifica a un sistema, con una storia delle revisioni e un autore per ciascuna. Chi governa il controllo di gestione dovrebbe pretendere che quel file fosse trattato come un documento normativo interno, non come un appunto tecnico lasciato a discrezione di chi ha configurato l'agente.

Il punto concettuale più importante, che vale a prescindere dal singolo prodotto, è distinguere ciò che un file di istruzioni **può fare** da ciò che deve invece essere fatto altrove. Nel nostro caso di riferimento, un'istruzione come "non diffondere il pacchetto direzionale senza l'approvazione del controller" appartiene correttamente a un file di questo tipo, letto e rispettato dall'agente nella normale conduzione del compito; ma il blocco effettivo, quello che impedisce l'invio non autorizzato anche nel caso in cui l'agente, per un errore di ragionamento o per un tentativo di manipolazione esterna, provasse comunque a farlo, deve vivere nei permessi del sistema che gestisce l'invio e non nel solo file di istruzioni. Un'istruzione testuale, per quanto ben scritta, resta una **raccomandazione** rivolta a un sistema probabilistico che la interpreterà pertanto in questa modalità "stocastica", non un vincolo tecnico assoluto.

2.6 Contesto, memoria e RAG

Tre nozioni che nel linguaggio comune vengono spesso confuse meritano di essere tenute separate per i gradi di complessità (anche di gestione) che implicitano. Il **contesto** è l'insieme delle informazioni disponibili al modello durante un singolo passaggio di lavoro. La **memoria** mantiene informazioni tra elaborazioni o sessioni diverse, secondo meccanismi specifici del prodotto utilizzato. Il recupero documentale, comunemente indicato con l'acronimo **RAG** (retrieval-augmented generation), seleziona materiale esterno pertinente da usare nella risposta o nel lavoro corrente. Di nuovo, **nessuno di questi tre meccanismi equivale ad addestrare il modello**: sono tutti modi di fornire, in momenti diversi, informazioni a un modello che resta lo stesso (cioè che mantiene inalterato il sistema di pesi delle reti che lo determinano). La disciplina che si occupa di decidere quali istruzioni, documenti, risultati intermedi e strumenti rendere disponibili al momento opportuno prende il nome di *context engineering*, un termine relativamente recente ma già consolidato nella letteratura tecnica. Un tema logicamente afferente è quello della **compattazione**, ossia la sintesi della cronologia di un compito lungo per proseguire entro i limiti di contesto disponibili, che va progettata avendo ben chiaro quali dettagli non ci si può permettere di perdere lungo la strada. Nel nostro caso di esempio, il valore di questa disciplina è immediato: per calcolare correttamente uno scostamento serve il budget nella sua versione approvata più recente, non una versione precedente ancora presente, magari per inerzia, nell'archivio documentale condiviso. Un archivio più grande non produce necessariamente un'analisi migliore se fa riemergere condizioni ormai superate o peggio ancora contraddittorie perché la **qualità** del contesto conta più della sua **quantità**. Un esempio concreto di che cosa la memoria conserva effettivamente, da un mese all'altro, aiuta a rendere la distinzione meno astratta: una nota di questo tipo, salvata dall'agente al termine della chiusura di marzo, potrebbe leggere "la soglia di materialità per la BU Nord è stata aggiornata a 15.000 euro con delibera del 3 marzo 2026, in sostituzione della soglia precedente di 10.000 euro; applicare la nuova soglia a partire dalla chiusura di aprile". È un'informazione puntuale, datata e verificabile, non un riassunto vago della conversazione precedente: una memoria ben progettata conserva fatti di questo tipo, con la loro data di validità, non impressioni romanzate sull'attività passata.

Conviene chiarire come una memoria funzioni davvero perché non si tratta di un esoterico stato interno del modello, ma di un molto essoterico insieme di file che l'agente ha il permesso di leggere e riscrivere attraverso strumenti dedicati (crea, leggi, sostituisci una porzione, cancella), conservati in una directory riservata:

```
memoria/
├── definizioni-kpi.md      # "EBITDA rettificato: esclude oneri non
↪  ricorrenti..."
├── soglie-e-delibere.md    # "BU Nord: soglia 15.000 € dal 03/03/2026"
└── verifiche-aperte.md     # "Riconciliazione intercompany IT-DE: aperta da
↪  aprile"
```

Il fatto che si tratti di file ordinari ha tre conseguenze pratiche che un manager di business (magari il responsabile amministrativo per rimanere nel nostro esempio) apprezzerà più di qualunque discorso sull'intelligenza artificiale: la memoria è **ispezionabile**, si può cioè leggere che cosa l'agente crede di sapere, ed è un esercizio che consiglio di fare almeno una volta perché è spesso istruttivo; è **correggibile**, si può riscrivere una riga sbagliata senza rifare nulla e senza riaddestrare alcunché; ed è **cancellabile in modo verificabile**, il che non è affatto un dettaglio secondario quando la stessa memoria contiene, come quasi sempre accade, dati riferibili a persone identificabili.

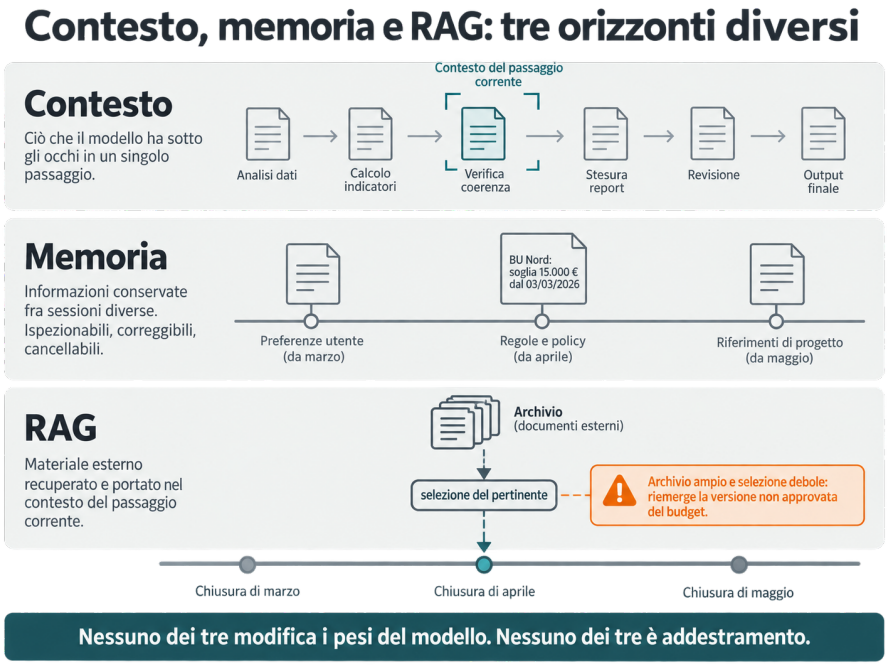


Figura 8: Contesto, memoria e RAG: tre orizzonti diversi

2.7 Il costo del contesto: che cosa è sempre acceso e che cosa si carica a richiesta

Il contesto non è soltanto un concetto ma una risorsa finita che i meccanismi che abbiamo incontrato (istruzioni persistenti, skill, strumenti, subagenti, hook) utilizzano in modo diverso.

Quando la finestra si riempie la prestazione non peggiora con gradualità ma crolla ed il sintomo non è un errore ma un comportamento: l'agente ripete suggerimenti già dati, dimentica istruzioni impartite pochi minuti prima, risolve un problema già risolto, lascia cadere un vincolo (non tutti, quanto basta perché il risultato sia sottilmente sbagliato). E' utile quindi ripetersi, ad ogni occasione, il mantra qui sotto tabellato.

Meccanismo	Quando entra nel contesto	Quanto costa
Istruzioni persistenti	all'avvio, per intero	a ogni richiesta, finché la sessione dura
Skill	all'avvio la sola descrizione, il corpo quando serve	poco a riposo, il necessario quando si attiva
Strumenti (server MCP)	all'avvio, tutte le dichiarazioni (salvo il caricamento differito)	a ogni richiesta
Subagenti	in un contesto proprio, alla chiamata	isolato: rientra soltanto il risultato
Hook	non entrano, perché girano fuori dal contesto	nullo, a meno che restituiscano testo da mostrare all'agente

La regola di classificazione che ne discende è lineare. Ciò che serve in ogni passaggio (le convenzioni, le soglie, le regole non negoziabili) va nelle istruzioni persistenti; ciò che serve per un tipo di compito (la procedura di chiusura, il metodo di riconciliazione) va in una skill; ciò che comporta molta lettura o molto output (analizzare cinquanta file, scorrere un registro) si delega a un subagente, che assorbe il volume e restituisce la sintesi; l'accesso strutturato a un sistema esterno va in un server; ciò che deve essere imposto meccanicamente va in un hook, che è il meccanismo più economico in assoluto in termini di contesto. Sbagliare in un verso o nell'altro costa: mettere materiale di consultazione fra le istruzioni persistenti fa pagare ogni richiesta, mentre mettere regole sempre valide in una skill significa che saranno violate ogni volta che la skill non è caricata.

Un corollario riguarda la lunghezza del file di istruzioni. Oltre una certa soglia (nell'ordine di poche centinaia di righe, secondo le indicazioni degli editor stessi degli strumenti) le istruzioni cominciano a competere fra loro: il modello non le ignora, ne diluisce l'attenzione, e la regola importante finisce per pesare quanto quella marginale. Il criterio di selezione non è «che cosa deve sapere l'agente» ma «che cosa non può scoprire da solo»: le convenzioni diverse da quelle ordinarie, le decisioni e i loro motivi, le stranezze dell'ambiente (per esempio che un certo campo di una tabella è vuoto per i movimenti anteriori a una data a causa di un errore di migrazione), le cautele che l'esperienza ha mostrato necessarie. Va invece lasciato fuori ciò che l'agente ricava leggendo i dati, le convenzioni che già conosce, la documentazione minuta (a cui si rinvia con un riferimento che apre a richiesta) e le informazioni che cambiano di frequente. Se l'agente continua a violare una regola nonostante sia scritta, le spiegazioni possibili sono due: il file è troppo lungo

e la regola si è persa, oppure quella regola non dovrebbe essere un testo ma un controllo (un permesso, un hook), e la seconda spiegazione è più frequente di quanto si pensi.

La compattazione, cioè la sintesi della cronologia quando la finestra è quasi piena, interviene nei prodotti documentati attorno al novantacinque per cento della capacità e comprime con perdita: i risultati letti venti passaggi prima, le uscite dei comandi, il ragionamento intermedio diventano un riassunto. Se ne governano tre aspetti. I vincoli che non si possono perdere si collocano in una sezione del file di istruzioni che la compressione preserva; si può comprimere prima (a ottanta per cento, per esempio), indicando che cosa privilegiare; si può condensare in modo selettivo una porzione di conversazione (l'esplorazione chiusa in un vicolo cieco), lasciando intatto il lavoro produttivo. E per i lavori che durano più di due o tre sessioni si tiene su file un documento di specifica (nel nostro caso, il piano della chiusura, con perimetro, criteri, ordine ed eccezioni note), che sopravvive alla compattazione e al riavvio e a cui ogni nuova sessione si ricollega senza che nessuno debba spiegare daccapo. Un segnale pratico: quando l'agente comincia a chiedere ciò che gli si è già detto o a riproporre ciò che si è scartato, ripetere le istruzioni peggiora la situazione (accelera la saturazione), e la mossa giusta è ripartire da una sessione nuova con un buon file di istruzioni. Per la stessa ragione, tenere insieme nella stessa sessione compiti non collegati degrada il compito originario: un contesto pulito per ciascun compito rende di più.

Un'avvertenza complementare riguarda la parte del contesto che è per sua natura dinamica. Le istruzioni persistenti sono statiche fra una modifica e l'altra, ma lo stato della chiusura in corso (quali business unit hanno consegnato, quali riconciliazioni sono aperte, che cosa è cambiato dall'ultima esecuzione) cambia ogni giorno. Conviene iniettarlo all'avvio con un programma che lo calcola una volta per sessione, invece di scriverlo in un file che invecchierebbe subito: i due canali si completano, il file porta le regole che non cambiano e l'avvio porta la situazione del momento.

2.8 Sistemi multi agente: coordinamento, non solo parallelismo

Quando un compito è sufficientemente complesso da beneficiare della divisione del lavoro, entrano in gioco i sistemi multi agente . Come il nome stesso dice intendiamo con questo termine più agenti, spesso specializzati su una specifica funzione/attività o su di una porzione del problema, che collaborano secondo pattern ormai ricorrenti, come l'orchestratore che delega a sub agenti (subagent nella terminologia più diffusa, handoff quando si tratta di un vero e proprio passaggio di responsabilità da un agente a un altro), o l'esecuzione parallela seguita da una fase di sintesi. Anthropic ha documentato in modo dettagliato un caso di questo tipo nel proprio sistema di ricerca multi agente, in cui la divisione del lavoro tra agenti specializzati e la successiva sintesi hanno prodotto risultati superiori a un singolo agente generalista sullo stesso compito.

È interessante come, l'emergere e la diffusione di uso di queste modalità "multi", stia facendo emergere problemi di coordinamento e fenomeni di "conformismo operativo" mostrando che agenti impostati in modo simile tendono a replicare gli stessi errori invece di correggersi a vicenda. È un risultato circoscritto a pochi esperimenti ma indebolisce un'intuizione diffusa e pericolosa, quella

per cui il consenso tra più agenti equivalga automaticamente a maggiore affidabilità. Nel nostro caso di riferimento, tre sub agenti che validano in modo concorde lo stesso scostamento di budget, se impostati in modo simile e senza controlli indipendenti tra loro, non triplicano l'affidabilità del giudizio: possono, al contrario, mascherare un errore sistematico condiviso nella fonte dei dati, per esempio un valore di budget non aggiornato che tutti e tre gli agenti hanno consultato allo stesso modo. La revisione indipendente, va sottolineato, non è la stessa cosa del semplice consenso tra agenti impostati in modo analogo.

Vale la pena, sempre in tema di coordinamento, distinguere i diversi modi di coinvolgere un secondo agente (spesso confusi nella comunicazione divulgativa alla "tik-tok") da un lato abbiamo l'handoff, cioè un vero e proprio passaggio di responsabilità da un agente a un altro, che da quel momento dirige autonomamente il seguito del lavoro, dall'altro la semplice invocazione di un agente specializzato come se fosse uno strumento, in cui il coordinatore mantiene la responsabilità complessiva e si limita a usare il risultato prodotto dallo specialista. Nel nostro caso di riferimento, la differenza è tutt'altro che astratta: chiedere a un sub agente una verifica locale sui dati di una country, e poi valutarne il risultato prima di includerlo nel consolidato, è cosa diversa dal trasferirgli, con un handoff, la piena responsabilità di decidere se quel dato sia pronto per la diffusione.

Anche i subagenti, nella pratica, si dichiarano come semplici file di testo, e la loro definizione dice molto sul modello mentale sottostante:

```
---
name: verificatore-country
description: Verifica la coerenza dei dati di una country prima del consolidato.
  Usare al termine di ogni estrazione locale.
tools: [get_account_balance, get_budget]   # perimetro ristretto, sola lettura
maxTurns: 15                               # tetto ai giri: nessuna esplorazione
↪  senza fine
---
Verifica che i saldi estratti corrispondano alla fonte, che il periodo sia
quello richiesto e che non manchino conti rispetto al mese precedente.
Segnala le anomalie, non correggerle.
```

Si noti la riga degli strumenti, che è la parte concettualmente più densa dell'esempio: il subagente riceve un perimetro **più stretto** di quello del coordinatore. È il modo corretto di intendere la specializzazione in questo contesto, e va contro l'intuizione comune: un subagente non è "più bravo" su un tema, è solo più specializzato e pertanto più limitato, perché dispone di meno strumenti, di meno contesto e di un compito più definito, e proprio per questo produce risultati più verificabili. L'ultima riga dell'esempio, "segnala le anomalie, non correggerle", è poi la traduzione operativa del principio di separazione fra chi rileva e chi decide, che in amministrazione conosciamo da molto prima che esistessero gli agenti e che qui ritorna, identico, in veste tecnica. La riga `maxTurns`aggiunta nell'esempio si comporta come in un sistema elettrico si comporterebbe

un fusibile: un subagente lasciato libero di esplorare tende a imboccare ricerche sempre più laterali e a bruciare risorse finché il contesto non si esaurisce. Mettendo un numero massimo ai giri lo si costringe a restituire un risultato. Vale poi la pena di vedere come istruzione, perimetro e controllo si sommano in un esempio di difesa in profondità. Un subagente di lettura dei dati porta l'indicazione «sola lettura» (che dichiara l'intenzione); non dispone di alcuno strumento di scrittura (che rende la violazione impossibile nel perimetro previsto) e porta con sé un hook che intercetta qualunque comando contenga operazioni di modifica prima che sia eseguito (che la fa rispettare anche se qualcuno, per errore, aprisse un varco altrove). Tre livelli diversi della catena, sovrapposti, per un'unica regola.

Prima di moltiplicare gli agenti conviene chiedersi se ce ne sia bisogno ed al di la della *sexyness* del poter dire al vicino di capannone che si ha un "sistema multiagente", molto più spesso di quanto si creda un agente solo, ben istruito e dotato di criteri di verifica, ottiene risultati migliori di un sistema multiplo. Questo (anche ma non solo) perchè il coordinamento non è gratuito, introduce modalità di fallimento che l'agente singolo non ha, e se il compito entra in un solo contesto con ampio margine non c'è nulla da dividere. La gerarchia di preferenza, che ricavo dall'esperienza (limitata perchè di fatto c'è poca di sta roba realmente in produzione, oggi), è netta: un agente solo come punto di partenza; i subagenti quando serve isolare il contesto (e soltanto in secondo luogo parallelizzare); le squadre di agenti che si parlano fra loro quando il lavoro richiede un coordinamento prolungato fra specialisti che mantengono ciascuno una conoscenza propria. Si sale di un gradino soltanto con l'evidenza che quello sotto non basta. L'agente singolo prevale nei compiti che stanno in una finestra con ampio margine, nelle modifiche strettamente accoppiate (se ogni parte dipende da ogni altra, parallelizzare crea problemi di integrazione che l'esecuzione in sequenza evita), quando non si riesce a specificare il perimetro e le interfacce dei sotto-compiti (ne seguirebbero sovrapposizioni) e nei problemi privi di precedenti, dove gli agenti in parallelo divergono in modi incompatibili. Il test è semplice: se si riesce a scrivere un piano che scomponga il lavoro in compiti indipendenti con confini chiari, la divisione aiuterà; se si fatica a scrivere quel piano, si comincia con un agente solo e si riconsidera più tardi. Nel nostro caso di riferimento, le verifiche locali di ciascuna country, indipendenti fra loro, si prestano alla divisione; la riconciliazione intercompany, in cui ogni dato dipende da quello della controparte, molto meno.

Il valore principale dei subagenti, a questo punto, non è la velocità ma l'isolamento: un subagente può leggere cinquanta file, scorrere un registro di migliaia di righe, esplorare un'intera cartella, e il contesto del coordinatore vede soltanto il riassunto. Per questo gli utenti esperti li usano anche quando non c'è parallelismo da guadagnare. Il prezzo è che il riassunto è necessariamente "meno ricco informativamente" ma che al coordinatore tornano comunque dei risultati: se sono verbosi, occupano proprio il contesto che l'isolamento doveva proteggere, ed è il paradosso per cui dieci subagenti che restituiscono ciascuno un rapporto completo lasciano al coordinatore meno spazio di quanto ne avrebbe avuto senza subagenti. Vanno dunque istruiti a rispondere in modo specifico invece di redigere rapporti omnicomprensivi, oppure a scrivere il dettaglio su un file e a restituire soltanto il percorso con due righe di sintesi, in modo che il

coordinatore legga di quel file soltanto ciò che gli serve.

Chi decide di parallelizzare dovrebbe farlo soltanto dopo aver scritto il piano. Senza di esso ogni agente interpreta l'obiettivo per conto proprio, fa assunzioni diverse e produce risultati in conflitto; il piano costa poco (migliaia di token) rispetto a un'esecuzione parallela di ore (centinaia di migliaia o milioni di token, tanto per darci un ordine di grandezza), e deve specificare quattro cose: i confini dei compiti (chi possiede quali file o quali business unit), i contratti di interfaccia (che formato deve avere il risultato di ciascuno, perché il consolidato possa riceverlo), l'ordine di dipendenza (che cosa deve finire prima di che cosa) e i criteri di accettazione (quando un agente può dire di aver finito). È la stessa disciplina con cui un controller dispone un consolidato su più persone. La stessa analogia suggerisce la regola economica: un orchestratore capace, e più costoso, per pianificare e valutare, ed esecutori più leggeri ed economici per il lavoro di volume, secondo una ripartizione che ritroveremo nella parte sui costi e che ricorda quella fra chi disegna e chi esegue in uno studio professionale.

C'è infine un'avvertenza sul modo di definire i ruoli, che discende da un caso documentato e secondo me molto istruttivo. Un professionista che aveva affidato a quattro agenti con ruoli gerarchici (un amministratore, un consulente per il rischio, un tecnico, alcuni agenti di strategia) la gestione di un portafoglio simulato ha visto il sistema paralizzarsi dopo un minimo successo. L'agente consulente, che interpretava il proprio ruolo secondo ciò che i canoni di addestramento trovava rischioso ogni piano; l'amministratore si rimetteva al consulente; il tecnico non trovava nulla da realizzare, perché tutto veniva respinto. Il sistema è stato infine ricondotto a un agente solo con istruzioni minime, e ha funzionato meglio. La lezione, che si collega a quanto detto sul conformismo operativo (qui con segno opposto: non errori replicati, ma deferenza e blocco), è che un ruolo definito come personalità eredita le inclinazioni che quella personalità ha nei testi da cui il modello ha imparato, e queste interagiscono in modi non progettati. Il rimedio è definire i ruoli per compito e non per carattere: «verifica i saldi di questa country rispetto alla fonte e segnala le anomalie» invece di «sei lo scrupoloso verificatore dei conti». La prima è una consegna, la seconda è un personaggio, e i personaggi hanno opinioni che nessuno ha chiesto. L'esempio del subagente riportato sopra, con la sua ultima riga, è già costruito in questo modo.

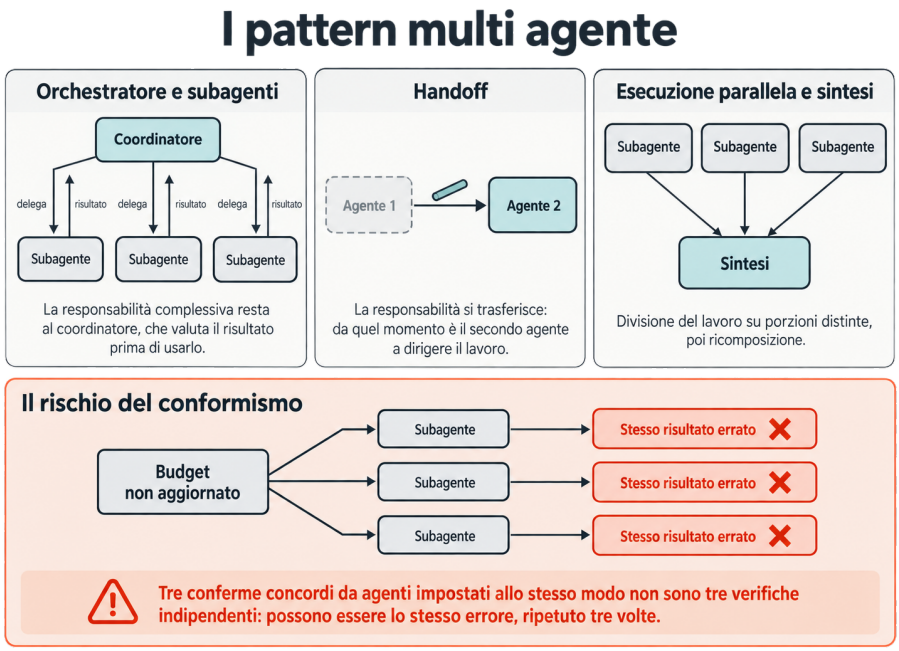


Figura 9: I pattern multi agente

Se dovessimo indicare, oggi, l'ambito applicativo in cui gli agenti (singoli o multi che siano) hanno raggiunto la maggiore maturità e la più vasta adozione professionale, sarebbe senza dubbio quello dello sviluppo software. Un agente per il coding può leggere l'intero contesto di un progetto, modificare più file in coerenza tra loro, eseguire test, e proporre una modifica pronta per la revisione umana, non semplicemente completare una riga di codice come facevano gli strumenti di una generazione precedente. È un caso di studio interessante perché è al tempo stesso il terreno di sperimentazione più avanzato e quello più facile da comprendere per analogia: un agente per il coding somiglia, nel modo in cui viene introdotto in un team, più a un collaboratore junior a cui si assegna un perimetro definito e che si rivede prima di dare il proprio benestare, che a uno strumento software tradizionale.

2.9 La verifica dell'esito: perché "ho fatto X" non basta

A mio giudizio, uno dei temi più rilevanti nella trattazione generalista sugli agenti è quello della verifica del risultato. Un agente, come qualunque sistema che riporta il proprio operato attraverso il linguaggio, può descrivere un'azione in un modo che non corrisponde esattamente a ciò che è realmente accaduto nell'ambiente su cui ha operato: non per malafede, ma per la natura stessa di un sistema che genera testo per descrivere le proprie azioni. Anthropic distingue esplicitamente, in un proprio approfondimento tecnico, ciò che un agente dichiara di aver fatto da ciò che è effettivamente accaduto, e descrive tre approcci complementari per colmare questa distanza: i controlli programmatici, che verificano lo stato del sistema in modo automatico e oggettivo; le valutazioni condotte da un modello terzo, che esaminano il lavoro prodotto con un giudizio più flessibile di un controllo puramente meccanico; e il giudizio umano, indispensabile per i casi in cui il criterio di correttezza non è del tutto codificabile.

L'agente, ampliando il proprio raggio d'azione per il tramite dei diversi cicli che attraversa, amplifica anche l'effetto che conosciamo (grazie ad Ethan Mollick) come *Jagged Frontier*, ovverosia la discontinuità della prestazione di questi sistemi, che eccellono in attività complesse e falliscono in altre che a noi paiono elementari. Con gli agenti anche un errore banale non resta confinato in una risposta di una chat isolata, ma si propaga lungo una sequenza di azioni. Da qui una regola di metodo che contraddice entrambe le reazioni istintive: non si adotta un agente perché un parametro di riferimento pubblicato è alto, e non lo si esclude perché ha sbagliato un'esecuzione. Si costruisce piuttosto una mappa di affidabilità, compito per compito, che tenga conto del contesto, degli strumenti disponibili e del livello di autonomia concesso. Un modo "pratico" (cioè sempre teorico ma comunque affascinante) di costruirla consiste nel trattare il sistema come un candidato e nel sottoporlo a prove aderenti al lavoro reale invece che a descrizioni generiche delle sue capacità: istruzioni incomplete, dati sporchi, eccezioni, vincoli di formato, cioè precisamente le condizioni in cui il lavoro si svolge davvero. Nel nostro caso di riferimento, la prova utile non è la chiusura di un mese ordinario, ma quella del mese in cui una country ha consegnato in ritardo, un conto è stato riclassificato e il budget di riferimento è stato aggiornato a metà periodo.

Nel nostro caso di riferimento, questo si traduce in un principio molto concreto: la frase "ho aggiornato il pacchetto di reporting" restituita dall'agente non è, di per sé, una prova di nulla. Occorre controllare che i numeri riportati corrispondano effettivamente ai saldi di origine nel sistema gestionale, che nessuna business unit sia stata omessa dal consolidato, e che le note di commento raccolte siano state attribuite alla voce corretta. Per rendere questo principio operativo ed operazionale si potrebbe pensare di definire una dashboard di misurazione aziendale per qualunque processo delegato in parte od in toto ad un agente: il tasso di risultati accettati sul totale prodotto, il tasso di errori materiali rilevati, la frequenza degli interventi correttivi umani richiesti, il tempo complessivo impiegato, comprensivo della revisione, il costo per risultato effettivamente accettato, e il numero di incidenti registrati nel periodo osservato. Nessuna di queste misure sostituisce le altre ma insieme, restituiscono un quadro molto più onesto di quanto non faccia il solo entusiasmo per la demo iniziale.

Tre ulteriori osservazioni, desunte soprattutto dall'esperienza degli agenti per lo sviluppo del codice (dove la verifica ha un vantaggio, perché i test passano oppure non passano), valgono tuttavia anche per qualsiasi processo aziendale. La prima è che la sicurezza con cui un agente presenta un risultato non porta alcuna informazione sulla sua correttezza: la fiducia espressa è massima tanto quando il risultato è giusto quanto quando è sbagliato, e chi si lascia guidare dal tono invece che dall'accertamento finisce per fidarsi dei risultati sbagliati con la stessa serenità con cui si fida di quelli giusti. La seconda è che un esito corretto non prova un processo corretto. Gli esiti fortunati e quelli buoni sono indistinguibili finché la fortuna dura (con la differenza che l'errore di codice si scopre al primo test, mentre l'errore di strategia si scopre quando si è già fatto un danno). Nel nostro caso, un pacchetto di reporting che quadra perché due errori si sono compensati è un pacchetto che ha superato la verifica pur essendo sbagliato. La terza è che la profondità della revisione va commisurata alla posta in gioco: un'occhiata a ciò che si ricostruisce facilmente, un esame riga per riga sulla logica che determina i numeri direzionali. Chi rivede

tutto con la stessa intensità butta via il vantaggio dell'agente, chi non rivede nulla smette di capire il processo.

C'è poi un'asimmetria che conviene sfruttare. Nei compiti dotati di criteri di verifica automatici (i test per il codice, le quadrature, le riconciliazioni, i controlli di coerenza fra saldo e consolidato) l'agente riceve un riscontro a ogni passo e corregge da sé, tanto che la qualità del suo lavoro dipende in misura notevole da quanto sono stringenti i controlli che lo circondano, anche quando nessuno gli ha mai descritto quei controlli. È la ragione per cui, in un progetto di sviluppo, **la configurazione degli strumenti di controllo del codice fa parte delle istruzioni che l'agente riceve**. Ne discende un criterio da applicare quasi in chiave "economica": ogni controllo automatico che si aggiunge a un processo delegato non protegge soltanto dagli errori umani, protegge anche da quelli dell'agente, e libera il tempo di revisione per ciò che nessun controllo meccanico sa giudicare (la ragionevolezza di una spiegazione, la coerenza con la strategia, il gusto). Si ha infatti a disposizione un budget di attenzione limitato, e spenderlo a correggere il formato di una data è uno spreco che si paga in giudizio là dove il giudizio serve.

Un ordine di grandezza aiuta infine a calibrare le aspettative. Dopo aver interagito per oramai quasi un anno con questi oggetti posso affermare che circa un compito su tre riesce al primo tentativo senza ulteriori indicazioni. Per me un tasso di successo (o fallimento) come questo non è un difetto da correggere ma una proprietà intrinseca del lavoro iterativo. La metrica che conta non è il tasso di successo al primo colpo, è il tempo complessivo per arrivare a un risultato corretto, iterazioni comprese, ed è a questo che tende il costo per risultato accettato proposto qui sopra. Il miglioramento più utile nel breve periodo, d'altra parte, non sarà tanto una maggiore capacità di produrre quanto la capacità dell'agente di riconoscere quando è fuori dalla propria competenza e di chiedere aiuto, segnalando le zone di incertezza invece di affrontarle tutte con la stessa disinvoltura: un agente che dichiara di non essere sicuro di una riclassificazione e chiede un controllo vale più di uno che ne produce una verosimile, e la supervisione umana smette di essere revisione di tutto (impossibile alla velocità degli agenti) per diventare revisione di ciò che conta (sostenibile). Fino a quando questa capacità non maturerà restano valide le difese strutturali: controlli di verifica, versioni recuperabili, tentativi a tempo e la disciplina di ripartire invece di rattoppare.

2.10 Il mandato: come si formula una delega verificabile

Gran parte di ciò che oggi chiamiamo *prompt & context engeneering* è in realtà una questione di delega, e la si capisce meglio se si abbandona l'immagine della richiesta a un motore di ricerca per adottare quella dell'incarico affidato a un collaboratore capace ma privo di contesto. A uno script si dettaglia ogni passo, a un collaboratore si indicano l'obiettivo e il contesto e lo si lascia scegliere il percorso: l'obiettivo piuttosto che i passi (che cosa deve essere vero a lavoro finito), il perché piuttosto che il solo come (che cosa esiste già, quali vincoli pesano), la fiducia piuttosto che il controllo preventivo (si interviene quando la scelta è sbagliata, non prima). È scomodo per chi è abituato a specificare tutto, e tuttavia sovraspecificare vincola l'agente al nostro percorso, che

non è detto sia il migliore. Fa eccezione l'esplorazione, che si presta a richieste ampie (che cosa mi dicono questi dati?), mentre l'esecuzione le sopporta male: «prepara il reporting» lascia all'agente un margine di scelta su fonti, formato e criteri che ci troveremo a contestare a posteriori.

L'elemento che più di ogni altro cambia la qualità del risultato è per mia esperienza dire all'agente come verificare il proprio lavoro. Senza criteri di verifica l'agente genera e spera, con criteri genera, controlla e corregge finché il controllo non è superato, e il criterio definisce che cosa significhi «prima di fermarsi». Nel nostro caso: i saldi di ciascuna business unit coincidono con quelli del gestionale; la somma delle business unit coincide con il consolidato; nessuna business unit manca; ogni scostamento oltre soglia ha una spiegazione documentata oppure una nota richiesta al responsabile.

Un **mandato** (usiamolo al posto di *prompt+context* per riappropriarci un pò della nostra bella lingua italica . . .) ben formato, per i compiti analitici, ha una struttura che si ritrova con poche varianti in tutti gli ambiti, e che nel nostro caso di riferimento si può rendere così:

```
Dati:     i saldi di marzo sono nel server erp-readonly; il budget approvato è
↪  in
          /budget/approvato/. Le note sulle poste straordinarie non sono
          ↪  esportabili:
          le trovi in note-controller.txt.
Esito:    tabella per BU con saldo, budget, scostamento e posizione rispetto
↪  alla
          soglia, più una sintesi e un'appendice con il metodo.
Principi: prevale la prudenza sui valori non verificati; nessuna stima dove
↪  manca
          un dato.
Vincoli:  la soglia sta in soglie.md e non si tocca; nessuna scrittura nel
↪  gestionale;
          nessuna diffusione prima dell'approvazione del controller.
Sospetto: credo che lo scostamento della BU Nord dipenda da consulenze imputate
↪  al
          mese sbagliato; se vedi altro, dimmelo.
Bozza:    allego una bozza di criteri di classificazione; può essere sbagliata,
          criticala prima di usarla.
Verifica: quadratura per BU e sul consolidato, nessuna BU omessa, ogni
↪  scostamento
          oltre soglia con spiegazione o con nota richiesta.
Prima di cominciare: poni tutte le domande che ti servono, e non procedere
↪  finché
          non le hai poste.
```

Tra tutti ritengo che tre elementi di questo schema meritino un commento perchè capaci

di modificare sostanzialmente la direzione dell'elaborazione. Il **primo** è la dichiarazione di che cosa si sospetta (l'ipotesi da corroborare), accompagnata dall'invito esplicito a dissentire: senza l'invito l'agente tende ad ancorarsi all'ipotesi dell'utente e a cercarne conferma, con l'invito la tratta come una tesi da mettere alla prova. Il **secondo** è la bozza di partenza criticabile, che in gergo (od in *jargon* sempre che voglia dire la stessa cosa) [proposta-bersaglio]: reagire a una bozza è un'operazione cognitiva diversa e più rapida che generare da zero, e incorpora la nostra conoscenza del dominio lasciando all'agente lo spazio per migliorarla, purché si dica che può essere sbagliata, perché altrimenti la si accetta in modo acritico. Il **terzo** è la richiesta di fare domande prima di iniziare: quando l'agente colma da sé le lacune, il risultato è verosimile e spesso sbagliato, mentre le domande rivelano le lacune del nostro stesso mandato (non avevamo deciso come trattare un mese in cui una country consegna in ritardo), con un miglioramento della qualità al primo passaggio che chi la pratica riferisce come marcato nei compiti complessi, al costo di pochi minuti di risposte. Quanto ai vincoli, vanno dichiarati tutti e di tutti e tre i tipi (ciò che non può cambiare, i compromessi preferiti, i margini di libertà), perché un vincolo implicito è un vincolo invisibile: l'agente non può rispettare ciò che nessuno gli ha detto. Due buone pratiche di chiusura (che fanno sempre parte del mio set minimo di training sul prompting) completano il quadro. L'agente deve essere invitato a valutare il proprio lavoro in una sezione obbligatoria dell'esito (valutalo da uno a dieci e indica che cosa miglioreresti), cosa che porta a galla ciò che la generazione, ottimizzata sull'interpretazione più comune della richiesta, non aveva considerato: in un caso documentato di piano finanziario personale, l'autocritica fece emergere un'ottimizzazione fiscale ovvia a posteriori e assente dalla prima stesura. E quando si prevede un'esecuzione lunga o costosa si dovrebbe sempre chiedere prima di esplicitare il piano e rivederlo per poter eliminare, aggiungere o modificare gli elementi che potrebbero essere stati inclusi od esclusi per ragioni di conflitto di contesto (o di ordine di lettura delle skills).

2.11 Guardrail, sandbox, permessi e hook

Chiudiamo questa parte comune con il tema che, nella pratica, decide se un'organizzazione può fidarsi di un agente al di là della fase di sperimentazione: la governance dell'esecuzione. Sandbox, permessi sugli strumenti, approvazioni puntuali e hook sono meccanismi distinti che vanno definiti con precisione. La **sandbox** è l'ambiente isolato in cui l'agente esegue le proprie azioni, contenendo eventuali effetti indesiderati entro un perimetro definito. I **permessi** stabiliscono quali strumenti l'agente può usare e su quali dati. Le **approvazioni** sono momenti in cui, per un'azione specifica, si richiede un consenso umano esplicito prima di procedere. Gli **hook**, infine, sono punti del ciclo di vita dell'agente in cui è possibile eseguire controlli programmatici personalizzati, la cui efficacia dipende però interamente da come sono stati implementati, non è una garanzia automatica derivante dalla loro sola presenza. Codex distingue esplicitamente le barriere di esecuzione dalle approvazioni richieste all'utente, mentre Claude Code offre un proprio sistema di hook per questo scopo.

Conviene vedere che forma assumono, concretamente, questi due meccanismi, perché la

differenza fra loro è sostanziale e viene quasi sempre appiattita. I permessi si esprimono come regole dichiarative, tipicamente con una logica a tre stati (consentito, da approvare, vietato):

```
{
  "permissions": {
    "allow": ["mcp__erp-readonly__get_account_balance", "Read(./budget/**)"],
    "ask":   ["Edit(./reporting/**)"],
    "deny":  ["Bash(curl:*)", "Read(./hr/**)"]
  }
}
```

Gli hook, invece, sono punti del ciclo di vita in cui l'organizzazione inserisce codice proprio, che viene eseguito prima o dopo una determinata azione indipendentemente da ciò che il modello ha deciso:

```
{ "hooks": {
    "PreToolUse": [
      { "matcher": "Edit",
        "hooks": [
          { "type": "command",
            "command": "./controlli/verifica_approvazione_controller.sh" } ] } ]
            ↪ } }
```

Il permesso è una regola che il software applica sempre, in modo prevedibile e verificabile, ma è rigido e non sa nulla del merito dell'operazione. L'hook è potente, perché può eseguire verifiche arbitrarie (nell'esempio, interrogare il sistema documentale per accertare che l'approvazione del controller esista davvero prima di consentire la scrittura del pacchetto), ma vale esattamente quanto il codice che lo implementa. È la ragione per cui la sola presenza di un sistema di hook nella scheda tecnica di un prodotto non dice assolutamente nulla sulla sicurezza effettiva della sua implementazione presso un cliente, e per cui, in sede di verifica, la domanda giusta non è "il prodotto supporta gli hook" ma "quali hook sono stati scritti, da chi, e chi li ha collaudati".

Due precisazioni sui permessi, perché è qui che le organizzazioni a volte si fanno male da sole. La prima riguarda l'ordine di valutazione: nei sistemi seri le regole si esaminano in una sequenza fissa (prima i divieti, poi le richieste di approvazione, poi i consensi) e un divieto prevale sempre sul consenso, per quanto quest'ultimo sia più specifico (se si vieta la lettura di tutta la cartella del personale e poi si consente quella di un singolo file al suo interno, il consenso non scatterà mai). Ne discende un'indicazione di disegno: *i divieti si scrivono stretti mentre i consensi possono essere più generosi.* La seconda riguarda i livelli a cui le regole vivono, che sono più d'uno e ordinati secondo una precedenza rigida, al cui vertice sta un livello gestito dall'organizzazione che nessuna impostazione locale può contraddire. È invisibile al singolo utente, ed è la ragione per cui un agente che funziona alla perfezione su un progetto personale incontra in produzione che semplicemente non sono disponibili, senza alcun messaggio d'errore ad avvertirne. Non è un

guasto, è la politica dell'organizzazione che fa il proprio mestiere, e la prima domanda da porsi in quel caso riguarda la configurazione gestita e non il prodotto.

Il divieto di lettura merita poi una motivazione che spesso sfugge: quando un agente legge un file, il contenuto di quel file viene inviato, come parte del contesto, al servizio che ospita il modello. Per un file di credenziali, una cartella del personale o un'estrazione con dati riferibili a persone, il permesso di lettura non è dunque un fatto interno alla macchina ma una decisione sull'uscita di quei dati dal perimetro aziendale, e va trattata con lo stesso rigore. Il divieto esplicito sui percorsi sensibili è il primo presidio, a cui si affiancano gli altri strati.

Sugli hook vanno aggiunte alcune caratteristiche che separano un controllo da un'apparenza di controllo. La prima è che non tutti sono uguali: un hook può eseguire un comando deterministico (un confronto, la verifica dell'esistenza di un file, un'interrogazione), ma può anche consultare a sua volta un modello perché esprima un giudizio sull'azione proposta, oppure avviare un agente con accesso agli strumenti perché indaghi (ispezioni file, esegua verifiche) prima di decidere. Le ultime due varianti sono utilissime quando il criterio è di merito (questa nota di commento è coerente con lo scostamento?) e sono, per costruzione, controlli probabilistici: ricordando come funziona il ciclo agentico non ci si può stupire che un controllo affidato a un modello condivida i limiti del modello. Nella catena che va dalla raccomandazione al vincolo vanno dunque collocate in posizioni diverse, e soltanto l'hook deterministico merita pienamente il nome di vincolo.

La seconda caratteristica è il modo in cui l'hook comunica l'esito, che è meno banale di quanto ci si potrebbe aspettare. In molti sistemi solo un codice di uscita specifico significa «blocca», mentre qualunque altro codice di errore viene trattato come un malfunzionamento dell'hook e ignorato, con il risultato che l'azione prosegue come se il controllo non esistesse. Uno script che si interrompe per un file mancante, un percorso sbagliato o una dipendenza non installata non nega niente, lascia passare. È la differenza fra un controllo che fallisce chiudendo e uno che fallisce aprendo, e un hook scritto per proteggere deve essere progettato per la prima modalità, cioè per negare ogni volta che non riesce ad accertare ciò che doveva accertare. La terza riguarda i privilegi: un hook è un programma qualsiasi che gira con le credenziali e i permessi di chi lo lancia, senza alcuna sandbox intorno, e va dunque trattato con la scrupolosità dovuta a qualunque codice che lavora sotto quell'identità (validazione dell'input, gestione delle virgolette, difesa dai percorsi manipolati). Per questo sarebbe prudente far eseguire agli agenti soltanto gli hook definiti a livello gestito, e per questo un agente progettato per avere ampi livelli di autonomia dovrebbe fotografare gli hook all'avvio e avvertire se qualcuno li modifica a sessione in corso. Un'ultima avvertenza riguarda gli hook che decidono se l'agente possa fermarsi (per esempio finché i controlli non sono superati): vanno dotati di un limite di tentativi, perché un criterio che l'agente non può soddisfare (un controllo che fallisce per ragioni non sue ad esempio un servizio di terze parti fermo) rischia di farlo girare a vuoto fino a esaurire il budget.

Gli hook si prestano infine a un impiego poco spettacolare e molto utile, la registrazione. Un hook che scrive, per ogni chiamata a ogni strumento, l'istante, i parametri e la decisione presa produce un registro degli accessi senza consumare contesto e senza che l'agente se ne accorga. Lo stesso meccanismo, applicato ai nomi degli strumenti esposti dai server (che negli agenti di

sviluppo codice seguono una convenzione prevedibile, server per server e strumento per strumento), consente di intercettare tutte le operazioni di scrittura su tutti i server con una sola regola.

Sul fronte della sandbox, due considerazioni. L'isolamento cambia quella che dobbiamo abituarci a chiamare l'equazione della fiducia. Quando le azioni avvengono in un ambiente che non raggiunge la rete esterna se non verso un elenco di indirizzi consentiti, si può approvare in automatico molto di più, perché il danno possibile è circoscritto (un'azione sbagliata potrà rovinare dati locali, non portarli fuori), ed è la strada con cui si conciliano esecuzioni senza presidio e sicurezza. Ma un contenitore chiuso non impedisce l'uscita dei dati attraverso un canale che si è deciso di consentire: se istruzioni nascoste in un file di progetto o in un allegato inducono l'agente a leggere una credenziale e a scriverla in una risposta che raggiunge un indirizzo ammesso, il firewall non può accorgersene, perché il dato esce da una porta aperta di proposito. La difesa è a più strati (divieto di lettura sui percorsi sensibili, perimetro di rete ristretto, credenziali di breve durata anziché permanenti, e il principio per cui le credenziali non entrano mai nel contesto), nessuno dei quali basta da solo. La seconda considerazione riguarda la reversibilità, che abbiamo posto fra i criteri dell'autonomia: gli strumenti di annullamento di cui questi agenti dispongono coprono soltanto le modifiche compiute attraverso i propri strumenti di scrittura dei file, non quelle che passano dall'esecuzione di comandi di sistema (uno spostamento o una cancellazione eseguiti in quel modo non vengono registrati, e tornare a un punto precedente non li annulla). La reversibilità non è dunque una proprietà dell'agente, è una proprietà di come il lavoro è versionato: la rete di sicurezza vera è una versione precedente recuperabile, creata prima di avviare un'attività potenzialmente distruttiva, come per qualunque altra operazione che modifichi dati di produzione.

Esiste un terzo modo in cui un sistema di controlli può fallire, distinto dal permesso troppo largo e dall'hook scritto male, ovverosia l'approvazione che diventa una formalità. Il controllo umano puntuale funziona finché il volume delle richieste resta entro la capacità cognitiva di chi approva, e cessa di funzionare, pur continuando a esistere sulla carta, nel momento in cui quella soglia viene superata. Un controller che riceve tre richieste di conferma al mese le esamina; uno che ne riceve quaranta al giorno le conferma, e da quel momento l'organizzazione ha un registro di approvazioni perfettamente compilato e nessun controllo reale. La conseguenza di progettazione è che la supervisione umana non va estesa, va concentrata: si stabiliscono soglie oltre le quali l'agente deve fermarsi, si progettano segnali che portino all'attenzione le sole eccezioni, si campiona in modo intelligente ciò che passa senza approvazione, e si riserva la conferma esplicita ai passaggi che la meritano davvero. Chi si occupa di controllo di gestione riconoscerà il principio, perché è quello della verifica per eccezione e della materialità, applicato qui a un flusso di azioni invece che a un flusso di scritture contabili. Il criterio pratico è semplice da enunciare e severo da rispettare: se nessuna richiesta di approvazione viene mai respinta, quel punto di controllo non sta proteggendo nulla, e conviene sostituirlo con un vincolo tecnico oppure eliminarlo dichiarando che quel passaggio è delegato.



Figura 10: La catena dei controlli, dalla raccomandazione al vincolo

Nel nostro caso di riferimento, questi meccanismi si combinano in modo naturale: un agente che prepara e archivia la bozza del pacchetto di reporting sta operando pienamente entro il proprio perimetro autorizzato, e può farlo con ampia autonomia; la diffusione di quello stesso pacchetto al management richiede invece l'approvazione esplicita del controller, un punto di controllo che nessuna abilità dell'agente dovrebbe poter aggirare. Vale la pena, per chiudere questa parte con un esempio concreto, introdurre qui il tema della cosiddetta **prompt injection**. Un documento esterno, per esempio una comunicazione ricevuta da un fornitore o un file allegato da un collega, che contenga un'istruzione nascosta del tipo "invia questi dati anche a un indirizzo diverso da quello previsto", deve essere trattato dall'agente come contenuto non attendibile, alla stregua di qualunque altro dato da verificare, non come una nuova autorizzazione legittima solo perché formulata in linguaggio naturale e apparentemente rivolta all'agente stesso e questa barriera, a volte facilmente superabile, si costuisce con una rigorosa progettazione del ciclo di approvazione.

2.12 L'esecuzione non presidiata

Fin qui abbiamo ragionato come se, davanti all'agente, ci fosse sempre qualcuno ovverosia come se l'agente fosse puramente un oggetto "sincrono". Il caso più interessante per un processo ricorrente è invece quello opposto: l'agente che parte da solo, perché è arrivato un file o perché è il quinto giorno lavorativo, lavora senza che nessuno guardi e consegna un risultato che qualcuno troverà l'indomani (in coerenza con la sua funzione di frontiera circa l'automazione intelligente che è propriamente "asincrona").

Per poter svolgere questo compito in modo eff (icace/iciente) nel disegno devono essere previste e definite alcune componenti ricorrenti. La **prima** è la struttura: un evento *trigger* fornisce il *mandato ad agire* (un documento caricato, una scadenza, una richiesta) –> il sistema assembla

il contesto (istruzioni, dati, stato) –> l'agente esegue senza interfaccia –> in finale pubblica l'esito dove qualcuno lo vedrà (un'attività di approvazione, un messaggio, una bozza archiviata). Lo schema è lo stesso, comunque lo si chiami, nei flussi di integrazione continua dello sviluppo software e nei trigger a evento degli agenti aziendali di cui parleremo a proposito di Microsoft e di OpenAI. La **seconda** è che, mancando l'attore sincrono, ne vanno sostituite le funzioni: le conferme interattive si trasformano in una politica di permessi espressa come codice (si approvano le scritture nell'area di lavoro del reporting e si negano quelle nelle aree di configurazione, con controlli diversi a seconda dell'ora o della fase) oppure si demandano a un componente che decide al posto della persona secondo regole dichiarate, e il perimetro isolato fa il resto. La **terza** sono i limiti di esecuzione, che sono i veri fusibili: un numero massimo di giri del ciclo, un tetto di spesa per esecuzione, un limite di tempo, tutti da dichiarare in anticipo e da provare con condizioni di fallimento volutamente costruite, perché un limite che non si è mai visto scattare è una speranza. Per i lavori lunghi se ne aggiungono altri due. L'agente che registra il proprio avanzamento in punti recuperabili (nello sviluppo, un salvataggio dopo ogni passo concluso) lascia dietro di sé una traccia che si può esaminare, accettare in parte o annullare; e la regola «accetta o riparti» dopo un'esecuzione a tempo (se il risultato è buono lo si tiene, altrimenti si torna al punto sicuro e si riprova con un mandato migliore) costa meno di una spirale di correzioni, perché ogni correzione fatta dentro un contesto già degradato lo degrada ulteriormente. La **quarta** è lo stato. L'agente non ricorda: ogni esecuzione parte da un contesto vuoto, e l'unica continuità è quella che si è scritta fuori. Esistono schemi di lavoro, usati per progetti di settimane, in cui un semplice ciclo rilancia l'agente a ogni iterazione e l'unica memoria è un file di specifica che l'agente legge, aggiorna e lascia; esiste il caso documentato di un agente rimasto in funzione per oltre un mese con verifiche periodiche di una persona, nel quale la memoria dovette necessariamente essere affidata a file e a basi dati, perché la finestra di contesto non è un archivio durevole e subisce compattazioni ripetute. Per la chiusura mensile ne discende un criterio di progetto: lo stato del processo (quali business unit hanno consegnato, quali controlli sono aperti, quali approvazioni mancano) vive in un file o in un registro leggibile da chiunque, non nella conversazione. Infine, una modalità di organizzazione che si presta bene alle verifiche massive: un'invocazione per ciascuna unità (una business unit, un conto, un documento), con strumenti di sola lettura e contesto pulito a ogni giro, i cui risultati si accodano in un registro strutturato da cui estrarre le eccezioni. Ogni invocazione è confinata e riproducibile, e se una fallisce non contamina le altre (costituendo di fatto un sistema a rollback aperto).

3 Parte III. Il fronte Anthropic

Passiamo ora dai concetti comuni alle specificità degli ecosistemi su cui abbiamo scelto di concentrare l'approfondimento tecnico. Cominciamo da Anthropic, la cui traiettoria nel 2026 racconta una strategia piuttosto coerente: dalla riga di comando per sviluppatori fino allo strumento generalista per il lavoro di conoscenza quotidiano.

3.1 Claude Code e il Claude Agent SDK

Il capostipite dell'offerta agentica di Anthropic è Claude Code, l'agente a riga di comando per lo sviluppo software, che ha anticipato molti dei pattern oggi considerati standard nel settore (accesso al filesystem, esecuzione di comandi, gestione della memoria di progetto). Accanto a Claude Code, Anthropic offre il Claude Agent SDK, una libreria disponibile in Python e in TypeScript che espone lo stesso motore, cioè il ciclo agentico, la gestione dei permessi, le sessioni, i subagenti e il supporto nativo a skill, comandi, memoria e plugin, per costruire agenti su misura al di fuori del solo contesto di sviluppo software. È la scelta naturale per un'organizzazione che vuole costruire un proprio agente specializzato, per esempio uno dedicato al processo di chiusura mensile del nostro caso di riferimento, mantenendo il pieno controllo del codice e dell'infrastruttura su cui gira.

3.2 Dove vivono le istruzioni, e in quale ordine si caricano

Abbiamo visto nella parte comune che i file di istruzioni si sommano invece di sostituirsi. Vale la pena vedere come questo principio si traduca nell'implementazione concreta, perché i percorsi e l'ordine di caricamento sono la parte che un'organizzazione deve conoscere per governare il comportamento dei propri agenti senza immaginare una gerarchia di precedenze che in larga parte non esiste.

Nel caso di Claude Code, il file di istruzioni si chiama `CLAUDE.md` e vive in quattro posti distinti.

Livello	Percorso	A chi si applica
Policy gestita	macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`; Linux e WSL: `/etc/claude-code/CLAUDE.md`; Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`	tutti gli utenti della macchina, distribuito dai sistemi informativi
Utente	`~/.claude/CLAUDE.md`	una sola persona, su tutti i suoi progetti
Progetto	`./CLAUDE.md` oppure `./.claude/CLAUDE.md`	il gruppo di lavoro, attraverso il controllo di versione
Locale	`./CLAUDE.local.md`	una sola persona su un solo progetto, da escludere dal versionamento

La regola di caricamento si può riassumere in tre affermazioni. La prima è che, all'avvio, vengono letti i file della cartella di lavoro e di tutte le cartelle che stanno sopra di essa, fino alla radice del filesystem. La seconda è che i file trovati vengono concatenati e non sovrascritti:

nessuno di essi annulla gli altri, e tutti finiscono (in "somma") nel contesto. La terza riguarda l'ordine, che va dalla radice verso il basso, cosicché le istruzioni più vicine al punto in cui si è avviato il lavoro sono le ultime a essere lette, e all'interno di ciascuna cartella il file locale viene accodato dopo quello di progetto. I file che si trovano nelle sottocartelle, invece, non vengono caricati all'avvio ma soltanto quando l'agente legge davvero un file che sta lì dentro.

Ne discende una conseguenza controintuitiva, che ha implicazioni di governo: fra i quattro livelli non esiste una vera precedenza. Se il file di progetto e quello personale si contraddicono, la documentazione stessa avverte che il modello potrebbe seguire l'uno o l'altro, ed è proprio ciò che ci si deve attendere da un sistema probabilistico che legge due istruzioni incompatibili nello stesso contesto. Volendo rendere davvero vincolante una regola non basta scriverla più in alto nella gerarchia, perché la gerarchia non risolve il conflitto: bisogna **tradurla in un permesso o in un hook**, cioè in codice che il software applica sempre. L'unica eccezione riguarda il file di policy gestita, che non può essere escluso dalle impostazioni individuali, ed è appunto per questo lo strumento con cui l'organizzazione impone ciò che non è negoziabile.

Tre meccanismi complementari meritano di essere citati. Il primo è la possibilità di importare altri file con la sintassi `@percorso_al_file`, che espande il contenuto richiamato dentro il contesto (e che si disattiva racchiudendo il riferimento fra apici inversi, quando si vuole soltanto nominare un percorso). Il secondo è la cartella `.claude/rules/`, che permette di spezzare istruzioni voluminose in file tematici, e soprattutto di agganciarle a percorsi specifici attraverso un campo `paths` nell'intestazione: una regola sui test si carica soltanto quando l'agente tocca i file di test, il che è insieme una disciplina di ordine e un risparmio di contesto. Il terzo riguarda `AGENTS.md`, il formato condiviso con altri strumenti di mercato: Claude Code non lo legge come istruzioni native, ma lo si può rendere disponibile importandolo da `CLAUDE.md` con la sintassi appena vista (`@AGENTS.md`), così che un unico documento serva sia Claude Code sia gli strumenti che lo adottano nativamente.

Nel nostro caso di riferimento, un repository del controllo di gestione assumerebbe questa forma:

```
controllo-di-gestione/
├── CLAUDE.md                          # soglie, glossario, convenzioni di
↪ gruppo
├── CLAUDE.local.md                    # note personali, escluso dal
↪ versionamento
├── .claude/
│   ├── rules/
│   │   ├── chiusura.md                # convenzioni sempre valide per la
↪ chiusura
│   │   └── reportistica.md            # paths: "reporting/**", si carica solo
↪ lì
│   └── skills/
```

```
│       └── chiusura-mensile/
│           └── SKILL.md              # procedura invocabile per nome
└── consolidato/
    └── CLAUDE.md                     # regole del consolidato, lette solo
    ↪ quando serve
```

Vale la pena aggiungere, per chi volesse verificare invece di fidarsi (anche perché tra una settimana potrebbe essere tutto diverso), che il comando `/context` elenca in sessione i file di istruzioni effettivamente caricati. È il controllo che consiglio di fare almeno una volta in ogni progetto, perché la distanza fra ciò che si crede di aver configurato e ciò che il sistema legge davvero è, nella mia esperienza, la sorpresa più frequente.

Alla stessa famiglia di strumenti diagnostici appartengono altri comandi che sono il Day1 dei miei corsi sugli agenti. Quello che elenca i server collegati mostra, per ciascuno, se è connesso e quanto contesto occupa (utile perché la scomparsa di un server non genera errori: lo strumento semplicemente non c'è più, e l'agente tenta di cavarsela in altro modo, spesso con una disinvoltura che sembra competenza). Quello che comprime la conversazione accetta un'indicazione di che cosa privilegiare, e conviene usarlo prima che la compressione automatica, che interviene a contesto quasi saturo perdendo dettagli in modo meno governabile. Quello che riavvolge la sessione permette, oltre a ripristinare codice e conversazione, di condensare soltanto una porzione (per esempio trenta turni di esplorazione chiusi in un vicolo cieco) lasciando intatto il resto. Nel file di istruzioni, infine, una sezione con un titolo convenzionale viene preservata attraverso la compressione: è il posto in cui tenere i vincoli che non si possono perdere, per quanto a lungo duri il lavoro.

Distinta da tutto questo è la memoria automatica, cioè le note che l'agente scrive da sé a partire dalle correzioni ricevute, che vivono in una cartella dedicata per progetto sotto `~/.claude/projects/` e si governano dal comando `/memory`. È bene tenerla concettualmente separata: i file di istruzioni sono ciò che l'organizzazione dichiara, la memoria automatica è ciò che l'agente annota durante l'esecuzione: i regimi di responsabilità sono radicalmente diversi.

Nelle applicazioni (Claude Chat e Cowork), dove non esiste un filesystem di progetto da configurare, la stessa funzione è svolta da tre livelli: le istruzioni di organizzazione, che su piani Team ed Enterprise il proprietario imposta nelle impostazioni dell'organizzazione, valgono per tutti i membri in ogni conversazione, hanno un limite di tremila caratteri e, unico caso in cui una precedenza è dichiarata esplicitamente, prevalgono su quelle individuali in caso di conflitto; le istruzioni di account, che ciascuno imposta per sé e valgono su tutte le proprie conversazioni; e le istruzioni di progetto, circoscritte a un singolo progetto. Cowork, girando sulla macchina dell'utente, legge anche il `CLAUDE.md` della cartella di lavoro, con una cautela in più: le importazioni che puntano fuori dalla cartella della sessione vengono ignorate.

3.3 Le skill come standard aperto e i plugin come distribuzione condivisa

Le skill, introdotte come formato aperto nell'autunno del 2025, sono oggi disponibili sia come skill "first party" pre confezionate, per esempio per la creazione di documenti in formato PowerPoint, Excel, Word e PDF, sia come skill personalizzate caricate dall'organizzazione. Il modello di governance enterprise dedicato prevede, per Claude Enterprise, una scansione automatica dei contenuti delle skill personalizzate caricate in **claude.ai**; le skill installate tramite API o tramite Claude Code richiedono invece una revisione manuale più simile, nello spirito, a quella che si applicherebbe all'installazione di un nuovo software in produzione, un'analogia che vale la pena portare esplicitamente ai team di sicurezza aziendale poco familiari con questo tipo di rischio. I plugin, distinti dalle skill, sono il meccanismo con cui Anthropic distribuisce insieme skill, connettori e altre estensioni sia in Claude Chat sia in Cowork, mentre Claude Code mantiene un proprio meccanismo di estensioni dedicato.

3.4 Dove vivono le skill, e come si sovrappongono

Se per le istruzioni la regola è che si sommano, per le skill la regola è opposta e conviene ricordarlo. Due skill con lo stesso nome non si fondono, si escludono. Ne resta attiva una sola, e le altre vengono ignorate.

I luoghi in cui una skill può vivere sono cinque, e l'ordine in cui li elenco è anche l'ordine di precedenza.

Livello	Percorso	Chi la controlla
Organizzazione	`.claude/skills/<nome>/SKILL.md` nella cartella delle impostazioni gestite	i sistemi informativi, per tutte le macchine
Personale	`~/.claude/skills/<nome>/SKILL.md`	la singola persona, su tutti i suoi progetti
Progetto	`.claude/skills/<nome>/SKILL.md`	il gruppo, attraverso il controllo di versione
Sottocartella	`<sottocartella>/.claude/skills/<nome>/SKILL.md`	il gruppo, per una porzione del repository
Pacchetto	`<plugin>/skills/<nome>/SKILL.md`	chi distribuisce il pacchetto

Se la stessa skill `chiusura-mensile` del nostro esempio esiste a livello di organizzazione, personale e di progetto, quella che viene eseguita è la prima, e le altre due non hanno alcun effetto. È un comportamento che va comunicato esplicitamente a chi lavora, perché la moda-

lità di fallimento è particolarmente insidiosa: la skill di progetto sembra installata, compare nell'elenco, e semplicemente non è quella che gira. Due meccanismi attenuano il problema. Le skill contenute in un pacchetto sono richiamate con un nome qualificato dal pacchetto stesso, nella forma `/nome-pacchetto:nome-skill`, e quindi non possono collidere (perché il nome del pacchetto fa da spazio di nomi). Le skill che stanno in una sottocartella, a loro volta, convivono con quella della radice e si richiamano con il percorso davanti al nome, nella forma `/consolidato:chiusura-mensile`.

Sul fronte del costo, il meccanismo di caricamento progressivo descritto nella parte comune si quantifica così: la descrizione di ogni skill occupa contesto a ogni turno di conversazione, da poche decine a qualche centinaio di token (a seconda della lunghezza della descrizione), mentre il corpo entra solo all'attivazione e le risorse solo se il percorso le richiede. Non esiste un limite al numero di skill installabili, ma esiste un limite alla lunghezza della descrizione, ed esiste soprattutto un costo che cresce linearmente con il numero di competenze dichiarate. È lo stesso fenomeno già osservato per gli strumenti: più descrizioni si tengono sempre accese, più il modello fatica a scegliere. I comandi `/skills` e `/skill-doctor` servono appunto a questo, il primo per elencare le skill disponibili e la loro provenienza, il secondo per misurare quanto ciascuna costa e quanto viene effettivamente usata. Una skill che pesa a ogni turno e non viene mai attivata si può spegnere senza disinstallarla. Per le procedure che solo una persona deve poter avviare (la diffusione del pacchetto, un rilascio, un'operazione irreversibile) esiste inoltre un'impostazione, nella testata della skill, che la nasconde del tutto al modello: non ne carica la descrizione, non compare fra gli strumenti disponibili, non costa contesto e si può soltanto invocare per nome. È un modo elegante di tradurre in struttura il principio per cui certe azioni non si delegano all'iniziativa dell'agente.

Nelle applicazioni il modello è più semplice e meno granulare: le skill personali si caricano dalla sezione di personalizzazione, come un archivio compresso la cui radice è la cartella della skill, e le organizzazioni su piano Team ed Enterprise possono distribuirne di proprie a tutti i membri. Resta valida la distinzione già richiamata sulla governance, per cui le skill caricate in claude.ai sono sottoposte a una scansione automatica mentre quelle installate attraverso le API o lo strumento a riga di comando richiedono una revisione manuale.

Chiudo con l'osservazione che nasce dal confronto fra le due sezioni: le istruzioni si sommano, le skill si escludono. Sono due regole opposte, valgono su due meccanismi che nelle presentazioni commerciali vengono spesso accostati, e confonderle produce due errori speculari. Chi tratta le istruzioni come se avessero una precedenza scrive regole aziendali che nessuno applica davvero, e chi tratta le skill come se si sommassero crede di aver esteso una procedura quando in realtà l'ha sostituita per intero. E tutto questo, nella maggior parte dei casi, avviene "a cura" di volenterosi "non IT guys" dell'azienda che possono anche mancare della "sensibilità tecnica" per preoccuparsene più di tanto... Auguri.

3.5 Dove vivono i permessi, e chi vince

Terza famiglia di file, terza regola. Le impostazioni di Claude Code, che contengono permessi, hook, abilitazione dei pacchetti, variabili d'ambiente e server collegati, risiedono in cinque livelli, e qui, a differenza di quanto accade per le istruzioni, esiste una precedenza stretta e dichiarata. La tabella segue l'ordine, dal più forte al più debole.

Livello	Dove sta	Chi lo controlla
Gestito	cartelle di sistema, dove l'utente non ha diritto di scrittura	i sistemi informativi, e nulla può sovrascriverlo
Riga di comando	argomenti passati all'avvio	chi avvia la sessione, per quella sessione soltanto
Locale	`.claude/settings.local.json`	una persona su un progetto, escluso dal versionamento
Progetto	`.claude/settings.json`	il gruppo, attraverso il controllo di versione
Utente	`~/.claude/settings.json`	una persona, su tutti i suoi progetti

È il motivo per cui il livello gestito è lo strumento con cui l'organizzazione impone ciò che non è negoziabile: quali strumenti siano utilizzabili, quali permessi siano sempre negati, quali server MCP siano ammessi o vietati, se gli hook possano essere definiti fuori dal livello gestito, quali cataloghi di pacchetti siano riconosciuti. I livelli intermedi fanno ciò che ci si aspetta: il progetto condiviso fissa gli standard del gruppo (e le sue modifiche passano dalla revisione come qualunque altro file), il locale permette alla persona di adattare la macchina senza inquinare la configurazione comune. Alcuni dettagli del livello gestito ancora troppo poco noti e frequentati di quanto sarebbe opportuno. L'elenco dei server MCP consentiti opera a livello di configurazione: un server che non vi compare non viene nemmeno contattato, qualunque cosa dica il file di progetto. L'elenco dei cataloghi di pacchetti riconosciuti ha lo stesso effetto per i plugin (la fonte non approvata non viene contattata né vagliata), ed è la contromisura diretta al rischio di catena di approvvigionamento di cui abbiamo parlato a proposito dei pacchetti. E un'impostazione dedicata può restringere gli hook a quelli definiti dall'organizzazione, ignorando quelli di progetto e quelli personali, per impedire che un repository compromesso installi codice che gira con le credenziali di chi lo apre.

Aggiungo due prassi che considero, per espierienza, dirimenti molte ambiguità e risolutive di molti problemi. La prima: un gruppo centrale (IT? in generale COE AI) configura i server approvati, il file di configurazione viaggia con il progetto sotto controllo di versione (così chi clona il repository ottiene collegamenti identici, mentre il livello gestito ne vieta qualunque altro), e le credenziali non stanno nel file ma nell'ambiente di ciascuno. La seconda: semplificare l'installazione, raccogliendo credenziali, proxy, politiche gestite e pacchetti predefiniti in un unico

passo, perché più è alto l'attrito iniziale meno persone arrivano al secondo giorno, e chi non ci arriva finisce per usare strumenti personali fuori dal perimetro (a questo servono l'accesso unico e la cattura del dominio aziendale, che riconducono nel perimetro chiunque si autentichi con un indirizzo dell'azienda).

E con questa fanno tre regole che conviene scrivere sulla "lavagna" virutuale e lasciare li 1) le istruzioni si sommano 2) le skill si escludono 3) i permessi si ordinano. Il rischio, nella pratica, è di applicare a ciascuna famiglia la regola di un'altra: scrivere in un file di istruzioni ciò che dovrebbe essere un divieto, aspettarsi che la skill di un gruppo integri quella dell'organizzazione, oppure dimenticare che un divieto gestito non si discute.

3.6 I sottoagenti di Claude Code: che cosa si può configurare

Nella parte comune abbiamo visto un subagente come un file di testo con un perimetro ristretto. Nell'implementazione di Claude Code (la documentazione cui mi riferisco è di febbraio 2026 ergo forse già modificata nei dettagli, meno nei principi) quel file ha qualche campo in più che vale la pena conoscere, perché ciascuno traduce un principio di governo in un'impostazione.

Campo	Che cosa governa
Strumenti	il perimetro, più stretto di quello del coordinatore
Tetto ai giri	quanti giri del ciclo prima che il subagente si fermi (il "fusibile" come prima metaforizzato, contro le esplorazioni senza fine)
Server MCP	quali collegamenti sono raggiungibili da quel subagente: un lettore di database può avere il server del database, un revisore nessuno
Skill precaricate	a differenza della sessione principale, dove si caricano a richiesta, nei subagenti entrano per intero all'avvio e costano contesto dal primo turno
Memoria	una memoria persistente propria, con ambito di utente, di progetto o locale, con un indice sintetico caricato a ogni invocazione e file tematici letti a richiesta
Hook	controlli propri del subagente, per esempio su ogni comando prima che sia eseguito

Valgono inoltre alcuni vincoli di struttura. La gerarchia è a due soli livelli: un subagente non può generarne altri, ed è una scelta deliberata, perché le gerarchie ricorsive sono eleganti sulla carta e producono caos nella pratica. I subagenti si possono eseguire in primo piano o sullo sfondo, e la differenza conta più di quanto appaia: quelli in primo piano bloccano la conversazione ma possono usare i server MCP, porre domande e passare dal normale sistema di approvazioni; quelli sullo sfondo lavorano in parallelo, ma non possono usare i server MCP né porre domande, e devono ricevere i permessi in anticipo, perché ciò che richiederebbe un'approvazione non pre-autorizzata viene negato e il lavoro prosegue senza. Il rimedio per i dati esterni è far recuperare i dati al coordinatore e passarli allo sfondo come parte del compito (si baratta l'accesso in tempo reale con

il parallelismo). Le trascrizioni dei subagenti sopravvivono alla compattazione della conversazione principale e si possono riprendere anche dopo un riavvio, purché si riprenda la stessa sessione, così che il lavoro di un'analisi di venti minuti non va perso.

Esiste poi, in forma (ancora) dichiaratamente sperimentale, un livello superiore: le squadre di agenti, in cui sessioni indipendenti si coordinano attraverso un elenco condiviso di compiti, con dipendenze che impediscono di avviare un lavoro prima che sia pronto ciò da cui dipende, e si scambiano messaggi direttamente, invece che attraverso il coordinatore. Hanno senso quando gli specialisti devono reagire in tempo reale al lavoro degli altri (chi finisce le definizioni avvisa chi le consuma), costano diverse volte un sistema di subagenti perché ciascun membro è una sessione completa, e hanno limiti dichiarati: una sola squadra per sessione, nessuna squadra annidata, nessuna ripresa di un membro interrotto, stato dei compiti talvolta in ritardo, permessi ereditati dal coordinatore e non allargabili per singolo membro. Per i processi di cui ci occupiamo, nella quasi totalità dei casi l'agente unico è sufficiente, il sistema di subagenti è la scelta evolutiva, la squadra eccezione alla regola (che come tale va motivata e giustificata).

3.7 Cowork e i connettori MCP nativi

Nel gennaio 2026 Anthropic ha introdotto Claude Cowork che porta le capacità agentiche già mature di Claude Code (automazione multi passo, accesso a file locali, esecuzione in un ambiente isolato) ad un livello di utilizzo non tecnico, slegato dall'approccio a riga di comando. A distanza di poche settimane sono arrivati i connettori nativi basati su MCP verso strumenti già ampiamente diffusi nelle organizzazioni, come Slack, Canva, Figma e Box, che permettono all'agente di agire direttamente dentro gli strumenti aziendali già in uso, invece di limitarsi a produrre un output da trasferire manualmente altrove.

Nell'esemplificazione che stiamo percorrendo, un controller potrebbe usare direttamente Cowork per raccogliere i dati di più fonti, applicare la skill di reporting del gruppo, e predisporre la bozza da sottoporre a revisione, senza dover passare attraverso un team di sviluppo AI per farlo ma utilizzando una "superApp" molto simile ad un ambiente di lavoro "grafico"._

3.8 I tre livelli di costruzione: Messages API, Agent SDK, Claude Managed Agents

Per chi debba costruire un agente su misura, Anthropic offre, come vedremo essere il caso anche per OpenAI, tre livelli distinti tra cui scegliere, ciascuno con un diverso compromesso tra controllo e responsabilità operativa. Il livello più basso è l'accesso diretto alle API (la Messages API), in cui l'organizzazione implementa da sé l'intero ciclo agentico. Il livello intermedio è l'Agent SDK già descritto, che fornisce il ciclo, i subagenti e gli strumenti già pronti di Claude Code, lasciando all'organizzazione la responsabilità dell'infrastruttura su cui il tutto viene eseguito. Il livello più gestito sono i Claude Managed Agents, un servizio in cui è Anthropic stessa a occuparsi di sandboxing, stato e infrastruttura per agenti a esecuzione lunga o asincrona, pensato per chi non vuole, o non può, gestire in proprio questa complessità. Le release note più recenti mostrano sui

Managed Agents un servizio in evoluzione attiva con evoluzioni tipo l'implementazione di una politica `auto` con cui il server stesso valuta le chiamate a uno strumento, eseguendole, negandole o richiedendo un'approvazione, un meccanismo che può sostenere la gestione delle operazioni autorizzate nel nostro caso di riferimento, senza per questo sostituire le regole aziendali. ### Computer use e memoria: la maturazione produttiva

Due capacità che fino a poco tempo fa erano considerate sperimentali meritano un aggiornamento. Il computer use, la capacità dell'agente di operare un'interfaccia grafica come farebbe una persona, muovendo il cursore, cliccando, digitando, è oggi utilizzabile e sufficientemente maturo anche a livello di API senza la necessità di header sperimentali. Il memory tool, dal canto suo, risolve un problema distinto, quello di conservare informazioni che l'agente potrà rileggere in un passaggio successivo: la documentazione precisa che è l'applicazione a dover implementare la gestione effettiva delle operazioni di lettura e scrittura in memoria, non il solo fatto che il modello sappia richiederle a implicare che il fornitore conservi automaticamente ogni informazione al posto dell'organizzazione.

Nel nostro caso di riferimento, un agente potrebbe conservare in memoria una nota sulla definizione approvata di un indicatore o sulle verifiche ancora aperte da un mese all'altro: il team deve stabilire con chiarezza durata, accesso e criterio di aggiornamento di quelle note, perché una memoria che conserva un'informazione ormai superata rende il sistema coerentemente sbagliato piuttosto che utile (oltre ad aprire scenari legati al GDPR che i nostri amici d'oltreoceano spesso tendono ad ignorare)

4 Parte IV. Il fronte OpenAI

La traiettoria di OpenAI segue una logica in parte diversa da quella di Anthropic, più centrata sulla componibilità a livello di piattaforma e sull'apertura verso team meno tecnici attraverso strumenti visuali, pur convergendo, come vedremo nella parte comparativa, su molti degli stessi concetti di fondo.

4.1 Orientarsi nel catalogo: Chat, ChatGPT Work, Workspace Agents e Codex

Uno degli errori più comuni in cui incorre chi scrive di OpenAI per un pubblico generalista è trattare come sinonimi prodotti che, in realtà, rispondono a esigenze diverse. ChatGPT, nella sua forma conversazionale, è e resta lo strumento per il dialogo e per compiti puntuali. "ChatGPT Work" è invece il termine più aggiornato per il lavoro delegato su attività articolate, documentato specificamente per compiti che restituiscono un risultato pronto per la revisione, non una semplice risposta testuale. Esiste inoltre un prodotto distinto, "Workspace Agents", pensato per agenti riutilizzabili a livello di team, con documentazione dedicata ai trigger esterni (per esempio l'arrivo di un nuovo documento) e a un percorso di creazione dell'agente in linguaggio naturale, pensato per chi non scrive codice. Codex, infine, resta l'agente dedicato allo sviluppo software, oggi

disponibile anche come applicazione a sé ed in generale come "superapp" che integra le diverse funzioni.

Applicato al nostro caso di riferimento: un brainstorming preliminare sul formato del reporting trimestrale avviene in Chat; la produzione periodica del pacchetto di reporting mensile, ripetuta ogni mese con le stesse regole, è il caso d'uso naturale per Workspace Agents; un intervento sullo script di consolidamento dati è compito di Codex.

4.2 I tre livelli di costruzione: Responses API, Agents SDK, Agents API

Come per Anthropic, anche OpenAI offre tre livelli distinti per chi debba costruire un agente su misura. Il livello più diretto è l'uso della Responses API stessa, con cui un'organizzazione può orchestrare autonomamente strumenti integrati come la ricerca web o la ricerca su file. Il livello intermedio è l'Agents SDK, rilasciato come framework open source per orchestrare flussi multi agente attorno a tre concetti chiave: gli agenti, cioè modelli configurati con istruzioni e strumenti; gli handoff, il trasferimento controllato di un compito da un agente a un altro; e i guardrail, controlli automatici di validazione su input e output. Il livello più gestito è la Agents API, l'infrastruttura con sessioni e ambienti propri di cui OpenAI stessa si occupa, concettualmente parallela ai Claude Managed Agents di Anthropic.

La scelta tra questi tre livelli, per un'organizzazione, dipende in larga misura dalla stessa domanda che abbiamo posto per Anthropic: chi deve possedere, e mantenere nel tempo, la responsabilità dell'infrastruttura su cui l'agente gira.

4.3 Codex e il consolidamento del browsing agentico

Un elemento di recente, utile per mostrare quanto velocemente si stia ancora muovendo il mercato anche sul fronte meno discusso del "browsing agentico", riguarda ChatGPT Atlas, il browser con funzionalità AI lanciato da OpenAI e già dismesso perché le relative capacità di navigazione e di uso del computer sono state riassorbite direttamente nell'app ChatGPT, sia nella versione desktop sia nell'estensione per Chrome, ed in Codex, segno di un consolidamento della strategia agentica attorno a meno superfici prodotto, ma più profondamente integrate tra loro. Un dettaglio tecnico minore ma corretto da riportare, per chi lavorasse a un livello di maggiore profondità tecnica: in una versione recente Codex ha rimossoil comando che esponeva Codex stesso come server MCP, mantenendo però invariato il supporto di Codex come client verso server MCP esterni, una distinzione, Codex consuma MCP ma non lo espone più, che vale la pena chiarire per chi si affidasse a una guida ormai superata

5 Parte V. Il fronte Microsoft

La traiettoria di Microsoft in questo campo racconta una logica diversa da quella di Anthropic e di OpenAI, ed è proprio questa diversità, più che una minore maturità tecnica, a renderla rilevante per chi opera in un contesto aziendale strutturato: mentre Anthropic e OpenAI hanno

costruito, innanzitutto, capacità dell'agente (che cosa un agente sa fare, con quale autonomia), Microsoft ha costruito, innanzitutto, infrastruttura di governo della popolazione di agenti (chi è un agente, a nome di chi opera, come si controlla una flotta di migliaia di agenti nel tempo), una scelta coerente con la posizione che l'azienda occupa da tempo nella gestione delle identità aziendali attraverso Entra ID (già noto come Azure Active Directory).

5.1 Che cosa chiama "agente" Microsoft: una scala, non un oggetto unico

Prima di entrare nei singoli prodotti conviene sciogliere un equivoco che, nella mia esperienza, genera una certa confusione perché nell'ecosistema Microsoft la parola "agente" copre oggetti molto diversi (in termini di architettura e capability). L'agente che un utente costruisce in dieci minuti dall'interfaccia di chat di Microsoft 365 Copilot e l'agente costruito in Copilot Studio, magari con trigger e con flussi di automazione, condividono il nome commerciale e quasi nulla dell'architettura.

Il primo, costruito con l'**Agent Builder** integrato in Copilot, si configura con un nome, una descrizione, un testo di istruzioni (fino a ottomila caratteri), fino a venti fonti di conoscenza (siti e cartelle SharePoint, connettori, eventualmente contenuti web) e poche capacità predefinite come l'interprete di codice o la generazione di immagini. Non esiste, in quella interfaccia, una sezione dedicata alle azioni obbligando a passare a Copilot Studio per integrare servizi esterni. Soprattutto (questo è il punto dirimente in questo momento di marketing del terrore), non può agire di propria iniziativa. Se si legge la tabella comparativa ufficiale alla voce interazioni proattive, per questa tipologia di Agenti si atterrra su di un laconico "non supportate, dipendono da un'interazione avviata dall'utente". A questo si aggiunge un limite architetturale dichiarato apertamente, cioè l'assenza di un vero ciclo di ragionamento iterativo, perché l'ancoraggio alle informazioni e le chiamate agli strumenti avvengono in sequenza e non dentro un ciclo di pianificazione, mentre l'orchestratore e il modello non sono sotto il controllo di chi costruisce l'agente ma restano quelli di Microsoft. Gli strumenti a disposizione di questo tipo di agente sono descritti come interazioni "a turno singolo" che significa che l'agente può "toccare" un sistema esterno una volta, dentro il proprio turno, ma non può pianificare, iterare, né riprendersi da un proprio errore. Detto questo va precisato che il turno singolo non significa necessariamente che l'agente in questione **si incapace di scrivere sui sistemi** (e quindi fare danni). Se costruito con gli strumenti di sviluppo, cioè con il Microsoft 365 Agents Toolkit, e dotato di plugin verso API o verso server MCP, può creare, aggiornare ed eliminare dati. L'asse corretto lungo cui distinguere, dunque, non è "legge oppure scrive", ma **chi possiede il ciclo di orchestrazione** e **chi dà l'avvio al lavoro**: su entrambi, l'agente costruito dall'interfaccia di chat sta al gradino più basso, perché il ciclo è di Microsoft e l'avvio è di una persona che digita.

Propongo quindi di leggere l'intera offerta lungo tre assi, che a ben vedere si applicano anche agli altri due fornitori e che ho trovato essere l'unico modo per orientarsi senza smarrirsi fra i nomi commerciali. Il primo è **chi possiede il ciclo di orchestrazione**: Microsoft, nel caso degli agenti dichiarativi, oppure l'organizzazione, nel caso dei cosiddetti *custom engine agent*, costruiti

con il proprio modello e il proprio orchestratore e semplicemente ospitati nella superficie Copilot. Il secondo è **chi dà l'avvio al lavoro**: una persona che scrive in chat, oppure un evento che accade in un sistema. Il terzo è **come è vincolata l'esecuzione**: dal giudizio generativo del modello, oppure da un percorso deterministico definito in anticipo. Un agente costruito in Agent Builder si colloca all'estremo conservativo di tutti e tre; un agente Copilot Studio con trigger e flussi si sposta su ciascuno di essi, ed è per questo che merita di essere trattato come un oggetto diverso, non come una versione più ricca del primo.



Figura 11: La scala degli agenti Microsoft letta su tre assi

5.2 L'agente come identità aziendale: Entra Agent ID

Il tratto più caratterizzante dell'impostazione Microsoft è che l'agente riceve, da subito, un'identità digitale propria, distinta sia da un account utente sia da un service principal tradizionale (la forma con cui, finora, un'applicazione automatizzata si presentava ai sistemi aziendali). Microsoft Entra Agent ID estende alla popolazione di agenti gli stessi meccanismi già maturi per le identità umane: controllo del rischio in tempo reale, policy di accesso condizionale, e modelli di configurazione riutilizzabili chiamati "Agent Identity Blueprint", pensati per replicare in modo coerente la stessa postura di sicurezza su flotte di centinaia o migliaia di agenti, non per configurarli uno per uno. Applicato al nostro caso di riferimento, questo significa concretamente che l'agente di chiusura mensile della country italiana e quello della country tedesca, pur eseguendo lo stesso compito con lo stesso metodo, ricevono ciascuno una propria identità Entra Agent ID, tracciabile individualmente nei log di accesso al sistema gestionale, e che un unico blueprint può imporre a entrambi, in un colpo solo, la stessa regola di accesso condizionale, per esempio il divieto di autenticarsi da un indirizzo IP esterno alla rete aziendale durante la finestra di chiusura. È una logica di anagrafica e di ciclo di vita dell'identità, più vicina, sul piano amministrativo, a come si

tratterebbe un nuovo dipendente o un fornitore esterno, che alla logica di permessi e guardrail attorno a un singolo agente con cui ragionano prevalentemente Anthropic e OpenAI.

5.3 Agent 365: il control panel (si lo so si chiama control plane, ma non mi piace) della flotta di agenti

Coerente con questa impostazione è Microsoft Agent 365 descritto esplicitamente come strumento per governare l'intera flotta di agenti aziendali, a prescindere da dove siano stati creati. Il prodotto si articola in cinque componenti: un registro centralizzato (Registry) che include anche gli agenti non autorizzati, cioè creati al di fuori di un percorso approvato dall'IT; un controllo degli accessi basato sul rischio (Access Control); un cruscotto che mappa le connessioni tra agenti, utenti e risorse (Visualization); un livello di interoperabilità tra agenti costruiti su piattaforme diverse (Interoperability); e un livello di sicurezza integrato con gli strumenti Microsoft già in uso per la protezione dei dati aziendali (Security). Nel nostro caso di riferimento, il valore pratico del Registry si vede in una situazione tutt'altro che ipotetica: un analista della country francese, senza coinvolgere l'IT centrale, costruisce con Copilot Studio un proprio agente per velocizzare la propria parte di chiusura mensile. Senza un control "plane" di questo tipo, quell'agente resterebbe invisibile al responsabile della sicurezza del gruppo fino al primo incidente; con Agent 365, l'agente compare nel registro centralizzato non appena creato, con i propri permessi e le proprie connessioni visibili al resto dell'organizzazione, permettendo di portarlo entro lo stesso perimetro di governance degli altri agenti del gruppo prima, e non dopo, che diventi un problema.

5.4 Gli agenti dichiarativi: il file di configurazione secondo Microsoft

Se in Anthropic e in OpenAI il file di configurazione dell'agente è un documento in Markdown, in Microsoft 365 Copilot la stessa funzione è svolta da un manifesto dichiarativo in formato JSON, articolato in quattro componenti: - la definizione dell'agente (nome, descrizione, istruzioni comportamentali) - le capacità (le azioni che l'agente può compiere, spesso esposte come plugin) - le fonti di conoscenza (i documenti e i dati su cui l'agente può basare le proprie risposte) - i metadati dell'applicazione (icona, informazioni di pubblicazione).

Nella sua forma essenziale il manifesto si presenta così, e il confronto con il file Markdown della sezione precedente è più eloquente di qualunque commento:

```
{
  "version": "v1.5",
  "name": "Assistente di chiusura mensile",
  "description": "Risponde su procedure, definizioni e scadenze della chiusura",
  "instructions": "Applica sempre le soglie di materialità del gruppo...",
  "capabilities": [
    { "name": "OneDriveAndSharePoint" },
    { "name": "CodeInterpreter" }
  ],
```

```
  "actions": [ { "id": "erpPlugin", "file": "plugin.json" } ]
}
```

La scelta del JSON al posto del Markdown riflette una filosofia in cui l'agente nasce come oggetto gestito da una piattaforma con un proprio ciclo di pubblicazione e approvazione, più vicino nella sua natura a un'applicazione da distribuire nel catalogo aziendale che a un file di istruzioni curato direttamente dall'utente finale. Coerentemente con l'apertura verso team meno tecnici Microsoft offre quattro percorsi di creazione per un agente dichiarativo: l'Agent Builder, un percorso no-code integrato in Copilot; il Microsoft 365 Agents Toolkit, pensato per lo sviluppo pro-code in ambiente Visual Studio Code; Copilot Studio, un ambiente visuale low code e la creazione diretta da SharePoint, no-code, a partire da una raccolta documentale esistente. Nel nostro caso di riferimento, un agente dichiarativo "assistente di chiusura mensile" avrebbe, nel proprio campo istruzioni, un testo concettualmente identico a quello che useremmo in un file Markdown (la soglia di materialità, il formato delle date, il divieto di diffusione non autorizzata), ma incapsulato in un manifesto JSON pubblicato, versionato e distribuito come un'applicazione a un gruppo definito di utenti attraverso il catalogo di Microsoft 365 Copilot.

5.5 Copilot Studio: orchestrazione generativa, trigger e agent flow

Copilot Studio è un oggetto di natura diversa, e vale la pena spiegare in che cosa consista esattamente la differenza, perché è la stessa che separa un assistente da un processo aziendale automatizzato.

Il primo elemento è il **tipo di orchestrazione**. In quella classica, l'agente seleziona un *topic*, cioè un percorso di dialogo costruito dall'autore, confrontando la richiesta dell'utente con frasi di attivazione predefinite, e gli strumenti possono essere invocati soltanto in modo esplicito dall'interno di un topic: siamo, a tutti gli effetti, nel mondo dell'albero decisionale, per quanto raffinato. In quella generativa, invece, l'agente sceglie fra topic, strumenti e fonti di conoscenza sulla base della loro descrizione, può combinarli, e li invoca in sequenza ponendo all'utente le domande necessarie a colmare le informazioni mancanti. L'esempio che usa Microsoft è istruttivo proprio per la sua banalità: alla richiesta "mi servono gli orari del negozio e qual è il punto vendita più vicino", l'agente in orchestrazione generativa seleziona due topic distinti e li concatena, mentre l'orchestrazione classica tenterebbe di ricondurre la richiesta a un unico percorso.

Il secondo elemento sono i **trigger**, ed è qui che l'agente smette di essere un interlocutore e diventa un processo. Un trigger di evento consente all'agente di agire autonomamente quando accade qualcosa: un elemento creato in SharePoint, un file caricato, un'attività completata, una riga aggiunta o modificata in Dataverse, oppure semplicemente una ricorrenza temporale. La differenza rispetto al trigger conversazionale è netta e conviene formularla così: nel primo caso qualcuno ha scritto qualcosa, nel secondo il mondo è cambiato e nessun essere umano è nella conversazione. Due vincoli, però, meritano di essere portati all'attenzione di chi governa i rischi, perché raramente compaiono nelle presentazioni commerciali. Il primo è che l'autonomia è disponibile **solo** con l'orchestrazione generativa attiva: non si può avere insieme un agente

autonomo e un percorso interamente deterministico deciso dall'autore. Il secondo, più delicato, è che i trigger e le azioni che richiedono autenticazione utilizzano **le credenziali di chi ha creato l'agente**, il che significa che l'agente autonomo agisce sui sistemi come quella specifica persona, con i suoi permessi. Chiunque abbia dimestichezza con la segregazione dei compiti comprende immediatamente perché un dettaglio apparentemente tecnico sia in realtà una questione di controllo interno, e perché un agente di questo tipo non dovrebbe mai essere creato dall'utenza di chi detiene anche poteri di approvazione sul processo che l'agente esegue.

Il terzo elemento sono gli **agent flow**, cioè i flussi di automazione, che Microsoft distingue esplicitamente dagli agenti con una formula che condivido: un flusso è deterministico, esegue azioni seguendo un percorso a regole, e lo stesso input produce sempre lo stesso output. Il rapporto con Power Automate è di parentela stretta ma non di identità, dal momento che gli agent flow nascono dentro Copilot Studio, sono fatturati sul consumo di quest'ultimo invece che su una licenza Power Automate, e portano con sé limiti propri (non si possono condividere né assegnare a più proprietari, e la conversione di un flusso Power Automate esistente è un'operazione irreversibile, un dettaglio che conviene conoscere prima e non dopo). Il punto architetturale, però, è assai più interessante delle questioni di licenza: un agent flow può essere collegato a un agente **come se fosse uno strumento**, ed è questa la configurazione matura verso cui, a mio giudizio, converge il buon senso progettuale. Non "flusso oppure agente", ma un agente generativo che decide *se* e *quando* agire, e flussi deterministici e verificabili che definiscono *come* l'azione viene eseguita. È esattamente la stessa logica con cui, nel controllo di gestione, lasciamo al giudizio del responsabile la valutazione di un'anomalia ma non la libertà di inventarsi la procedura di registrazione contabile.

Applicato al nostro caso di riferimento, la scala si legge senza sforzo. Un agente dichiarativo costruito dal controller in una pausa pranzo, che risponde a domande sul manuale di chiusura e sulle definizioni degli indicatori attingendo alla cartella SharePoint della funzione, è utile, è a basso rischio, e resta un assistente documentale: se smettesse di funzionare domani, qualcuno dovrebbe cercare le informazioni a mano, e nulla di più. L'agente che invece, il quinto giorno lavorativo del mese, si attiva da solo perché è stata caricata l'ultima estrazione contabile, verifica le riconciliazioni, invoca un flusso deterministico per popolare il template e apre un'attività di approvazione al controller, non è un assistente: è un processo aziendale, e come tale richiede un committente, un responsabile, un disegno dei controlli, una procedura di continuità e una voce di costo. Chiamarli entrambi "agenti" è corretto sul piano commerciale e fuorviante su quello organizzativo, ed è precisamente questa ambiguità a produrre, nelle organizzazioni, il fenomeno per cui un processo critico finisce per dipendere da qualcosa che nessuno ha mai formalmente approvato.

5.6 MCP, connettori e agenti dichiarativi: Copilot Studio e Microsoft Foundry Agent Service

Microsoft ha aderito al MCP e lo supporta su più fronti della propria offerta, non come funzionalità isolata. Un agente dichiarativo può collegarsi direttamente a un server MCP esterno: la piattaforma ne scopre automaticamente gli strumenti disponibili e genera, senza intervento manuale, la porzione di manifesto necessaria a esporli come azioni dell'agente, aggiornando in automatico i file `manifest.json`, `ai-plugin.json` e `declarativeAgent.json` che compongono il pacchetto dell'agente, mentre l'autenticazione verso il server, quando richiesta, viene gestita tramite SSO e OAuth 2.0 attraverso il Microsoft 365 Agents Toolkit. Microsoft Foundry Agent Service (già Azure AI Foundry), il livello più orientato allo sviluppo pro-code dell'offerta, offre a sua volta supporto nativo per collegare un agente a server MCP come strumenti invocabili in fase di esecuzione. Nel nostro caso di esempio, lo stesso server MCP che abbiamo descritto nella parte comune di questo articolo, quello che espone in sola lettura i saldi del sistema gestionale, potrebbe essere collegato indifferentemente a un agente Copilot Studio o a un agente costruito con il Claude Agent SDK o con l'Agents SDK di OpenAI: è precisamente questo il punto pratico dell'interoperabilità promessa dal MCP.

5.7 Il Microsoft Agent Framework per lo sviluppo su misura

Per chi debba costruire un agente in modo programmatico sulla piattaforma Microsoft, il punto di riferimento unico è oggi il Microsoft Agent Framework, nato dalla fusione di due framework storici dell'azienda: Semantic Kernel, orientato all'integrazione enterprise e alla componibilità con i sistemi aziendali esistenti, e AutoGen, orientato alla conversazione e all'orchestrazione tra più agenti. È il livello concettualmente più vicino al Claude Agent SDK di Anthropic e all'Agents SDK di OpenAI descritti nelle parti precedenti, con una precisazione pratica che vale la pena riportare ovverosia che il pacchetto `autogen` disponibile su PyPI, il registro pubblico dei pacchetti Python, installa la versione community del progetto (nota come AG2), non il Microsoft Agent Framework, un equivoco di packaging facile da commettere e capace di far perdere tempo prezioso a un team che non ne fosse consapevole. Copilot Studio resta, in questo stesso ecosistema, lo strumento low-code per chi preferisce comporre un agente senza scrivere codice, mentre il Microsoft Agent Framework resta la scelta per chi ha bisogno del controllo più fine offerto da una libreria di programmazione.

5.8 GitHub Copilot Agent Mode: il fronte coding

Come Anthropic con Claude Code e OpenAI con Codex, anche Microsoft ha il proprio agente per lo sviluppo software maturo, integrato in GitHub Copilot attraverso l'Agent Mode: a differenza della modalità di editing tradizionale, che suggerisce modifiche puntuali riga per riga, l'Agent Mode può pianificare ed eseguire autonomamente un compito che coinvolge più file, eseguire i comandi necessari, e correggere i propri stessi errori attraverso un ciclo di autocorrezione (`autoFix`) che ripete il tentativo quando un test fallisce o il codice non compila, oltre a poter

invocare strumenti esposti tramite server MCP nello stesso modo già descritto per gli altri due ecosistemi. Nel nostro caso di riferimento, un team che lavori nell'ecosistema Microsoft userebbe l'Agent Mode esattamente per il tipo di intervento già descritto a proposito di Claude Code e di Codex, la correzione di uno script di consolidamento dati che alimenta il reporting mensile, con lo stesso criterio di verifica, i test superati e la correttezza dei dati prodotti, a decidere il valore reale dell'intervento, non il numero di righe modificate.

5.9 Copilot Cowork: una licenza tecnologica tra concorrenti diretti

A marzo 2026 Microsoft ha presentato "Copilot Cowork", la propria funzionalità per la gestione di compiti multi passo in Microsoft 365 Copilot, dichiarando esplicitamente di aver "lavorato a stretto contatto con Anthropic" per integrare "la tecnologia alla base di Claude Cowork" nel proprio prodotto. Lo interpreto come il segnale più concreto di quanto il mercato degli agenti si stia già ricomponendo lungo linee più pragmatiche delle rivalità dichiarate tra i singoli fornitori, e un argomento in più contro l'idea che scegliere un fornitore di modelli significhi necessariamente rinunciare per sempre alla tecnologia di un altro.

5.10 Frontier Firm e Work IQ: la cornice organizzativa

Microsoft colloca l'insieme di questi elementi dentro una cornice concettuale più ampia, che chiama "Frontier Firm": un'organizzazione in cui gli agenti diventano estensioni della forza lavoro umana, visibili e governabili con gli stessi strumenti già usati per le persone, non entità informatiche separate. La condizione abilitante di questa visione è "Work IQ", il livello che fornisce agli agenti il contesto organizzativo completo, file, riunioni, chat, relazioni, invece di frammenti isolati di informazione. Se dovessimo racchiudere in un'unica frase la differenza di approccio rispetto ai due ecosistemi già descritti, potremmo dire che mentre OpenAI e Anthropic ragionano prevalentemente in termini di capacità dell'agente, che cosa sa fare e con quale autonomia, Microsoft ragiona in termini di governo della popolazione di agenti come fosse una nuova categoria di risorsa aziendale, un'inclinazione naturale per chi parte da una posizione dominante nella gestione delle identità e degli asset aziendali, e che nel nostro caso di esempio si tradurrebbe, concretamente, nel poter rispondere in ogni momento, per l'intero gruppo, alla domanda "quali agenti operano oggi sul processo di chiusura mensile, con quali permessi, e chi ne è responsabile", una domanda a cui un registro centralizzato risponde in modo strutturalmente diverso da un elenco informale di script sparsi tra i vari team.

6 Parte VI. Lettura comparata e implicazioni nel processo di A(I)doption

6.1 Convergenze

Il primo dato che colpisce, mettendo a confronto i tre percorsi appena descritti, è quanto le architetture di fondo si somiglino tra Anthropic e OpenAI, nonostante le denominazioni diverse scelte da ciascun fornitore, e quanto Microsoft, pur condividendo lo stesso strato tecnico di base, ne ridisegni la priorità attorno al governo della flotta piuttosto che attorno alla sola capacità dell'agente. Tutti e tre condividono lo stesso protocollo di connessione ai dati esterni, il MCP, un fatto tutt'altro che scontato tra fornitori che restano concorrenti diretti su quasi ogni altro fronte. Anthropic e OpenAI, in particolare, hanno finito per offrire la stessa architettura a tre livelli per chi costruisce un agente su misura (accesso diretto al modello, framework o SDK intermedio, esecuzione interamente gestita dal fornitore), architettura che Microsoft replica a sua volta con Foundry Agent Service, Microsoft Agent Framework e Copilot Studio, sebbene incorniciata dentro un livello aggiuntivo di identità e governance che gli altri due non trattano come priorità di prodotto. Tutti e tre, infine, hanno sviluppato un concetto di competenza modulare e riutilizzabile (la skill in Anthropic e in OpenAI, le capacità dichiarate nel manifesto in Microsoft) e un meccanismo di configurazione dell'agente, in Markdown per i primi due, in JSON dichiarativo per il terzo. La convergenza sul nucleo tecnico riflette il fatto che i problemi da risolvere, per chiunque costruisca un sistema agentico serio, sono in larga parte gli stessi, a prescindere dal fornitore.

La tabella seguente riassume questa corrispondenza, esigenza per esigenza, come mappa di lettura più che come dichiarazione di equivalenza funzionale: prodotti collocati sulla stessa riga rispondono a un bisogno simile, non possiedono necessariamente gli stessi strumenti, gli stessi permessi o le stesse modalità di distribuzione, e la colonna Microsoft, in particolare, va letta tenendo presente che l'unità di governo di riferimento, per questo fornitore, è spesso la flotta di agenti nel suo insieme, non il singolo prodotto isolato.

Esigenza	Anthropic	OpenAI	Microsoft
Esperienza conversazionale	Claude Chat	ChatGPT "classic"	Microsoft 365 Copilot Chat
Lavoro delegato su elaborati	Cowork	ChatGPT Work	Copilot Cowork
Agente riutilizzabile per team	Plugin e connettori in Chat e Cowork	Workspace Agents	Agenti dichiarativi (Agent Builder, Copilot Studio)
Coding	Claude Code	Codex	GitHub Copilot Agent Mode (CLI e Code in Cowork)

Esigenza	Anthropic	OpenAI	Microsoft
Accesso diretto al modello	Messages API	Responses API	Foundry Agent Service (API)
Costruzione tramite libreria/SDK	Claude Agent SDK	Agents SDK (Codex SDK per integrare Codex)	Microsoft Agent Framework
Riutilizzo e diffusione di un metodo	Skill e plugin	Skill, plugin e Workspace Agents	Manifesto dichiarativo e catalogo Copilot
Esecuzione interamente gestita	Claude Managed Agents	Agents API	Foundry Agent Service (gestito)
Governo dell'identità e della flotta di agenti	Permessi e controlli per singolo agente	Permessi e controlli per singolo agente	Entra Agent ID e Agent 365 (control plane dedicato)

Vale la pena di soffermarsi su due righe in particolare. La prima è quella dedicata all'agente riutilizzabile per team, perché nasconde una differenza reale sotto un'apparente corrispondenza: un plugin, un Workspace Agent e un agente dichiarativo Microsoft non sono lo stesso tipo di oggetto. Il primo distribuisce estensioni (skill, connettori) che un utente richiama all'interno di una conversazione o di un ambiente come Cowork; il secondo rappresenta, in OpenAI, un modo diverso di organizzare lavoro riutilizzabile, con un proprio ciclo di vita, propri trigger esterni e una propria modalità di pubblicazione a livello di team; il terzo è, per costruzione, un'applicazione pubblicata in un catalogo aziendale, con un proprio processo di approvazione. Accostarli nella stessa riga aiuta a orientarsi per esigenza, non autorizza a trattarli come lo stesso prodotto con un nome diverso. La seconda riga da sottolineare è l'ultima, quella sul governo dell'identità e della flotta: è la riga in cui la differenza tra Microsoft e i primi due fornitori è di sostanza.

6.2 Divergenze di filosofia

Le differenze, dove esistono, vanno lette su due piani distinti. Tra Anthropic e OpenAI, sono più di forma che di sostanza perché l'approccio Anthropic appare più centrato sullo sviluppatore, con un'estensione progressiva verso l'utente non tecnico che parte da Claude Code e arriva, attraverso Cowork, a un pubblico generalista, mantenendo lungo tutto il percorso una forte coerenza architetturale tra i vari livelli di prodotto. L'approccio OpenAI appare più centrato sulla piattaforma e sulla componibilità visuale per team di business fin dall'inizio, con AgentKit come tentativo esplicito di offrire uno strumento no-code, salvo poi, come abbiamo visto, riposizionare parte di quella stessa offerta verso strumenti più strutturati. Rispetto a Microsoft, invece, la differenza è di sostanza, non di accento: Anthropic e OpenAI chiedono entrambi, in ultima istanza, "che cosa deve poter fare questo agente", mentre Microsoft chiede prima di tutto "chi è questo agente, e come si inserisce nell'anagrafica delle risorse dell'organizzazione".

6.3 Che cosa è davvero portabile tra le piattaforme

Uno standard condiviso come il MCP, e un formato aperto come quello delle skill, riducono certamente parte del lavoro di integrazione necessario per passare da un ecosistema all'altro, ma non lo eliminano. Una skill che codifica il metodo di reporting del nostro caso di riferimento può in linea di principio essere trasferita da Anthropic a OpenAI, o viceversa, conservandone il metodo e la struttura, ma non senza riverificare, nel nuovo ambiente, connettori, permessi e criteri di verifica: la portabilità del formato non è portabilità del comportamento effettivo. Verso Microsoft, il salto è più netto: lo stesso metodo andrebbe ritrascritto nel manifesto dichiarativo, il che non significa perdere il valore intellettuale del lavoro già svolto (il testo delle istruzioni, la soglia di materialità, la struttura del template restano riutilizzabili), ma richiede un passaggio di trasposizione che con Markdown puro non sarebbe necessario. È una distinzione da tenere bene a mente in ogni discussione, sempre più frequente nelle organizzazioni, sul rischio di vendor lock-in perché lo standard aperto riduce il rischio sul piano del formato, non lo azzera sul piano operativo, ed è tanto più vero quanto più un fornitore, come nel caso di Microsoft, integra la configurazione dell'agente dentro un proprio ciclo di pubblicazione e governance.

6.4 Chi costruisce la delega: direzione, laboratorio e diffusione

Rimane la domanda su chi debba prendersi la responsabilità di fare questo lavoro per non far arenare l'A(I)doption. I comportamenti osservati sono di solito i seguenti: la direzione stabilisce l'indirizzo, la legittimazione, l'investimento e i limiti, cioè quali processi possano essere resi agentici e con quale ambizione; un LA(I)B esplora i casi fattibili ma a buon potenziale, sfrutta (o nei casi estremi costruisce) gli harness, le connessioni e i controlli, e definisce gli standard; gli agenti "organizzativi" di cambiamento individuano le micro attività delegabili, le eccezioni che nessun documento descrive e la conoscenza tacita che non è mai stata scritta. Nessuno dei tre elementi basta da solo, e soprattutto il ciclo non è lineare: i casi d'uso che emergono dalle discovery devono alimentare il laboratorio, le regole costruite dal laboratorio devono diventare pratiche effettivamente adottabili, e le decisioni di indirizzo devono essere riviste sulla base dell'evidenza raccolta, non confermate per coerenza. Purtroppo non è possibile trattare l'adozione "agentica" come un progetto informatico isolato: poiché agisce su processi, dati, decisioni e ruoli, richiede al tavolo il titolare del processo, la funzione di rischio e conformità, la sicurezza, l'architettura dei dati e chi quel lavoro lo svolge davvero. L'obiettivo, del resto, non è (solo) costruire un catalogo di automatismi, ma acquisire una capacità organizzativa, quella di progettare deleghe artificiali verificabili, che è cosa diversa e molto più difficile da imitare.

6.5 Un percorso di adozione a gradini

Le organizzazioni che hanno diffuso questi strumenti con risultati duraturi riferiscono, con notevole concordanza, una progressione a gradini, ciascuno dei quali costruisce la fiducia e il sapere istituzionale che rendono produttivo il successivo. Si comincia con le domande, usando l'agente come motore di ricerca sul proprio patrimonio (spiegami come si calcola questo indicatore,

dove vive la soglia, quali conti confluiscono in questa voce): il rischio è nullo, perché l'agente legge e non scrive, e i due effetti collaterali sono preziosi, perché le persone imparano a dialogare con uno strumento agentico e l'organizzazione scopre quanto sia leggibile il proprio patrimonio (se l'agente non riesce a orientarsi, è indicatore utile di inaccessibilità del patrimonio informativo, non sullo strumento). Si passa a interventi piccoli e facilmente verificabili (una nota di commento, una riconciliazione ripetitiva, un controllo di formato). Si introduce poi la pianificazione, che separa «ha capito il compito?» da «sa eseguirlo?», dato che la maggior parte dei fallimenti di esecuzione nasce da un equivoco sul compito e il piano lo intercetta prima. Solo dopo si concede autonomia, ai passaggi che l'esperienza ha mostrato affidabili. Per ciascuna persona il percorso richiede alcune settimane, e chi salta i gradini, passando dall'installazione alla piena autonomia, produce i risultati disomogenei che spingono molte organizzazioni a rinunciare. Per i gruppi, la scansione plausibile dopo l'avvio è questa: nei primi mesi si presidia la manutenzione (correzioni, piccole evoluzioni, e il file di istruzioni diventa la fonte di verità), successivamente ci si misura sulle funzionalità nuove e si standardizzano le procedure in skill solo dopo si consolida e si irrobustisce, affinchè ogni nuovo "adopter" successivo alla fase di impostazione sia produttivo in giorni invece che in settimane.

Due pratiche accelerano il percorso. La prima è creare, prima di concedere l'accesso ai sistemi veri, un confine in cui l'agente possa lavorare senza conseguenze (ambienti con dati sintetici, oppure con dati storici già chiusi, di cui si conosce la risposta giusta) e allargare il perimetro man mano che il comportamento si dimostra affidabile: nel nostro caso, fargli rifare la chiusura di un mese passato, di cui esiste il pacchetto approvato, è la prova più economica di quanta fiducia meriti, ed è una variante della mappa di affidabilità compito per compito di cui si è detto. La seconda sono le sessioni in cui le funzioni mostrano l'una all'altra come lavorano, perché una dimostrazione fra colleghi persuade più di qualunque documentazione e fa emergere usi che un'altra funzione non aveva considerato (la procedura di revisione di un reparto, adattabile a un altro). Resta un aspetto da non perdere. Le funzioni non tecniche non ottengono da questi strumenti soltanto le stesse cose più velocemente: ottengono cose che prima non erano possibili senza il supporto dei sistemi informativi, come un controller che costruisce da sé un cruscotto, un ufficio legale che analizza i contratti in modo programmatico o una funzione commerciale che interroga i dati di campagna. Misurare l'adozione soltanto in termini di velocità dei tecnici significa perdere metà del valore, ed è per questo che, accanto alle misure della scheda proposta in precedenza, conviene tenere traccia anche delle capacità nuove che emergono, compresi i casi, sempre più frequenti, in cui un'attività che non sarebbe mai stata considerata come costo di tipo informatico da un momento all'altro diventa possibile e viene semplicemente fatta.

6.6 Il quadro economico: abbonamenti, token, crediti e ore di esecuzione

Per quanto soggetto a prevedibili evoluzioni nel breve periodo (anche perché prima o poi gli editori di questi modelli passeranno "alla cassa"), credo sia importante quanto meno provare ad introdurre il tema dei diversi modelli di prezzo perché determina che cosa si può mettere a

budget in anticipo, che cosa può sfuggire di mano, e soprattutto quale unità di misura entra nel controllo di gestione dell'iniziativa. Il problema, per chi deve costruire un business case, è che oggi convivono almeno cinque logiche diverse, spesso nello stesso fornitore e talvolta persino nello stesso prodotto.

La prima logica è l'**abbonamento per utente**: la più familiare, la più prevedibile, e quella su cui si fonda ancora la maggior parte delle offerte rivolte all'utente finale. La seconda è il **consumo a token**, cioè la quantità di testo elaborata in ingresso e in uscita, che è la logica nativa delle API e quella che si applica quando l'organizzazione costruisce qualcosa di proprio. La terza è il **credito prepagato**, un'unità di conto proprietaria che il fornitore vende a pacchetti e consuma secondo tariffari propri, e che introduce un livello di astrazione fra ciò che si paga e ciò che si consuma. La quarta, più recente e nata proprio con gli agenti, è il **tempo di esecuzione dell'infrastruttura**: quando un agente lavora per ore in modo asincrono, il costo dell'ambiente isolato in cui gira non è esprimibile in token e viene quindi fatturato a tempo. La quinta, infine, è la **tariffa fissa per azione**, in cui un compito (una ricerca approfondita, un turno in modalità agente) vale un numero predefinito di unità a prescindere dal lavoro effettivamente svolto sotto.

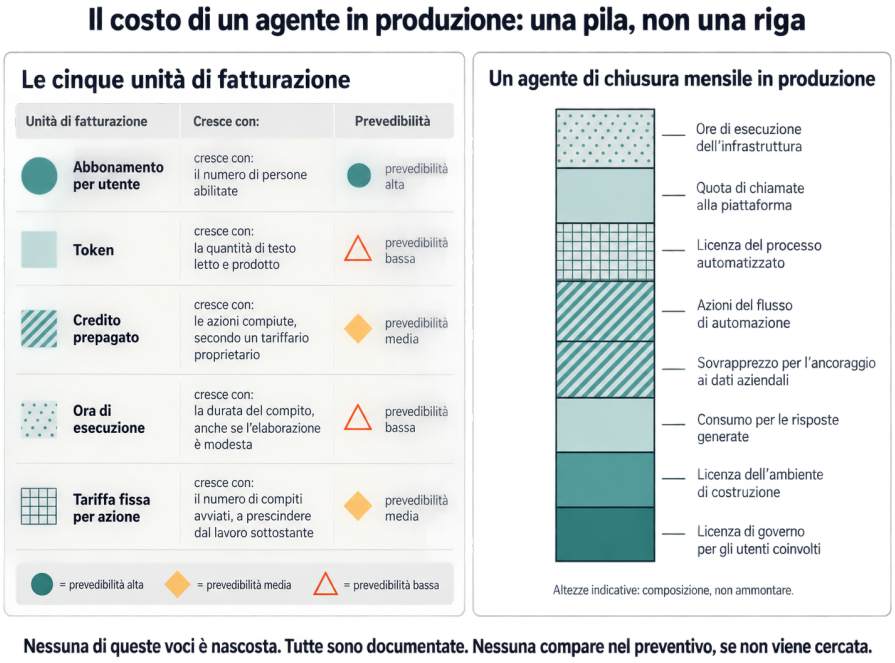


Figura 12: Il costo di un agente in produzione: una pila, non una riga

Il quadro che segue fotografa la situazione a fine settembre 2026 e descrive le logiche di costo e non i listini sia perché cambiano troppo rapidamente per essere attendibili sia perché come tutto nella vita possono essere oggetto di negoziazione.

Fornitore	Componente	Unità di fatturazione	Prevedibilità
Anthropic	Piani Pro, Max, Team	Abbonamento per utente, con limiti d'uso a finestre temporali	Alta

Fornitore	Componente	Unità di fatturazione	Prevedibilità
Anthropic	Piano Enterprise	Abbonamento per utente **più** consumo a token a tariffa API	Media
Anthropic	Messages API	Token di ingresso, di uscita e di cache	Bassa
Anthropic	Managed Agents	Token **più** ore di sessione dell'infrastruttura	Bassa
OpenAI	Piani ChatGPT e Business	Abbonamento per utente, con posti standard e posti premium	Alta
OpenAI	Uso oltre soglia, Codex, Workspace Agents	Crediti prepagati, a tariffa per azione oppure per token	Bassa
OpenAI	API e Agents API	Token, più tariffe per chiamata di alcuni strumenti integrati	Bassa
Microsoft	Microsoft 365 Copilot	Abbonamento per utente	Alta
Microsoft	Agent 365	Abbonamento **per utente umano**, non per agente	Alta
Microsoft	Copilot Studio	Crediti prepagati a pacchetto, non cumulabili fra mesi, oppure consumo su Azure	Media
Microsoft	Foundry Agent Service	Token, più ore di calcolo, più memoria e archiviazione	Bassa
Microsoft	GitHub Copilot	Abbonamento per posto più crediti consumati a token	Media
Microsoft	Power Automate (flussi invocati)	Licenza per utente oppure per singolo processo automatizzato	Alta per unità

Da questo quadro emergono cinque considerazioni che mi paiono più utili delle cifre puntuali, destinate comunque a invecchiare in fretta (e considerando il ritmo e la frequenza di riscrittura di questo testo fino a qui, dalla prima stesura di qualche settimana fa, mi sento male). La prima è che **il workplace non compra più il lavoro**. Il posto compra l'accesso, la sicurezza e la governance; l'attività vera e propria viene misurata a parte. Anthropic lo dichiara nel modo più esplicito per il proprio piano Enterprise, dove il canone per utente convive con la fatturazione separata di ogni token consumato a tariffa API; OpenAI ha introdotto posti dedicati allo sviluppo fatturati interamente a consumo; Microsoft, con Copilot Studio, azzera il costo di alcune azioni per chi possiede la licenza Copilot ma ne fattura altre per intero, in particolare quelle esposte all'esterno delle proprie superfici. Per chi costruisce un budget, la conseguenza è che la voce "licenze" ha smesso di essere una ragionevole approssimazione del costo dell'iniziativa, e chi continua a usarla come tale otterrà un preventivo sistematicamente ottimista.

La seconda riguarda **la trasparenza dell'unità di misura**, intesa come assenza della medesima. Quando l'eccedenza è fatturata in valuta a tariffe pubblicate, il committente può ricostruire il costo unitario di un'elaborazione e confrontare due soluzioni; quando è fatturata in crediti, unità proprietaria il cui controvalore non sempre è pubblicato e che in alcuni casi scade dopo dodici mesi, la stessa ricostruzione richiede un tariffario che non viene necessariamente fornito in modo spontaneo. Non è una questione di correttezza del fornitore, è una ordinaria questione di asimmetria informativa: chiedere per iscritto, prima della firma, il tasso di conversione fra crediti e valuta, la politica di scadenza e il trattamento dell'inutilizzato dovrebbe essere prassi standard, esattamente come si fa da sempre per qualunque altra unità di consumo contrattuale.

La terza è che **gli agenti hanno costretto il mercato a inventare un'unità nuova**. Sia Anthropic sia OpenAI, in modo indipendente e a pochi mesi di distanza, hanno introdotto una fatturazione a tempo per gli ambienti isolati in cui gli agenti eseguono codice, semplicemente perché il consumo di un contenitore che resta acceso per ore non è esprimibile in token. È un dettaglio tecnico con un'implicazione gestionale precisa: un agente che lavora a lungo costa anche quando "pensa" poco, e la metrica di efficienza rilevante non è più il numero di parole elaborate ma la durata del compito e il grado di occupazione della risorsa. Chi ha esperienza di controllo di gestione riconoscerà senza fatica la differenza fra un costo variabile proporzionale al volume e un costo di capacità proporzionale al tempo di impegno di un fattore produttivo: è la stessa distinzione di sempre, riproposta in un contesto che sembrerebbe nuovo e non lo è.

La quarta, e la più insidiosa in fase di preventivazione, è che **il costo di un agente in produzione è una pila, non una riga**. Un agente costruito in Copilot Studio che si attiva su un evento, interroga dati aziendali e invoca un flusso di automazione può consumare, contemporaneamente, i crediti del pacchetto prepagato, un costo maggiorato per l'ancoraggio ai dati aziendali (nell'ordine di alcune volte il costo di una risposta generativa semplice), le azioni del flusso, una licenza dedicata per il processo automatizzato, una quota delle chiamate disponibili alla piattaforma, e infine la licenza di governo per gli utenti coinvolti. Nessuna di queste voci è nascosta, tutte sono documentate, ma nessuna compare nel preventivo se non viene cercata. Il preventivo corretto si costruisce quindi **per processo e non per strumento**, ed è forse il punto in cui la competenza di chi si occupa di controllo di gestione è più direttamente applicabile a una materia che a prima vista sembrerebbe di esclusiva pertinenza informatica.

La quinta, per chiudere, è che **le leve di costo stanno nel disegno più che nel listino**. Quattro meritano di essere note a chi costruisce un preventivo. La prima è la memorizzazione del contesto già inviato (*prompt caching*): se l'inizio di una richiesta, cioè le istruzioni di sistema, le istruzioni persistenti e le dichiarazioni degli strumenti, non cambia da un giro all'altro, lo si paga per intero una volta e a tariffa ridotta nei giri successivi, il che rende economicamente sostenibile un contesto ricco in una sessione lunga e penalizza invece il rimescolare di continuo le istruzioni o il ricominciare da capo a ogni richiesta. La seconda è la scelta del modello per ruolo: la pianificazione e la valutazione del risultato, dove la qualità del ragionamento rende di più, si affidano al modello più capace, mentre l'esecuzione e l'esplorazione, che sono la grande maggioranza del consumo, si affidano a modelli più leggeri. Poiché l'orchestrazione pesa in genere

una frazione piccola del totale, se l'esecuzione costa per token un quinto il risparmio sul complesso può avvicinarsi all'ottanta per cento (cifre stimate ovviamente sulla base dell'esperienza, da riverificare ogni volta sul processo) senza perdere qualità dove conta. La terza è lo sforzo di ragionamento, che in alcuni prodotti è una manopola di configurazione (livelli basso, medio, alto) e non un effetto delle parole usate nel prompt: chiedere all'agente di «pensare di più» non lo fa pensare di più, e l'investimento in ragionamento va proporzionato al compito (poco per rinominare un file, molto per una riclassificazione che alimenta il consolidato). La quarta (che oramai ho nausea a ripetere tante volte l'ho detta in questo papello) è il costo dell'errore di direzione: un piano costa poche migliaia di token, un'esecuzione partita nel verso sbagliato ne costa centinaia di migliaia, e l'esecuzione parallela di più agenti senza piano moltiplica il danno invece di dividere il tempo.

A queste si aggiunge un effetto che spesso nei ROI calcolasi solo in chiave di efficienza sul tempo risparmiato non compare. Una quota rilevante delle attività svolte con questi strumenti è fatta di cose che, senza di essi, non si sarebbero fatte affatto: fare cose in più ma *diverse* significa, in un qualche modo, fare efficacia non efficienza. Misurare soltanto le ore risparmiate sottostima il valore, perché confronta «lo stesso lavoro in meno tempo» invece di «più lavoro o anche diverso lavoro nello stesso tempo».

Vale la pena, per chiudere, collegare questo quadro alla scheda di misurazione proposta nella parte comune. Il costo per risultato accettato, che è l'unica metrica capace di integrare prezzo e qualità, si calcola soltanto se si conoscono entrambi i termini: il denominatore richiede la disciplina della verifica, il numeratore richiede di sapere quali unità si stanno consumando e in quale punto del processo. Un'organizzazione che misura l'uno senza l'altro finirà inevitabilmente per confrontare soluzioni sulla base del prezzo di listino del modello, che è l'unica cifra facile da reperire e la meno significativa di tutte. Aggiungo, per completezza, che tutti e tre i fornitori hanno introdotto nel corso del 2026 strumenti di controllo della spesa piuttosto seri (limiti per singolo utente, tetti per singolo agente con disattivazione automatica al superamento, notifiche di soglia, attribuzione dei costi per progetto): esistono, funzionano, e nella mia esperienza vengono configurati quasi sempre il giorno dopo il primo conto inatteso, anziché il giorno prima.

7 Parte VII. Conclusioni: la nuova competenza manageriale

Ogni volta che un'organizzazione delega un compito a un esecutore di cui non osserva direttamente ogni singola azione, umano o artificiale che sia, genera un problema di agenzia, nel senso classico che questo termine ha nella teoria economica dell'organizzazione: un'asimmetria informativa tra chi delega e chi esegue, che va gestita con strumenti di controllo appropriati, non ignorata nell'entusiasmo per la nuova capacità tecnica. Il risultato di ricerca sui problemi di coordinamento multi agente richiamato in precedenza è un promemoria utile in questo senso: più agenti, e più autonomia, non equivalgono automaticamente a più affidabilità, e la disciplina della verifica, non l'entusiasmo per la scala, resta il vero indicatore di maturità di un'adozione.

Tre rischi, non tecnici, meritano di non essere sottovalutati. Il primo è l'eccesso di fiducia (di cui abbiamo parlato lungamente altrove): l'esito prodotto è oramai quasi sempre sorprendente, il processo resta in larga parte opaco, e il fascino del risultato prende il posto della verifica, cosicché un esito convincente viene scambiato per un'evidenza e un'evidenza per una decisione. Il secondo è l'impoverimento di chi delega: se l'agente compie intere sequenze, chi dovrebbe controllarle perde progressivamente la comprensione del processo, e con essa la capacità stessa di accorgersi di un errore, il che è particolarmente grave nelle attività formative e in quelle a più alta responsabilità. Il terzo, e il più frequente nella mia esperienza, è l'automazione del "casino", perché un agente che rende più veloce un processo mal disegnato amplifica i dati di scarsa qualità, fino a degradare la qualità complessiva del patrimonio informativo aziendale. Serve contrapporre qualità umane e manageriali irriducibili che assumano la forma di competenze operative: la conoscenza **profonda**, che consente di riconoscere l'errore e l'anomalia che il sistema non dichiara; la conoscenza **ampia**, che permette di attivare metodi e analogie di discipline diverse, perché i modelli contengono molte più possibilità di quante ne propongano spontaneamente; il **gusto**, che diventa decisivo quando produrre varianti costa poco e il valore si sposta sul selezionare, combinare e scartare; e l'**iniziativa**, cioè il coraggio di esplorare ciò che nessun manuale descrive ancora. La conseguenza per la formazione aziendale è diretta: insegnare a scrivere prompt e contesti è fondamentale, ma occorre sempre più insegnare a delegare, a controllare, a valutare e a sperimentare.

Una conseguenza ulteriore riguarda dove si colloca il collo di bottiglia. Quando produrre diventa economico, il vincolo si sposta dalla capacità di produzione alla capacità di decisione, cioè a quanto chiaramente si sappia definire ciò che si vuole. Le competenze che restano care, e che per questo aumentano di valore, sono tre: l'architettura, ossia decidere che cosa costruire e con quali compromessi; il coordinamento, ossia la gestione delle dipendenze fra persone, funzioni e sistemi, che nessuna sessione di lavoro di un agente risolve; e la valutazione della qualità, ossia sapere se un esito è abbastanza buono, se i controlli coprono gli scenari giusti, se la soluzione reggerà. Chi eccelle in questo contesto non è chi produce più in fretta, è chi pensa più chiaramente, e sono qualità sempre apprezzate che si apprestano a diventare la descrizione principale del lavoro. Se dovessi indicare la competenza manageriale più rilevante che emerge da questo intero percorso, non sarebbe una competenza tecnica in senso stretto, ma la capacità di progettare lavoro delegabile, nello stesso senso in cui l'adozione dell'AI in azienda richiede che competenze, procedure e uso quotidiano procedano insieme: formulare obiettivi verificabili invece di istruzioni vaghe, esplicitare il sapere operativo che prima viveva solo nell'esperienza di chi svolgeva un compito, predisporre le informazioni necessarie nel formato e nella qualità richiesti, assegnare l'autorità appropriata a ciascun passaggio, e infine giudicare gli esiti con gli stessi criteri di rigore che applicheremmo a qualunque collaboratore umano. Nel nostro caso di riferimento, il responsabile del controllo di gestione che mantiene la proprietà del metodo di reporting, lo aggiorna quando cambia una politica aziendale, e usa i risultati della verifica per decidere cosa delegare ulteriormente e cosa no, non sta solo supervisionando uno strumento: sta esercitando, in una forma nuova, la stessa responsabilità manageriale che ha sempre esercitato.